

```

    points: renderPoints,
    color: live.color,
    strokeWidth: live.strokeWidth,
    strokeStyle: live.strokeStyle,
    smoothing: live.smoothing,
    createdAt: live.createdAt,
    updatedAt: Date.now(),
  };

  const pathD = buildStrokePath(tempStroke, true);
  pathEl.setAttribute("d", pathD);
}, []];

const flushDraftStroke = useCallback(() => {
  draftStrokeRafRef.current = null;
  applyLiveDraftStroke();
}, [applyLiveDraftStroke]);

const scheduleDraftFlush = useCallback(() => {
  if (draftStrokeRafRef.current === null) {
    draftStrokeRafRef.current = requestAnimationFrame(flushDraftStroke);
  }
}, [flushDraftStroke]);

// A render triggered by something else mid-gesture would paint the stale
// store camera and make the page jump. Re-assert ours after every commit.
useLayoutEffect(() => {
  if (liveCamRef.current) applyLiveTransform();
  if (liveDraftStrokeRef.current && liveDraftGroupRef.current && liveDraftPathRef.current) {
    liveDraftGroupRef.current.style.display = "";
    liveDraftGroupRef.current.setAttribute(
      "transform",
      `translate(${liveDraftStrokeRef.current.originX}, ${liveDraftStrokeRef.current.originY})`
    );
    applyLiveDraftStroke();
  }
});

const flushPan = useCallback(() => {
  panRafRef.current = null;
  const { dx, dy } = panAccumRef.current;
  if (dx === 0 && dy === 0) return;
  panAccumRef.current = { dx: 0, dy: 0 };

  const live = liveCamRef.current;
  if (!live) {
    // No gesture in flight (a stray flush): go through the store as normal.
    panCamera(dx, dy);
    return;
  }
  const z = cameraRef.current.zoom;
  const fromX = live.x;
  const fromY = live.y;
  live.x -= dx / z;
  live.y -= dy / z;
  // Stop the paper at the edge under the finger. The store's own bound only
  // applies on release, so without this a drag could sweep the page off the
  // screen and let it snap back afterwards - the rubber band nobody asked
  // for. Clamping here means the page simply stops, like every other reader.
  const geom = flingGeomRef.current;
  if (geom.isNb) {
    const c = clampCameraToSheet(
      { x: live.x, y: live.y, zoom: z },
      viewportRef.current,
      geom.page,
      geom.doc
    );
    live.x = c.x;
    live.y = c.y;
  }
  // Bank the movement that actually happened, not the movement asked for, so
  // the single store update on release lands where the screen already is.
  panTotalRef.current.dx += (fromX - live.x) * z;
  panTotalRef.current.dy += (fromY - live.y) * z;
  applyLiveTransform();
}, [panCamera, applyLiveTransform]);

/** When the pending flush was scheduled, so a stalled one can be detected. */
const panScheduledAtRef = useRef(0);

```

```

/**
 * Records movement and makes sure a flush happens.
 *
 * The obvious version – "schedule an rAF only if none is pending" – WEDGES.
 * A WebView stops firing requestAnimationFrame whenever it decides the page
 * is not visible (app backgrounded, screen off, system gesture takeover).
 * A flush scheduled just before that never runs, so the pending-flag stays
 * set for ever, every later move refuses to schedule anything, and panning
 * silently stops working until the app is restarted. That is exactly how
 * this froze on the Galaxy Tab S9.
 *
 * So the flag is treated as a hint, not a lock: if the pending flush has not
 * run within a few frames, it is assumed dead and the movement is applied
 * straight away.
 */
const queuePan = useCallback(
  (dx: number, dy: number) => {
    panAccumRef.current.dx += dx;
    panAccumRef.current.dy += dy;

    const now = performance.now();
    const pending = panRafRef.current !== null;
    const stalled = pending && now - panScheduledAtRef.current > 80; // ~5 frames

    if (stalled) {
      cancelAnimationFrame(panRafRef.current!);
      panRafRef.current = null;
      flushPan();
      return;
    }
    if (!pending) {
      panScheduledAtRef.current = now;
      panRafRef.current = requestAnimationFrame(flushPan);
    }
  },
  [flushPan]
);

/** Recent pointer positions for the velocity estimate (last 100 ms only). */
const panSamplesRef = useRef<Array<{ t: number; x: number; y: number }>>([]);
/** Latest sheet geometry, read by the fling loop without a stale closure. */
const flingGeomRef = useRef({
  isNb: false,
  page: { width: 0, height: 0 },
  doc: { pageCount: 1, activeIndex: 0 },
});
// Re-assigned every render, so a fling that crosses a page boundary sees the
// new anchor rather than the one captured when the finger left the glass.
flingGeomRef.current = {
  isNb,
  page: pageDim,
  doc: { pageCount: allPages.length, activeIndex },
};

const recordPanSample = useCallback((x: number, y: number) => {
  const t = performance.now();
  const buf = panSamplesRef.current;
  buf.push({ t, x, y });
  while (buf.length > 2 && t - buf[0].t > VELOCITY_HORIZON_MS) buf.shift();
}, []);

const stopMomentum = useCallback(() => {
  if (momentumRafRef.current !== null) {
    cancelAnimationFrame(momentumRafRef.current);
    momentumRafRef.current = null;
  }
}, []);

/**
 * Coasts after a throw. Velocity is measured Android's way - only the last
 * 100 ms, and zero if the finger had been still for 40 ms before lifting.
 */
const startMomentum = useCallback(() => {
  const now = performance.now();
  const buf = panSamplesRef.current.filter((q) => now - q.t <= VELOCITY_HORIZON_MS);
  panSamplesRef.current = [];
  if (buf.length < 2) return;

```

```

const first = buf[0];
const last = buf[buf.length - 1];
if (now - last.t > POINTER_STOPPED_MS) return; // finger had stopped: no throw
const span = last.t - first.t;
if (span <= 0) return;

let vx = (last.x - first.x) / span;
let vy = (last.y - first.y) / span;
const speed = Math.hypot(vx, vy);
if (speed < FLING_MIN_V) return;
if (speed > FLING_MAX_V) {
  vx *= FLING_MAX_V / speed;
  vy *= FLING_MAX_V / speed;
}

const lnDecel = Math.log(FLING_DECEL_PER_MS);
let prev = performance.now();
const step = () => {
  const t = performance.now();
  const dt = Math.min(50, t - prev); // a stalled frame must not teleport
  prev = t;
  const decay = Math.pow(FLING_DECEL_PER_MS, dt);
  // Exact distance under exponential decay over dt, not v * dt.
  const k = (decay - 1) / lnDecel;
  const dx = vx * k;
  const dy = vy * k;
  vx *= decay;
  vy *= decay;

  const store = useWhiteboard.getState();
  const before = store.camera;
  store.panCamera(dx, dy);
  let after = useWhiteboard.getState().camera;

  // Reached the end of the notebook: stop exactly there instead of pushing.
  const geom = flingGeomRef.current;
  let clamped = false;
  if (geom.isNb) {
    const c = clampCameraToSheet(after, viewportRef.current, geom.page, geom.doc);
    const bitX = Math.abs(c.x - after.x) > 1e-6;
    const bitY = Math.abs(c.y - after.y) > 1e-6;
    if (bitX || bitY) {
      store.setCamera(c);
      after = useWhiteboard.getState().camera;
      // Kill ONLY the axis that hit the wall. Ending the whole fling on any
      // clamp looked right until the sheet became narrower than the screen:
      // x is then pinned to the centre, so it is "at the wall" on every
      // frame, and since no finger swipes perfectly straight, every scroll
      // lost its momentum on the first frame and the notebook felt dead.
      if (bitX) vx = 0;
      if (bitY) vy = 0;
      clamped = true;
    }
  }
  const moved = Math.hypot(after.x - before.x, after.y - before.y) * after.zoom;
  const wanted = Math.hypot(dx, dy);
  if (
    Math.hypot(vx, vy) < FLING_STOP_V ||
    // The store refused to move: something else is holding the camera.
    // Only meaningful on a frame that was not clamped, because a clamped
    // frame is SUPPOSED to move less than asked.
    (!clamped && wanted > 0.5 && moved < wanted * 0.5)
  ) {
    momentumRafRef.current = null;
    return;
  }
  momentumRafRef.current = requestAnimationFrame(step);
};
momentumRafRef.current = requestAnimationFrame(step);
}, []);

/** Begins a pan. Keeps `interaction` in step for the cursor and other branches. */
const beginPan = useCallback((x: number, y: number) => {
  stopMomentum();
  panSamplesRef.current = [{ t: performance.now(), x, y }];
  panLastRef.current = { x, y };
  panAccumRef.current = { dx: 0, dy: 0 };
  panTotalRef.current = { dx: 0, dy: 0 };

```

```

    liveCamRef.current = { x: cameraRef.current.x, y: cameraRef.current.y };
    setInteraction({ kind: "panning", lastX: x, lastY: y });
  }, [stopMomentum]);

  /** Nothing may outlive the component. */
  useEffect(
    () => () => {
      if (panRafRef.current !== null) cancelAnimationFrame(panRafRef.current);
      if (momentumRafRef.current !== null) cancelAnimationFrame(momentumRafRef.current);
    },
    []
  );

  /** Flushes and clears pan state however the gesture ended. */
  const endPanGesture = useCallback(() => {
    if (panRafRef.current !== null) {
      cancelAnimationFrame(panRafRef.current);
      panRafRef.current = null;
    }
    flushPan();
    panLastRef.current = null;

    // Hand the whole gesture to the store as ONE change, then let React paint
    // it. liveCamRef is cleared first so the layout effect stops re-asserting.
    const total = panTotalRef.current;
    liveCamRef.current = null;
    panTotalRef.current = { dx: 0, dy: 0 };
    if (total.dx !== 0 || total.dy !== 0) panCamera(total.dx, total.dy);
  }, [flushPan, panCamera]);

  // -----
  // Eraser cursor: a circle showing exactly what the eraser will take
  // -----
  // Like GoodNotes and Samsung Notes: while the eraser is the tool, a ring
  // follows the pen - hovering as well as touching - so you can see what will
  // go before any ink is lost. Its radius is the SAME value eraseAt and
  // findElementAt test against (eraserSize / 2, in page units), scaled by the
  // zoom, so the ring is the eraser rather than an approximation of it. That
  // holds in both modes: whole-stroke picking uses the eraser radius as its
  // hit tolerance too.
  //
  // It is moved by writing its transform straight onto the DOM node. Hover
  // reports arrive at the digitiser's rate; a React render for each would put
  // the ring visibly behind the nib, the same mistake that made scrolling lag.
  const eraserCursorRef = useRef<SVGElement | null>(null);
  const eraserCursorShownRef = useRef(false);
  const eraserOn = tool === "eraser";
  const eraserScreenR = Math.max((style.eraserSize || 24) / 2) * camera.zoom, 2);

  const placeEraserCursor = useCallback(
    (clientX: number, clientY: number, show: boolean) => {
      const g = eraserCursorRef.current;
      if (!g) return;
      if (!show) {
        if (eraserCursorShownRef.current) {
          g.setAttribute("visibility", "hidden");
          eraserCursorShownRef.current = false;
        }
        return;
      }
      const pt = getContainerPoint(clientX, clientY);
      g.setAttribute("transform", `translate(${pt.x}, ${pt.y})`);
      if (!eraserCursorShownRef.current) {
        g.setAttribute("visibility", "visible");
        eraserCursorShownRef.current = true;
      }
    },
    [getContainerPoint]
  );

  // -----
  // S Pen & Stylus Hover Preview Cursor
  // -----
  // When an S Pen (or mouse) hovers above the canvas, show a dual-ring dot
  // previewing the exact ink thickness and color before touching down.
  const penHoverCursorRef = useRef<SVGElement | null>(null);
  const penHoverCursorShownRef = useRef(false);

```

```

const placePenHoverCursor = useCallback(
  (clientX: number, clientY: number, show: boolean) => {
    const g = penHoverCursorRef.current;
    if (!g) return;
    if (!show) {
      if (penHoverCursorShownRef.current) {
        g.setAttribute("visibility", "hidden");
        penHoverCursorShownRef.current = false;
      }
      return;
    }
    const pt = getContainerPoint(clientX, clientY);
    g.setAttribute("transform", `translate(${pt.x}, ${pt.y})`);
    if (!penHoverCursorShownRef.current) {
      g.setAttribute("visibility", "visible");
      penHoverCursorShownRef.current = true;
    }
  },
  [getContainerPoint]
);

// The dot is the width the pen will actually lay down at normal pressure:
// each pen scales its thickness (a highlighter 3x, a fineliner 0.5x), so the
// raw thickness setting alone showed the wrong size for most pens.
const hoverInkRadius =
  ((getToolSize(style, tool) || 4) *
    (isStrokeTool(tool) ? PEN_TYPE_PRESETS[tool as StrokeStyle]?.sizeScale ?? 1 : 1) *
    camera.zoom) /
  2;

// Switching TO the eraser (including the S Pen button borrowing it) shows
// the ring where the pen already is, in place of the ink dot; switching away
// hides the ring at once.
useLayoutEffect(() => {
  if (!eraserOn) {
    placeEraserCursor(0, 0, false);
    return;
  }
  placePenHoverCursor(0, 0, false);
  const lp = lastPointerPosRef.current;
  if (lp) placeEraserCursor(lp.clientX, lp.clientY, true);
}, [eraserOn, placeEraserCursor, placePenHoverCursor]);

// -----
// Draw-and-Hold Auto-Straighten State
// -----
const autoStraightenTimerRef = useRef<ReturnType<typeof setTimeout> | null>(null);
const isAutoStraightenedRef = useRef(false);

const clearAutoStraightenTimer = useCallback(() => {
  if (autoStraightenTimerRef.current) {
    clearTimeout(autoStraightenTimerRef.current);
    autoStraightenTimerRef.current = null;
  }
}, []);

const capturePointer = useCallback((e: React.PointerEvent<SVGSVGElement>) => {
  const node: Element | null = svgRef.current ?? (e.currentTarget as Element);
  try {
    node?.setPointerCapture?.(e.pointerId);
  } catch {
    // setPointerCapture throws NotFoundError when the pointer is no longer
    // active – a real race on touch devices if the finger lifts between the
    // event firing and this handler running. Unguarded it escaped
    // handlePointerDown and aborted the rest of the gesture setup (tool
    // state, draft element, history), which looks like "drawing randomly
    // stops working". Losing capture only costs us events outside the SVG.
  }
}, []);

/**
 * Arm or disarm the context menu. Written through a ref as well so the
 * decision is readable synchronously inside a pointer handler, where the
 * state value would still be the previous render's.
 */
const armContextMenu = useCallback((next: boolean) => {
  if (contextMenuArmedRef.current === next) return;
  contextMenuArmedRef.current = next;

```

```

    setContextMenuArmed(next);
  }, []);

/** Disarm a pending long press (the finger moved, lifted, or the pen won). */
const cancelLongPress = useCallback(() => {
  if (longPressRef.current) {
    clearTimeout(longPressRef.current.timer);
    longPressRef.current = null;
  }
}, []);

// A timer that outlives the canvas would fire setSelected on an unmounted
// component – and, worse, keep the board alive in the closure.
useEffect(() => cancelLongPress, [cancelLongPress]);

/**
 * Arm Android's standard long press to select. Holding still for 500ms
 * selects the object (and the drag that follows moves it); moving first
 * means the gesture was a pan and nothing is selected at all.
 *
 * This is the half of the fix that works on a tablet with NO stylus, where
 * none of the pen-proximity rules in palm.ts can ever fire: a hand brushing
 * the glass is brief and moves, so it fails the hold, and a hand that comes
 * to rest is filtered by contact size.
 */
const armLongPressSelect = (
  e: React.PointerEvent<SVGSVGElement>,
  hit: WhiteboardElement
) => {
  cancelLongPress();
  // This gesture is a SELECTION, so Radix's own 700ms long press must not
  // race it: two timers on one press produced a menu that pre-empted the
  // selection, and the user was left holding neither. The options menu is
  // still one gesture away – press and hold again once the object is
  // selected (see the already-selected branch below), which is the same
  // press-to-select-then-press-for-options rhythm Android uses.
  armContextMenu(false);
  const pointerId = e.pointerId;
  const startX = e.clientX;
  const startY = e.clientY;
  const priorSelection = selectedIds;
  const timer = setTimeout(() => {
    longPressRef.current = null;
    setSelected([hit.id]);
    touchInteractionRef.current = { pointerId, priorSelection };
    // Locked elements select but never move, so no drag is queued for them.
    if (!hit.locked) pendingMoveRef.current = { pointerId, elementId: hit.id };
    // Confirm the change the way the platform does: at this moment the user
    // is looking at their hand, not at the screen.
    if (touchSettings.touchFeedback) {
      try {
        navigator.vibrate?.(12);
      } catch {
        /* unsupported or blocked; the haptic is a courtesy, never required */
      }
    }
  }, LONG_PRESS_MS);
  longPressRef.current = { timer, pointerId, elementId: hit.id, startX, startY };
};

/**
 * Retroactive palm rejection. The hand lands on the glass BEFORE the nib
 * does, so by the time the stylus proves that contact was a palm it has
 * already selected – or started dragging – whatever it came down on. Undo
 * that, rather than merely declining to carry it further.
 */
const retractTouchInteraction = () => {
  cancelLongPress();
  pendingMoveRef.current = null;
  const started = touchInteractionRef.current;
  if (!started) return;
  touchInteractionRef.current = null;
  if (interaction.kind === "moving") {
    for (const orig of interaction.originalElements) updateElement(orig.id, orig, false);
  }
  setSelected(started.priorSelection);
  setRubberBand(null);
  setInteraction({ kind: "idle" });
};

```

```

};

// Touchscreen Double-Tap Detection
const checkDoubleTap = (clientX: number, clientY: number): boolean => {
  const now = Date.now();
  const last = lastTapRef.current;
  const isDouble =
    now - last.time < 320 &&
    Math.hypot(clientX - last.x, clientY - last.y) < 30;
  lastTapRef.current = { time: now, x: clientX, y: clientY };
  return isDouble;
};

// Pointer Down
const handlePointerDown = (e: React.PointerEvent<SVGSVGElement>) => {
  // Any new contact catches a coasting page, as on every phone.
  stopMomentum();
  if (shapeNote) setShapeNote(null);
  placeEraserCursor(e.clientX, e.clientY, tool === "eraser");
  placePenHoverCursor(0, 0, false);
  clearAutoStraightenTimer();
  isAutoStraightenedRef.current = false;
  // The laser and text select point and read; they must never add a page
  // through the "+" between sheets.
  const pointingTool = tool === "laser" || tool === "textselect";
  if (isNb && !pointingTool) {
    const gw = getWorldPoint(e.clientX, e.clientY);
    const k = gapAt(gw.x, gw.y, isTouchPress(e) ? 10 : 0);
    if (k !== null) showGapPlus(k, 3000);
    else showGapPlus(null); // writing elsewhere: never leave a "+" under the nib
  }
  const isTouch = e.pointerType === "touch";
  const isPen = e.pointerType === "pen";

  if (isPen) {
    penActiveRef.current = true;
    lastPenAtRef.current = Date.now();
    armContextMenu(false);
    retractTouchInteraction();
  }

  syncStylusErase(e);

  // Palm rejection, ahead of anything that could react to this contact.
  // The old test here was `penActiveRef.current` alone – true only WHILE the
  // nib is on the glass – which misses both ways a palm actually arrives: it
  // comes down before you start writing, and it stays down while you lift
  // the pen between words. classifyTouch outlasts the pen and also weighs
  // the size of the contact patch. See lib/whiteboard/palm.ts.
  let touchVerdict: TouchVerdict = "allow";
  // Same rule for the mouse: a right-click with nothing selected used to
  // open the same all-disabled menu.
  if (!isTouch && !isPen) armContextMenu(selectedIds.length > 0);
  if (isTouch) {
    touchVerdict = classifyTouch(
      { width: e.width, height: e.height },
      { penDown: penActiveRef.current, lastPenAt: lastPenAtRef.current, now: Date.now() }
    );
    // A stylus on (or just off) the glass is decisive: the contact is a palm
    // and gets nothing at all.
    // A rejected contact is a palm: it gets no menu. A clean fingertip keeps
    // the deliberate long press, which Radix cancels on movement anyway – so
    // hold-and-drag moves the object and hold-and-wait opens the menu.
    // Mid-edit nothing does: while you are typing, a hand on the glass is a
    // resting hand, and the menu popping up over your text is the very
    // interruption this whole pass is about.
    // ...and only when the menu could actually DO something. Every item in
    // it acts on the current selection, so with nothing selected a long
    // press opened a menu whose five entries were all greyed out – it
    // appeared uninvited, explained nothing and did nothing. Selecting by
    // finger is the long press handled further down; the menu is the
    // SECOND hold, once something is selected.
    armContextMenu(touchVerdict === "allow" && !editingId && selectedIds.length > 0);

    // Context-aware multi-touch fast-bypass (R1-DEF-03):
    // If the pen is lifted (!penActiveRef) and a second fingertip arrives to pinch/scroll,
    // allow multi-touch to proceed if contacts are fingertip-sized (< PALM_CONTACT_PX).
    const isSecondFingertipPinch =

```

```

!penActiveRef.current &&
touchVerdict === "pen-recent" &&
activeTouchesRef.current.size >= 1 &&
!contactLooksLikePalm({ width: e.width, height: e.height }));

if ((touchVerdict === "pen-down" || touchVerdict === "pen-recent") && !isSecondFingertipPinch) {
  // Radix's long press lives on this same <svg>, composed AFTER this
  // handler, and it skips itself when the event is already default-
  // prevented. Using that rather than the `disabled` prop alone makes
  // the palm case synchronous and immune to render timing – and a palm
  // has nothing whose default we want anyway: suppressing the compat
  // mouse events also stops a resting hand from dismissing the text
  // editor you are in the middle of typing into.
  e.preventDefault();
  // Still recorded, so multi-touch maths and the matching release stay
  // consistent – just never allowed to reach content.
  rejectedPalmsRef.current.add(e.pointerId);
  activeTouchesRef.current.set(e.pointerId, {
    x: e.clientX,
    y: e.clientY,
    startX: e.clientX,
    startY: e.clientY,
  });
  return;
}

if (isSecondFingertipPinch) {
  // Un-reject existing fingertip contacts since this is a valid multi-touch gesture
  for (const [id] of activeTouchesRef.current.entries()) {
    rejectedPalmsRef.current.delete(id);
  }
}
// Contact size alone is weaker evidence, and on a stylus-free tablet it
// is the ONLY evidence there is – so an oversized patch is demoted to
// navigation (below) rather than silenced. A resting hand then does
// nothing because it does not move; a broad fingertip can still pan and
// can still long-press to select.
}

// Only finger pointers count toward multi-touch gestures.
if (isTouch) {
  activeTouchesRef.current.set(e.pointerId, {
    x: e.clientX,
    y: e.clientY,
    startX: e.clientX,
    startY: e.clientY,
  });

  // Exclude rejected palms from gesture calculations (Defect D5)
  const validTouches = Array.from(activeTouchesRef.current.entries())
    .filter(([id]) => !rejectedPalmsRef.current.has(id))
    .map([, t] => t);
  const currentTouchCount = validTouches.length;

  // A new gesture starts when the FIRST finger lands – not on every finger.
  if (currentTouchCount === 1) {
    touchStartTimeRef.current = Date.now();
    touchMovedRef.current = false;
  }
  if (currentTouchCount > maxTouchCountRef.current) {
    maxTouchCountRef.current = currentTouchCount;
  }

  // Multi-touch Pinch to Zoom and 2-Finger Pan Setup on Touchscreen
  // Guard against Defect D5: Inking takes absolute precedence; never hijack into pinch zoom while inking!
  if (
    currentTouchCount >= 2 &&
    !penActiveRef.current &&
    drawingPointerIdRef.current === null &&
    interaction.kind !== "drawing"
  ) {
    // Seal any in-progress gesture so it stays undoable.
    if (
      interaction.kind === "moving" ||
      interaction.kind === "resizing" ||
      interaction.kind === "rotating"
    ) {
      commitHistory();
    }
  }
}

```



```

    gestureSealedRef.current = true;
  }
  // Two fingers mean zoom or scroll, never pointing or selecting.
  laser.clear();
  textSelect.cancel();
  const touches = validTouches;
  const dist = Math.hypot(touches[0].x - touches[1].x, touches[0].y - touches[1].y);
  pinchStartDistanceRef.current = dist;
  pinchEngagedRef.current = false;
  pinchStartZoomRef.current = camera.zoom;
  const midClientX = (touches[0].x + touches[1].x) / 2;
  const midClientY = (touches[0].y + touches[1].y) / 2;
  lastTouchMidRef.current = { x: midClientX, y: midClientY };
  setInteraction({ kind: "idle" });
  setDraftElement(null);
  return;
}
}

const cp = getContainerPoint(e.clientX, e.clientY);
// While capture is armed the press draws an area, whatever tool is held:
// the pen's palm rule must not turn a finger's capture drag into a pan.
const isDrawingTool = isStrokeTool(tool) && !captureArmed;

// Palm Rejection. In stylus-only mode fingers are navigation-only for
// EVERY tool (the old isDrawingTool restriction let palms lasso/erase/
// connect). The palmRejection toggle keeps its drawing-tools-only scope.
const stylusOnlyFinger = isTouch && touchSettings.mode === "stylus-only";
const drawingPalmRejection = isTouch && touchSettings.palmRejection && isDrawingTool;
// A big contact patch only means "palm" if palms are what we are looking
// for. classifyTouch reports the size honestly, but acting on it while the
// user has palm rejection OFF made finger drawing impossible on ordinary
// Windows touchscreens, which commonly report a fingertip at 40-60 CSS px
// - past the 45px threshold. So the sensor stays sensitive and the policy
// lives here, with the setting.
const oversizedContact = touchVerdict === "contact-size" && touchSettings.palmRejection;

// A finger on one of the selection's HANDLES is a deliberate grab, not a
// resting hand: it lands on a small target that only exists because you
// selected something. Without this, stylus-only mode (and palm rejection
// with a pen tool) made resize and rotate impossible by touch. A contact
// big enough to be a palm is still never exempt.
const onSelectionHandle =
  isTouch &&
  touchVerdict === "allow" &&
  selectedIds.length > 0 &&
  Boolean((e.target as Element | null)?.getAttribute?.("data-handle"));

if ((stylusOnlyFinger || drawingPalmRejection || oversizedContact) && !onSelectionHandle) {
  // Navigation only - which this branch did not honour: it SELECTED
  // whatever the finger landed on, so a resting palm popped the transform
  // box open even with palm rejection switched on. Selection here is the
  // deliberate long press instead, and never mid-edit.
  const wp = getWorldPoint(e.clientX, e.clientY);
  const hit = findElementAt(wp.x, wp.y, true);
  // A patch this size is a palm, and a palm holds perfectly still - so it
  // would pass a long press with room to spare. It gets no way to select;
  // a real fingertip (well under the threshold) still does.
  if (hit && !editingId && !oversizedContact) armLongPressSelect(e, hit);
  beginPan(cp.x, cp.y);
  capturePointer(e);
  return;
}

// -----
// The live page follows the pen
// -----
// Zoomed out, two or three sheets are on screen but only the anchored
// one is live - the rest are drawn as pictures. A pen landing on the
// sheet above or below used to count as "off the page", so the ink gate
// turned it into a PAN: writing on a visible page scrolled instead. The
// same root cause made the eraser pan, selection miss, and a double-tap
// drop its text box on the wrong page.
//
// So a press on another visible sheet first makes THAT sheet the live
// one (re-anchoring shifts the camera by the same stride, so nothing
// moves on screen), then replays this very press once React has
// committed the new page - every tool then sees the right page's

```

```

// coordinates and elements, including hit-testing, which a coordinate
// patch-up alone could not give it.
if (
  isNb &&
  allPages.length > 1 &&
  tool !== "pan" &&
  // Pointing at another sheet must not make it the active page.
  tool !== "laser" &&
  // Nor may a capture area: it spans sheets in the frame it started in.
  !captureArmed &&
  !spaceHeld &&
  e.button !== 1 &&
  !(e.nativeEvent as PointerEvent & { __reanchored?: boolean }).__reanchored &&
  activeTouchesRef.current.size <= 1
) {
  const pw = getWorldPoint(e.clientX, e.clientY);
  const target = sheetIndexAt(
    pw.x,
    pw.y,
    activeIndex,
    allPages.length,
    pageDim.width,
    pageDim.height,
    SHEET_GRAB_MARGIN
  );
  if (target !== null && target !== activeIndex) {
    const delta = target - activeIndex;
    anchorLockRef.current = { camY: cameraRef.current.y - delta * pageStride };
    // Commit the page switch NOW. React otherwise applies it in a
    // microtask of its own - queued after ours - so the replay ran with
    // the old page's coordinates, fell "off the page" and panned.
    flushSync(() => reanchorPage(delta, pageStride));
    const n = e.nativeEvent;
    const replay = new PointerEvent("pointerdown", {
      pointerId: n.pointerId,
      pointerType: n.pointerType,
      isPrimary: n.isPrimary,
      clientX: n.clientX,
      clientY: n.clientY,
      screenX: n.screenX,
      screenY: n.screenY,
      pressure: n.pressure,
      tangentialPressure: n.tangentialPressure,
      tiltX: n.tiltX,
      tiltY: n.tiltY,
      twist: n.twist,
      width: n.width,
      height: n.height,
      button: n.button,
      buttons: n.buttons,
      shiftKey: n.shiftKey,
      ctrlKey: n.ctrlKey,
      altKey: n.altKey,
      metaKey: n.metaKey,
      bubbles: true,
      cancelable: true,
    });
    (replay as PointerEvent & { __reanchored?: boolean }).__reanchored = true;
    const origin = (n.target as Element | null) ?? svgRef.current;
    // After flushSync the new page is committed, so the replay sees it;
    // and a microtask still runs before the next pointermove arrives.
    queueMicrotask(() => origin?.dispatchEvent(replay));
    return;
  }
}

// Touch double-tap creates text ONLY in select/text tools – otherwise it
// would hijack the eraser/lasso/connect/pen tools (tap-tap-erase would
// spawn a text box instead of erasing).
const isDouble =
  isTouch &&
  !captureArmed &&
  (tool === "select" || tool === "text") &&
  checkDoubleTap(e.clientX, e.clientY);

if (isDouble) {
  const wp = getWorldPoint(e.clientX, e.clientY);
  const hit = findElementAt(wp.x, wp.y, true);

```

```

    if (hit) {
      if (hit.type === "text" || hit.type === "sticky") {
        setEditingId(hit.id);
        return;
      }
    } else {
      const textId = nanoid(8);
      const newText: TextElement = {
        id: textId,
        type: "text",
        x: wp.x,
        y: wp.y,
        width: 220,
        height: 48,
        rotation: 0,
        opacity: 1,
        text: "",
        color: style.textColor,
        fontSize: style.fontSize,
        fontFamily: style.fontFamily,
        fontWeight: style.fontWeight,
        align: style.textAlign,
        createdAt: Date.now(),
        updatedAt: Date.now(),
      };
      addElement(newText, true);
      setEditingId(textId);
      return;
    }
  }

  if (isTouch && touchSettings.mode === "touch-pan") {
    beginPan(cp.x, cp.y);
    capturePointer(e);
    return;
  }

  const wp = getWorldPoint(e.clientX, e.clientY);

  // Capture: draw the area. It creates nothing and selects nothing, so it
  // needs no page gate - a box may start on the desk and sweep over two
  // sheets. Space / middle button still pan, so you can reframe first.
  // A press on something already SELECTED moves it, even with the mode on:
  // the app selects a picture it has just added (a capture put "on this
  // page", a photo) so you can drag it into place.
  const onSelection =
    captureArmed !== null &&
    selectedIds.length > 0 &&
    (() => {
      const hit = findElementAt(wp.x, wp.y, isTouch);
      return !!hit && selectedIds.includes(hit.id);
    })();
  if (captureArmed && e.button === 0 && !spaceHeld && !onSelection) {
    // One area at a time: a second contact must not restart the one the
    // first is still drawing - except the pen, over an area a resting palm
    // began a moment before it.
    if (interaction.kind === "capturing" && !(isPen && interaction.byTouch)) return;
    setInteraction({
      kind: "capturing",
      shape: captureArmed,
      pointerId: e.pointerId,
      byTouch: isTouch,
      points: [{ x: wp.x, y: wp.y }],
    });
    capturePointer(e);
    return;
  }

  if (e.button === 1 || spaceHeld || tool === "pan") {
    beginPan(cp.x, cp.y);
    capturePointer(e);
    return;
  }

  // The laser points; it never selects, writes or moves anything. Anywhere
  // on screen, the desk beside the pages included.
  if (tool === "laser") {
    laser.down(e.pointerId, wp.x, wp.y);
  }

```

```

    capturePointer(e);
    return;
}

// Select text: a press lands on the page's text, never on your ink. Off
// the text (the margin, a scan) it scrolls, as a reader would.
if (tool === "textselect") {
    if (!textSelect.pointerDown(e)) beginPan(cp.x, cp.y);
    capturePointer(e);
    return;
}

// In a notebook, the area beside and between the sheets is a desk, not
// canvas. A content gesture that starts there pans the view instead of
// laying down ink that would live outside every page – unreachable once
// you scroll, and absent from every export and print.
// `select` is exempt: a marquee that begins on the desk and sweeps onto
// the page is a normal way to select, and it creates nothing.
const isContentTool =
    isDrawingTool ||
    tool === "eraser" ||
    tool === "text" ||
    tool === "sticky" ||
    ["rectangle", "ellipse", "diamond", "triangle", "star", "cloud", "line", "arrow"].includes(
        tool
    );
if (isContentTool && !canStartInkAt(wp.x, wp.y)) {
    beginPan(cp.x, cp.y);
    capturePointer(e);
    return;
}

// The press survived the gate, so it is aimed at the page – but it may
// still be in the grab margin just outside it. Content starts from `ip`,
// which is on the paper; everything else (hit-testing, panning, marquee)
// keeps using the honest `wp`.
const ip = isNb ? clampToSheet(wp.x, wp.y, pageDim.width, pageDim.height) : wp;

if (e.button !== 0 && e.pointerType === "mouse") return;

// 1. Check resize/rotate handle
const target = e.target as HTMLElement | SVGElement;
const handleName = target.getAttribute?.("data-handle") as ResizeHandle | null;
if (handleName && selectedIds.length > 0) {
    const el = elements.find((x) => x.id === selectedIds[0]);
    if (el && !el.locked) { // locked elements expose no resize/rotate handles
        commitHistory();
        if (handleName === "rotate") {
            const center = getElementCenter(el);
            const startAngle = Math.atan2(wp.y - center.y, wp.x - center.x);
            setInteraction({
                kind: "rotating",
                originalElement: { ...el },
                startAngle,
                centerX: center.x,
                centerY: center.y,
            });
        } else {
            setInteraction({
                kind: "resizing",
                handle: handleName,
                originalElement: { ...el },
                startWorldX: wp.x,
                startWorldY: wp.y,
            });
        }
        capturePointer(e);
        return;
    }
}

// 2. CONNECT Tool
if (tool === "connect") {
    const hit = findElementAt(wp.x, wp.y, isTouch);
    if (hit && activeBoardId) {
        const ports = getNodeAnchorPorts(hit);
        let bestAnchor: NodeAnchor = "right";
        let minDist = Infinity;
    }
}

```

```

    for (const [k, pt] of Object.entries(ports)) {
      const d = Math.hypot(pt.x - wp.x, pt.y - wp.y);
      if (d < minDist) {
        minDist = d;
        bestAnchor = k as NodeAnchor;
      }
    }
  }

  setInteraction({
    kind: "connecting",
    sourceElementId: hit.id,
    sourceAnchor: bestAnchor,
    currentWorldX: wp.x,
    currentWorldY: wp.y,
  });
  capturePointer(e);
  return;
}
// Connect tool with no node under the pointer: swallow the gesture –
// never fall through to the select tool's rubber-band branch.
return;
}

// 3. Freehand Drawing Tools (Pen, Highlighter, Marker, Brush, Calligraphy, Pencil)
if (isDrawingTool) {
  const strokeId = nanoid(8);
  // R1-DEF-02: Explicitly bind drawingPointerId to isolate inking stream
  drawingPointerIdRef.current = e.pointerId;

  // A pen reports 0..1 while touching. A 0 is a sample with no pressure
  // reading (it happens on the first sample of some strokes), not the
  // lightest touch: taken literally it drew a hairline start. It falls
  // back to the last real reading; a mouse reports 0.5 anyway.
  const rawPressure = e.pressure > 0 ? e.pressure : 0.5;
  lastPenPressureRef.current = rawPressure;
  // This pen's own feel (see penFeelFor).
  const feel = penFeelFor(touchSettings, tool);
  const pressure = calibratePressure(rawPressure, feel.sensitivity, feel.curve);

  const color = getToolColor(style, tool);
  const strokeWidth = getToolSize(style, tool);
  const strokeStyle = tool as StrokeStyle;
  const smoothing = feel.smoothing;

  deferSavesWhileWriting();
  // Initialize transient stroke buffer in ref (ZERO React state)
  liveDraftStrokeRef.current = {
    id: strokeId,
    originX: ip.x,
    originY: ip.y,
    color,
    strokeWidth,
    strokeStyle,
    smoothing,
    createdAt: Date.now(),
    points: [[0, 0, pressure]],
  };
  predictedPointsRef.current = [];

  // Configure direct DOM nodes immediately for zero-frame touchdown response
  const preset = PEN_TYPE_PRESETS[strokeStyle] || PEN_TYPE_PRESETS.gel;
  const g = liveDraftGroupRef.current;
  const pathEl = liveDraftPathRef.current;
  if (g && pathEl) {
    g.setAttribute("transform", `translate(${ip.x}, ${ip.y})`);
    g.style.display = ""; // Unhide
    pathEl.setAttribute("fill", color);
    pathEl.style.opacity = String(preset.opacity ?? 1);
    pathEl.style.mixBlendMode = preset.mixBlendMode || "normal";
    if (preset.grain) {
      pathEl.setAttribute("filter", "url(#pencil-grain)");
    } else {
      pathEl.removeAttribute("filter");
    }
  }
  // Render initial touchdown dot
  applyLiveDraftStroke();
}

```

```
// Auto-straighten deadband anchor initialization (R1-DEF-08 / D4)
autoStraightenAnchorRef.current = { x: ip.x, y: ip.y };

// Set interaction state (only 1 React state update at touchdown; none during move)
setInteraction({ kind: "drawing", elementId: strokeId, tool });
capturePointer(e);
return;
}

// 4. Eraser Tool
if (tool === "eraser") {
  // A new press starts a new sweep; otherwise it would erase a line all the
  // way from wherever the last stroke ended.
  lastErasePosRef.current = null;
  eraseAt(ip.x, ip.y, isTouch, true);
  setInteraction({ kind: "erasing" });
  capturePointer(e);
  return;
}

// 5. Shape Creation Tools
if (["rectangle", "ellipse", "diamond", "triangle", "star", "cloud", "line", "arrow"].includes(tool)) {
  const shapeId = nanoid(8);
  let newShape: WhiteboardElement;

  const common = {
    id: shapeId,
    x: ip.x,
    y: ip.y,
    rotation: 0,
    opacity: 1,
    strokeColor: style.shapeStrokeColor,
    fillColor: style.shapeFillColor,
    fillStyle: style.fillStyle,
    dashStyle: style.dashStyle,
    strokeWidth: style.strokeWidth,
    createdAt: Date.now(),
    updatedAt: Date.now(),
  };

  if (tool === "line") {
    newShape = { ...common, type: "line", x2: 1, y2: 1 };
  } else if (tool === "arrow") {
    newShape = {
      ...common,
      type: "arrow",
      x2: 1,
      y2: 1,
      startArrowhead: "none",
      endArrowhead: "arrow",
      arrowheadSize: 12,
    };
  } else {
    newShape = { ...common, type: tool as any, width: 2, height: 2 };
  }

  setDraftElement(newShape);
  setInteraction({ kind: "creating-shape", elementId: shapeId, startX: ip.x, startY: ip.y });
  capturePointer(e);
  return;
}

// 6. Text Tool Click -> Place text directly at clicked position
if (tool === "text") {
  const textId = nanoid(8);
  const posX = snapToGridActive ? snapToGrid(ip.x, paperSize) : ip.x;
  const posY = snapToGridActive ? snapToGrid(ip.y, paperSize) : ip.y;
  const newText: TextElement = {
    id: textId,
    type: "text",
    x: posX,
    y: posY,
    width: 220,
    height: 48,
    rotation: 0,
    opacity: 1,
    text: "",
    color: style.textColor,
  };
}
```

```

        fontSize: style.fontSize,
        fontFamily: style.fontFamily,
        fontWeight: style.fontWeight,
        align: style.textAlign,
        createdAt: Date.now(),
        updatedAt: Date.now(),
    };
    addElement(newText, true);
    setEditingId(textId);
    setTool("select");
    return;
}

// 7. Sticky Note Tool Click
if (tool === "sticky") {
    const stickyId = nanoid(8);
    const posX = snapToGridActive ? snapToGrid(ip.x - 100, paperSize) : ip.x - 100;
    const posY = snapToGridActive ? snapToGrid(ip.y - 80, paperSize) : ip.y - 80;
    const newSticky: StickyElement = {
        id: stickyId,
        type: "sticky",
        x: posX,
        y: posY,
        width: 200,
        height: 160,
        rotation: 0,
        opacity: 1,
        text: "",
        color: style.stickyColor,
        fontSize: style.fontSize,
        align: style.textAlign,
        createdAt: Date.now(),
        updatedAt: Date.now(),
    };
    addElement(newSticky, true);
    setEditingId(stickyId);
    setTool("select");
    return;
}

// 8. Lasso Tool: Freeform polygon selection. Strictness is captured
// here, not read live: flipping the setting mid-loop must not change
// what the loop in progress means.
if (tool === "lasso") {
    if (!e.shiftKey) clearSelection();
    setInteraction({
        kind: "lasso-selecting",
        points: [{ x: wp.x, y: wp.y }],
        screenPoints: [{ x: cp.x, y: cp.y }],
        strict: useWhiteboard.getState().lassoStrict,
    });
    capturePointer(e);
    return;
}

// 9. Select Tool: Hit test elements
const rawHit = findElementAt(wp.x, wp.y, isTouch);
const hit = rawHit && (lockedMode || passesSelectFilter(rawHit, selectFilter)) ? rawHit : null;
if (hit) {
    // A finger landing on an object it has not already selected does NOT
    // grab it. That one behaviour is what threw the transform box – resize
    // and rotate handles and all – onto the screen every time a hand came to
    // rest on a line of text, and dragged the text away when the hand
    // shifted. Selecting by finger is Android's long press now; a finger
    // drag pans, exactly as it does in every tablet note app.
    // Already selected: hold still and the options menu opens; drag and the
    // object moves (Radix cancels its own timer on movement).
    if (isTouch && selectedIds.includes(hit.id)) armContextMenu(true);

    if (isTouch && !e.shiftKey && !selectedIds.includes(hit.id)) {
        // A clean fingertip tap selects at once – the tap-to-select,
        // press-drag-to-move contract every tablet note app uses. The drag
        // that follows still pans (beginPan below), so finger navigation is
        // unchanged; only the mystery second press disappears. Anything else
        // (a palm-sized patch, mid-edit) keeps the deliberate long press, so
        // a resting hand still cannot pop the transform box open.
        if (
            shouldSelectOnTouchTap({

```

```

        verdict: touchVerdict,
        hitSelected: false,
        shiftKey: e.shiftKey,
        editing: editingId !== null,
      })
    ) {
      setSelected([hit.id]);
      touchInteractionRef.current = { pointerId: e.pointerId, priorSelection: selectedIds };
    } else if (!editingId) {
      // Mid-edit a touch is overwhelmingly a resting hand rather than a
      // reach for another object, so it does not even arm. Leaving the
      // editor still works – the tap blurs it as it always did.
      armLongPressSelect(e, hit);
    }
    beginPan(cp.x, cp.y);
    capturePointer(e);
    return;
  }

  if (e.shiftKey) {
    if (selectedIds.includes(hit.id)) {
      setSelected(selectedIds.filter((id) => id !== hit.id));
    } else {
      addToSelection(hit.id);
    }
  } else {
    if (!selectedIds.includes(hit.id)) {
      setSelected([hit.id]);
    }
  }
}

const targets = (
  selectedIds.includes(hit.id)
    ? elements.filter((el) => selectedSet.has(el.id))
    : [hit]
).filter((el) => !el.locked); // locked elements never move

setInteraction({
  kind: "moving",
  startWorldX: wp.x,
  startWorldY: wp.y,
  originalElements: targets.map((el) => ({ ...el })),
});
if (isTouch) {
  touchInteractionRef.current = { pointerId: e.pointerId, priorSelection: selectedIds };
}
capturePointer(e);
} else {
  if (!e.shiftKey) clearSelection();
  setInteraction({
    kind: "selecting",
    startX: cp.x,
    startY: cp.y,
    worldX: wp.x,
    worldY: wp.y,
  });
  capturePointer(e);
}
};

// Pointer Move
const handlePointerMove = (e: React.PointerEvent<SVGSVGElement>) => {
  lastPointerPosRef.current = { clientX: e.clientX, clientY: e.clientY };
  // A pen or mouse shows the ring while hovering; a finger has no hover, so
  // it shows only while touching.
  placeEraserCursor(
    e.clientX,
    e.clientY,
    tool === "eraser" && (e.pointerType !== "touch" || e.buttons > 0)
  );

  if (tool === "laser") {
    const lp = getWorldPoint(e.clientX, e.clientY);
    if (laser.isActive(e.pointerId)) {
      laser.move(lp.x, lp.y);
      // A finger pointing with the laser is still a finger: keep its
      // position current, or a second finger's pinch starts from where
      // this one first landed and the page jumps.
    }
  }
};

```



```

    const t = activeTouchesRef.current.get(e.pointerId);
    if (t) {
        if (Math.hypot(e.clientX - t.startX, e.clientY - t.startY) > 6) touchMovedRef.current = true;
        activeTouchesRef.current.set(e.pointerId, { ...t, x: e.clientX, y: e.clientY });
    }
    return;
}
// A hovering S Pen (or a mouse) carries the dot without pressing: the
// presenter's way to point.
if (e.buttons === 0 && e.pointerType !== "touch") laser.hover(lp.x, lp.y, e.pointerType);
}

// S Pen & Stylus Hover Preview Cursor
const isPen = e.pointerType === "pen";
const isDrawingTool = isStrokeTool(tool);
const showPenHover =
    touchSettings.penHoverIndicator !== false &&
    isDrawingTool &&
    tool !== "eraser" &&
    (isPen || e.pointerType === "mouse") &&
    e.buttons === 0;
placePenHoverCursor(e.clientX, e.clientY, showPenHover);
// A hovering pen (or mouse) over the gap between two sheets shows "+".
if (isNb && e.buttons === 0 && e.pointerType !== "touch" && !gapPlusTimerRef.current && tool !== "laser" &
    & tool !== "textselect") {
    const hw = getWorldPoint(e.clientX, e.clientY);
    showGapPlus(gapAt(hw.x, hw.y));
}

// Keep the suppression window fed while the stylus is drawing OR merely
// hovering: a hovering pen means the hand is in writing position, which is
// precisely when the palm is on the glass.
if (e.pointerType === "pen") lastPenAtRef.current = Date.now();

// The button is read here too, not only on contact: this is what makes
// hovering with the button held arm the eraser, and what frees the pen the
// instant the button goes up rather than when the nib finally lifts.
syncStylusErase(e);

const press = longPressRef.current;
if (press && press.pointerId === e.pointerId) {
    // Moved before the hold completed: this was a pan, not a grab.
    if (exceedsLongPressSlop(e.clientX - press.startX, e.clientY - press.startY)) {
        cancelLongPress();
    }
}

// A long press has selected an object; the drag that follows moves it. The
// origin is THIS event's world point rather than the press point, so the
// object cannot jump by however far the finger drifted while holding. The
// event is consumed so the in-progress pan does not also scroll the board.
const pending = pendingMoveRef.current;
if (pending && pending.pointerId === e.pointerId) {
    pendingMoveRef.current = null;
    const el = elements.find((x) => x.id === pending.elementId);
    if (el && !el.locked) {
        const origin = getWorldPoint(e.clientX, e.clientY);
        setInteraction({
            kind: "moving",
            startWorldX: origin.x,
            startWorldY: origin.y,
            originalElements: [{ ...el }],
        });
        return;
    }
}

// Update touch position
if (activeTouchesRef.current.has(e.pointerId)) {
    const orig = activeTouchesRef.current.get(e.pointerId);
    if (Math.hypot(e.clientX - orig.startX, e.clientY - orig.startY) > 6) {
        touchMovedRef.current = true;
    }
    activeTouchesRef.current.set(e.pointerId, { ...orig, x: e.clientX, y: e.clientY });
}

// Multi-touch Pinch to Zoom & 2-Finger Pan Handling
// Guard against Defect D5: Dual palm contacts during inking must never trigger pan/zoom.

```

```

const nonPalmTouches = Array.from(activeTouchesRef.current.entries())
  .filter(([id]) => !rejectedPalmsRef.current.has(id))
  .map([, t]) => t);

if (
  nonPalmTouches.length === 2 &&
  !penActiveRef.current &&
  drawingPointerIdRef.current === null &&
  interaction.kind !== "drawing"
) {
  const touches = nonPalmTouches;
  const currDist = Math.hypot(touches[0].x - touches[1].x, touches[0].y - touches[1].y);
  const midClientX = (touches[0].x + touches[1].x) / 2;
  const midClientY = (touches[0].y + touches[1].y) / 2;
  const midContainer = getContainerPoint(midClientX, midClientY);

  // 1. Pan delta tracking (pure 2-finger pan and pan while pinching)
  if (!lastTouchMidRef.current) {
    // A fresh two-finger gesture: its throw is measured from here.
    stopMomentum();
    panSamplesRef.current = [];
  }
  recordPanSample(midClientX, midClientY);
  if (lastTouchMidRef.current) {
    const dx = midClientX - lastTouchMidRef.current.x;
    const dy = midClientY - lastTouchMidRef.current.y;
    if (Math.hypot(dx, dy) > 0.5) {
      // Coalesced to one camera write per frame, exactly like the
      // single-pointer pan – two fingers used to be the WORST case,
      // because both fingers' moves each drove a full re-render.
      queuePan(dx, dy);
    }
  }
  lastTouchMidRef.current = { x: midClientX, y: midClientY };

  // 2. Pinch zoom – only once it is clearly a PINCH.
  //
  // The old gate was `|scale - 1| > 0.005`: half a percent. Two fingers
  // never travel perfectly parallel, so an ordinary two-finger SCROLL
  // drifted past that within a few pixels and the page zoomed slightly the
  // whole way down. That both looks like judder and re-renders the canvas
  // on every move, which is why scrolling felt heavy. Zoom now has to be
  // asked for, and once asked for it stays engaged for the gesture.
  if (pinchStartDistanceRef.current !== null && pinchStartDistanceRef.current > 10) {
    const scale = currDist / pinchStartDistanceRef.current;
    if (!pinchEngagedRef.current) {
      const spread = Math.abs(currDist - pinchStartDistanceRef.current);
      if (Math.abs(scale - 1) > 0.12 && spread > 32) pinchEngagedRef.current = true;
    }
    if (pinchEngagedRef.current) {
      zoomTo(pinchStartZoomRef.current * scale, midContainer.x, midContainer.y, viewport);
    }
  }
  return;
}

// After the pinch above, so a second finger always wins over a selection.
if (textSelect.pointerMove(e)) return;

const wp = getWorldPoint(e.clientX, e.clientY);
const cp = getContainerPoint(e.clientX, e.clientY);
// Content follows the clamped point so a gesture that runs off a notebook
// page stops at the margin instead of trailing ink across the desk.
const ip = isNb ? clampToSheet(wp.x, wp.y, pageDim.width, pageDim.height) : wp;

// Continuous drag-erasing (never while another tool's gesture is active)
if (interaction.kind === "erasing" || (tool === "eraser" && e.buttons > 0 && interaction.kind === "idle")) {
  // The idle branch means we never saw a pointerdown (pointer entered
  // already held, or capture was lost). Continuing the previous gesture's
  // sweep from there would erase a straight line between two unrelated
  // places, so start a fresh sweep instead.
  if (interaction.kind !== "erasing") lastErasePosRef.current = null;
  eraseAt(ip.x, ip.y, e.pointerType === "touch", false);
  return;
}

// Capture area: a box keeps its first point and follows the pointer; a

```

```

// loop records the trail, thinned like the lasso's.
if (interaction.kind === "capturing") {
  if (e.pointerId !== interaction.pointerId) return;
  const first = interaction.points[0];
  if (interaction.shape === "rectangle") {
    setInteraction({ ...interaction, points: [first, { x: wp.x, y: wp.y }] });
  } else {
    const lastPt = interaction.points[interaction.points.length - 1];
    if (!lastPt || Math.hypot(wp.x - lastPt.x, wp.y - lastPt.y) >= 2 / Math.max(camera.zoom, 0.01)) {
      setInteraction({ ...interaction, points: [...interaction.points, { x: wp.x, y: wp.y }] });
    }
  }
  return;
}

// Lasso Freeform Selection
if (interaction.kind === "lasso-selecting") {
  const lastPt = interaction.points[interaction.points.length - 1];
  if (!lastPt || Math.hypot(wp.x - lastPt.x, wp.y - lastPt.y) >= 2) {
    const updatedPoints = [...interaction.points, { x: wp.x, y: wp.y }];
    const updatedScreenPoints = [...interaction.screenPoints, { x: cp.x, y: cp.y }];
    setInteraction({
      kind: "lasso-selecting",
      points: updatedPoints,
      screenPoints: updatedScreenPoints,
      strict: interaction.strict,
    });
    // No selection while the loop is still open. Judging it live meant
    // judging the half-drawn shape SVG closes for you – a wedge hinged on
    // the start point – so things flicked in and out of the selection as
    // that chord swept across them. The loop is only a loop once the pen
    // lifts, which is when Samsung decides too.
  }
  return;
}

// Panning
if (interaction.kind === "panning") {
  // The gesture's last position is a REF: updating it through
  // setInteraction re-rendered the whole canvas on every pointer event.
  const last = panLastRef.current ?? { x: interaction.lastX, y: interaction.lastY };
  const dx = cp.x - last.x;
  const dy = cp.y - last.y;
  panLastRef.current = { x: cp.x, y: cp.y };
  recordPanSample(cp.x, cp.y);

  queuePan(dx, dy);
  return;
}

// Connecting (drawing arrow to target node)
if (interaction.kind === "connecting") {
  const hit = findElementAt(wp.x, wp.y, e.pointerType === "touch");
  let targetAnchor: NodeAnchor | undefined = undefined;
  if (hit && hit.id !== interaction.sourceElementId) {
    const ports = getNodeAnchorPorts(hit);
    let best: NodeAnchor = "left";
    let minDist = Infinity;
    for (const [k, pt] of Object.entries(ports)) {
      const d = Math.hypot(pt.x - wp.x, pt.y - wp.y);
      if (d < minDist) {
        minDist = d;
        best = k as NodeAnchor;
      }
    }
    targetAnchor = best;
  }

  setInteraction({
    ...interaction,
    currentWorldX: wp.x,
    currentWorldY: wp.y,
    targetElementId: hit && hit.id !== interaction.sourceElementId ? hit.id : undefined,
    targetAnchor,
  });
  return;
}

```

```

// Freehand Drawing (Direct DOM + RAF Coalescence + Prediction)
if (liveDraftStrokeRef.current) {
  // R1-DEF-02: Isolate pen stream from palm moves
  if (drawingPointerIdRef.current !== null && e.pointerId !== drawingPointerIdRef.current) {
    return;
  }

  const live = liveDraftStrokeRef.current;
  if (!live) return;

  // 1. High-Frequency Coalesced Events: S Pen hardware digitizer samples at 240-360Hz.
  const native = e.nativeEvent as unknown as {
    getCoalescedEvents?: () => PointerEvent[];
    getPredictedEvents?: () => PointerEvent[];
  };
  const rawEvents: PointerEvent[] =
    typeof native?.getCoalescedEvents === "function"
      ? native.getCoalescedEvents()
      : [];
  const eventsToProcess: Array<PointerEvent | React.PointerEvent<SVGSVGElement>> =
    rawEvents.length > 0 ? rawEvents : [e];

  const { sensitivity, curve } = penFeelFor(touchSettings, live.strokeStyle);

  // Cache container rect once per event batch
  const containerRect = containerRef.current?.getBoundingClientRect();

  const ptsToAdd: Array<[number, number, number]> = [];
  for (const ev of eventsToProcess) {
    const clientX = ev.clientX;
    const clientY = ev.clientY;
    const cx = containerRect ? clientX - containerRect.left : clientX;
    const cy = containerRect ? clientY - containerRect.top : clientY;
    const evWp = screenToWorld(cx, cy, camera, viewport);
    const evIp = isNb ? clampToSheet(evWp.x, evWp.y, pageDim.width, pageDim.height) : evWp;

    // R1-DEF-05: Accurate light pressure; speed fallback only when not pen
    const isPenEvent = ev.pointerType === "pen";
    let rawP: number;
    if (isPenEvent) {
      if (ev.pressure > 0) lastPenPressureRef.current = ev.pressure;
      rawP = lastPenPressureRef.current;
    } else {
      rawP = ev.pressure > 0 ? ev.pressure : computePressureFromSpeed(live.points);
    }
    const p = calibratePressure(rawP, sensitivity, curve);

    // Append relative coordinate to transient buffer
    live.points.push([evIp.x - live.originX, evIp.y - live.originY, p]);
    ptsToAdd.push([evIp.x, evIp.y, p]);
  }

  // R1-DEF-04: Speculative Hardware Prediction via getPredictedEvents()
  const rawPredicted: PointerEvent[] =
    typeof native?.getPredictedEvents === "function"
      ? native.getPredictedEvents() || []
      : [];

  const newPredictedPoints: Array<[number, number, number]> = [];
  if (rawPredicted.length > 0 && ptsToAdd.length > 0) {
    const lastReal = ptsToAdd[ptsToAdd.length - 1];
    const lastCalibratedP = lastReal[2];

    for (const pe of rawPredicted.slice(0, 3)) {
      const peClientX = pe.clientX;
      const peClientY = pe.clientY;
      const peCX = containerRect ? peClientX - containerRect.left : peClientX;
      const peCY = containerRect ? peClientY - containerRect.top : peClientY;
      const peWp = screenToWorld(peCX, peCY, camera, viewport);
      const peIp = isNb ? clampToSheet(peWp.x, peWp.y, pageDim.width, pageDim.height) : peWp;

      // Outlier & deadband guard: between 1px and 40px from latest real hardware point
      const distFromLast = Math.hypot(peIp.x - lastReal[0], peIp.y - lastReal[1]);
      if (distFromLast >= 1.0 && distFromLast <= 40.0) {
        const isPredPen = pe.pointerType === "pen";
        const predP =
          isPredPen && pe.pressure > 0
            ? calibratePressure(pe.pressure, sensitivity, curve)

```

```

        : lastCalibratedP;
        newPredictedPoints.push([peIp.x - live.originX, peIp.y - live.originY, predP]);
    }
}
}
// Replace previous predictions with new frame's predictions
predictedPointsRef.current = newPredictedPoints;

// 2. Draw-and-hold auto-straighten detection with 5px deadband (R1-DEF-08 & D4)
if (touchSettings.penAutoStraighten !== false) {
    const lastPt = live.points[live.points.length - 1];
    const lastWorldX = lastPt[0] + live.originX;
    const lastWorldY = lastPt[1] + live.originY;

    const anchor = autoStraightenAnchorRef.current;
    const distFromAnchor = anchor
        ? Math.hypot(lastWorldX - anchor.x, lastWorldY - anchor.y)
        : Infinity;

    // A held pen drifts a few SCREEN pixels; in world units that depends
    // on the zoom, so a fixed world distance was too tight zoomed out.
    if (distFromAnchor > HOLD_SLOP_PX / camera.zoom) {
        // Intentional movement: reset deadband anchor and rearm timer
        autoStraightenAnchorRef.current = { x: lastWorldX, y: lastWorldY };
        clearAutoStraightenTimer();

        if (!isAutoStraightenedRef.current) {
            autoStraightenTimerRef.current = setTimeout(() => {
                if (!liveDraftStrokeRef.current) return;
                const stroke = liveDraftStrokeRef.current;
                // The samples recorded while the pen was held still add length
                // the line never had - a ruler-straight line plus a one-second
                // hold measured as a curve - so they are left out of the test.
                const drawn = trimHoldTail(stroke.points, (HOLD_SLOP_PX * 1.5) / camera.zoom);
                const isStraight = isStrokeStraight(
                    drawn,
                    tool === "highlighter" ? 0.82 : 0.88
                );
                if (isStraight) {
                    isAutoStraightenedRef.current = true;
                    stroke.points = straightenStrokePoints(drawn, 8);
                    applyLiveDraftStroke();
                }
            }, 550);
        }
    }

    // If already auto-straightened, rubber-band straight line to current nib position
    if (isAutoStraightenedRef.current) {
        const first = live.points[0];
        const relX = ip.x - live.originX;
        const relY = ip.y - live.originY;
        const avgP = live.points[0][2] ?? 0.5;
        live.points = straightenStrokePoints([first, [relX, relY, avgP]], 8);
    }
}

// 3. Batch DOM updates to VSYNC via requestAnimationFrame
// ZERO React renders (setDraftElement completely removed)
scheduleDraftFlush();
return;
}

// Shape Creation Drag
if (interaction.kind === "creating-shape" && draftElement) {
    let startX = interaction.startX;
    let startY = interaction.startY;
    let currX = ip.x;
    let currY = ip.y;

    if (snapToGridActive) {
        startX = snapToGrid(startX, paperSize);
        startY = snapToGrid(startY, paperSize);
        currX = snapToGrid(currX, paperSize);
        currY = snapToGrid(currY, paperSize);
    }

    const minX = Math.min(startX, currX);

```

```

const minY = Math.min(startY, curY);
const width = Math.abs(currX - startX);
const height = Math.abs(currY - startY);

if (draftElement.type === "line" || draftElement.type === "arrow") {
  setDraftElement({
    ...draftElement,
    x: startX,
    y: startY,
    x2: currX - startX,
    y2: currY - startY,
  } as any);
} else {
  setDraftElement({
    ...draftElement,
    x: minX,
    y: minY,
    width: Math.max(width, 2),
    height: Math.max(height, 2),
  } as any);
}
return;
}

// Moving elements
if (interaction.kind === "moving") {
  const dx = wp.x - interaction.startWorldX;
  const dy = wp.y - interaction.startWorldY;

  for (const orig of interaction.originalElements) {
    let newX = orig.x + dx;
    let newY = orig.y + dy;
    if (snapToGridActive) {
      newX = snapToGrid(newX, paperSize);
      newY = snapToGrid(newY, paperSize);
    }
    // Dragging something off the sheet would hide it: the desk is not
    // exported, printed, or reachable once you scroll. Translate it back
    // rather than resize it, so the drawing keeps its shape.
    if (isNb) {
      const b = getElementBounds(orig);
      const fixed = clampBoxToSheet(
        newX,
        newY,
        b.width,
        b.height,
        pageDim.width,
        pageDim.height
      );
      newX = fixed.x;
      newY = fixed.y;
    }
    updateElement(orig.id, { x: newX, y: newY }, false);
  }
  return;
}

// Resizing element
if (interaction.kind === "resizing") {
  const { handle, originalElement, startWorldX, startWorldY } = interaction;
  const dx = wp.x - startWorldX;
  const dy = wp.y - startWorldY;

  // 1. Line & Arrow: Update (x2, y2) vector correctly instead of treating as bounding box
  if (originalElement.type === "line" || originalElement.type === "arrow") {
    const b = getElementBounds(originalElement);
    const p1 = { x: originalElement.x, y: originalElement.y };
    const p2 = { x: originalElement.x + (originalElement as any).x2, y: originalElement.y + (originalElement as any).y2 };
    const oldW = Math.max(b.width, 1);
    const oldH = Math.max(b.height, 1);

    let newX = b.x;
    let newY = b.y;
    let newW = b.width;
    let newH = b.height;

    if (handle.includes("e")) newW = Math.max(b.width + dx, 10);

```

```

    if (handle.includes("s")) newH = Math.max(b.height + dy, 10);
    if (handle.includes("w")) {
      const potentialW = b.width - dx;
      if (potentialW >= 10) {
        newX = b.x + dx;
        newW = potentialW;
      }
    }
    if (handle.includes("n")) {
      const potentialH = b.height - dy;
      if (potentialH >= 10) {
        newY = b.y + dy;
        newH = potentialH;
      }
    }
  }

  const u1 = (p1.x - b.x) / oldW;
  const v1 = (p1.y - b.y) / oldH;
  const u2 = (p2.x - b.x) / oldW;
  const v2 = (p2.y - b.y) / oldH;

  const newP1x = newX + u1 * newW;
  const newP1y = newY + v1 * newH;
  const newP2x = newX + u2 * newW;
  const newP2y = newY + v2 * newH;

  updateElement(
    originalElement.id,
    {
      x: newP1x,
      y: newP1y,
      x2: newP2x - newP1x,
      y2: newP2y - newP1y,
    } as any,
    false
  );
  return;
}

// 2. Rotated element resizing: compute handle displacement in local rotated coordinate space
const rot = originalElement.rotation || 0;
if (rot !== 0) {
  const cos = Math.cos(rot);
  const sin = Math.sin(rot);
  const localDx = dx * cos + dy * sin;
  const localDy = -dx * sin + dy * cos;

  const origW = (originalElement as any).width || 100;
  const origH = (originalElement as any).height || 100;
  const origCx = originalElement.x + origW / 2;
  const origCy = originalElement.y + origH / 2;

  let newW = origW;
  let newH = origH;
  let localCenterShiftX = 0;
  let localCenterShiftY = 0;

  if (handle.includes("e")) {
    newW = Math.max(origW + localDx, 20);
    localCenterShiftX += (newW - origW) / 2;
  }
  if (handle.includes("w")) {
    newW = Math.max(origW - localDx, 20);
    localCenterShiftX -= (newW - origW) / 2;
  }
  if (handle.includes("s")) {
    newH = Math.max(origH + localDy, 20);
    localCenterShiftY += (newH - origH) / 2;
  }
  if (handle.includes("n")) {
    newH = Math.max(origH - localDy, 20);
    localCenterShiftY -= (newH - origH) / 2;
  }
}

// Photos keep their proportions on a corner drag (see lockAspect).
if (originalElement.type === "image" && handle.length === 2) {
  const locked = lockAspect(origW, origH, newW, newH);
  newW = locked.width;

```

```

    newH = locked.height;
    localCenterShiftX = ((handle.includes("e") ? 1 : -1) * (newW - origW)) / 2;
    localCenterShiftY = ((handle.includes("s") ? 1 : -1) * (newH - origH)) / 2;
  }

  const worldCenterShiftX = localCenterShiftX * cos - localCenterShiftY * sin;
  const worldCenterShiftY = localCenterShiftX * sin + localCenterShiftY * cos;

  const newCx = origCx + worldCenterShiftX;
  const newCy = origCy + worldCenterShiftY;

  const newX = newCx - newW / 2;
  const newY = newCy - newH / 2;

  updateElement(
    originalElement.id,
    { x: newX, y: newY, width: newW, height: newH } as any,
    false
  );
  return;
}

// 3. Unrotated box resizing
const b = getElementBounds(originalElement);
let newX = b.x;
let newY = b.y;
let newW = b.width;
let newH = b.height;

if (handle.includes("e")) newW = Math.max(b.width + dx, 20);
if (handle.includes("s")) newH = Math.max(b.height + dy, 20);
if (handle.includes("w")) {
  const potentialW = b.width - dx;
  if (potentialW >= 20) {
    newX = b.x + dx;
    newW = potentialW;
  }
}
if (handle.includes("n")) {
  const potentialH = b.height - dy;
  if (potentialH >= 20) {
    newY = b.y + dy;
    newH = potentialH;
  }
}

// Photos keep their proportions on a corner drag, anchored at the
// opposite corner so the one you hold follows your finger.
if (originalElement.type === "image" && handle.length === 2) {
  const locked = lockAspect(b.width, b.height, newW, newH);
  newW = locked.width;
  newH = locked.height;
  newX = handle.includes("w") ? b.x + b.width - newW : b.x;
  newY = handle.includes("n") ? b.y + b.height - newH : b.y;
}

if (snapToGridActive) {
  newX = snapToGrid(newX, paperSize);
  newY = snapToGrid(newY, paperSize);
  newW = Math.max(snapToGrid(newW, paperSize), paperSize);
  newH = Math.max(snapToGrid(newH, paperSize), paperSize);
}

updateElement(originalElement.id, { x: newX, y: newY, width: newW, height: newH } as any, false);
return;
}

// Rotating element
if (interaction.kind === "rotating") {
  const { originalElement, startAngle, centerX, centerY } = interaction;
  const currentAngle = Math.atan2(wp.y - centerY, wp.x - centerX);
  const deltaAngle = currentAngle - startAngle;
  // Normalize to [0, 2π) so stored rotations are never negative
  const newRotation =
    (((originalElement.rotation || 0) + deltaAngle) % (Math.PI * 2) + Math.PI * 2) %
    (Math.PI * 2);
  // Snaps to straight within ~4 degrees: a photo you meant to leave level
  // otherwise ends up a hair crooked every time you touch the handle.

```



```

    updateElement(originalElement.id, { rotation: snapRotation(newRotation) }, false);
    return;
}

// Rubber band selection
if (interaction.kind === "selecting") {
    const x = Math.min(interaction.startX, cp.x);
    const y = Math.min(interaction.startY, cp.y);
    const w = Math.abs(cp.x - interaction.startX);
    const h = Math.abs(cp.y - interaction.startY);
    setRubberBand({ x, y, w, h });

    const p1 = screenToWorld(x, y, camera, viewport);
    const p2 = screenToWorld(x + w, y + h, camera, viewport);
    const hitIds: string[] = [];

    for (const el of elements) {
        if (el.type === "connection") continue;
        // Locked elements can't be rubber-band selected - except in locked-items mode, where ONLY they can.
        if (!pickableNow(el, lockedMode)) continue;
        if (!lockedMode && !passesSelectFilter(el, selectFilter)) continue;
        const b = getElementBounds(el);
        if (b.x + b.width >= p1.x && b.x <= p2.x && b.y + b.height >= p1.y && b.y <= p2.y) {
            hitIds.push(el.id);
        }
    }
    setSelected(hitIds);
}
};

// Pointer Up
/**
 * Drop every per-pointer gesture record for a pointer that has gone away.
 * A long-press timer left armed after the finger lifted would select an
 * object half a second later, with nothing on the glass to explain it.
 */
const releaseGestureRefs = (pointerId: number) => {
    // Re-arm for the NEXT press. Radix reads `disabled` when the press starts,
    // so arming mid-press is too late – disarming mid-press still works, since
    // Radix cancels its pending timer as soon as the prop flips.
    // Still gated on there being a selection for the menu to act on.
    armContextMenu(selectedIds.length > 0);
    if (longPressRef.current?.pointerId === pointerId) cancelLongPress();
    if (pendingMoveRef.current?.pointerId === pointerId) pendingMoveRef.current = null;
    if (touchInteractionRef.current?.pointerId === pointerId) touchInteractionRef.current = null;
};

/**
 * Set once the Android bridge has spoken. From then on the NATIVE signal
 * alone decides press and release: the strokes it forwards have their button
 * bits stripped (see MainActivity), so the web-side checks would read every
 * one of them as "button released" and hand the pen back mid-erase - and a
 * nib lifted between two erase strokes would do the same while the button is
 * still held.
 */
const nativeStylusRef = useRef(
    typeof window !== "undefined" &&
    (window as unknown as { Capacitor?: { getPlatform?: () => string } }).Capacitor?.getPlatform?.() ===
    "android"
);

/** Give the pen back after a stylus-button erase. */
const releaseStylusErase = (fromNative = false) => {
    if (nativeStylusRef.current && !fromNative) return;
    const restore = stylusEraseRestoreRef.current;
    if (!restore) return;
    stylusEraseRestoreRef.current = null;
    setTool(restore);
    // Restoring the tool is not enough on its own. An "erasing" interaction
    // keeps erasing from the move handler REGARDLESS of the current tool, so
    // without this the eraser would run on under the restored pen until the
    // nib lifted – exactly the "stuck erasing" the button release is meant
    // to end.
    lastErasePosRef.current = null;
    setInteraction((prev) => (prev.kind === "erasing" ? { kind: "idle" } : prev));
};

/**

```

```

* Borrow the eraser while the stylus barrel button (or the eraser end) is
* held, and hand the pen back the moment it is released.
*
* Called from pointerdown AND pointermove so the button alone drives it:
* press while hovering and the eraser is already armed before you touch the
* paper; let go and you are back on your pen mid-stroke.
*/
const engageStylusErase = () => {
  if (touchSettings.stylusButtonErases === false) return;
  if (stylusEraseRestoreRef.current !== null) return; // already borrowed
  // Already erasing by choice: there is nothing to borrow, and recording a
  // restore target here would strand the user in the eraser on release.
  if (tool === "eraser") return;
  // Pointing and reading tools have nothing to erase with: holding the
  // button while presenting must not wipe real ink.
  if (tool === "laser" || tool === "textselect") return;
  stylusEraseRestoreRef.current = tool;
  setTool("eraser");
};

const syncStylusErase = (e: React.PointerEvent<SVGSVGElement>) => {
  if (touchSettings.stylusButtonErases === false) return;
  if (nativeStylusRef.current) return; // Android: the bridge decides
  const action = stylusEraseTransition(stylusEraseRestoreRef.current !== null, e);
  if (action === "engage") engageStylusErase();
  else if (action === "release") releaseStylusErase();
};

// The NATIVE route, and on Android the only one that works.
//
// Chromium's WebView does not surface the S Pen side button to web content at
// all – the in-app diagnostic showed no event whatsoever while it was held.
// MainActivity.java reads Android's own MotionEvent.getButtonState() and
// dispatches this CustomEvent, so the gesture is driven from outside the web
// layer entirely. On Windows the pointer-event route above still does the job
// and this listener simply never fires.
const stylusBridgeRef = useRef({ engage: () => {}, release: () => {} });
stylusBridgeRef.current = {
  engage: engageStylusErase,
  release: () => releaseStylusErase(true),
};

useEffect(() => {
  const onNativeButton = (e: Event) => {
    nativeStylusRef.current = true;
    const pressed = Boolean((e as CustomEvent).detail?.pressed);
    if (pressed) stylusBridgeRef.current.engage();
    else stylusBridgeRef.current.release();
  };
  window.addEventListener("pronotes:stylusbutton", onNativeButton);
  return () => window.removeEventListener("pronotes:stylusbutton", onNativeButton);
}, []);

/**
* The pen can leave hover range with the button still down – lift it away
* from the glass and no pointerup or pointercancel ever arrives. Without
* this the borrowed eraser would still be held when the pen came back.
*/
const handlePointerLeave = (e: React.PointerEvent<SVGSVGElement>) => {
  laser.hideHover();
  placeEraserCursor(0, 0, false); // pen out of hover range, or mouse gone
  placePenHoverCursor(0, 0, false);
  clearAutoStraightenTimer();
  isAutoStraightenedRef.current = false;
  if (!gapPlusTimerRef.current) showGapPlus(null);
  if (e.pointerType === "pen") releaseStylusErase();
};

/**
* Turns a drawn area into a capture and opens the capture panel. A tap, a
* flick or a line is not an area: nothing opens and the mode stays armed,
* so you simply draw again.
*/
const finishCapture = (drawn: Extract<InteractionMode, { kind: "capturing" }>) => {
  if (!activeBoard) return;
  const pts = drawn.points;
  const zoom = Math.max(camera.zoom, 0.01);
  let drawnRect: ReturnType<typeof boundsOfPoints> = null;

```

```

let polygon: Array<{ x: number; y: number }> | undefined;
if (drawn.shape === "rectangle") {
  if (pts.length >= 2) drawnRect = rectFromCorners(pts[0], pts[pts.length - 1]);
} else if (pts.length >= 3 && polygonArea(pts) * zoom * zoom >= 24 * 24) {
  drawnRect = boundsOfPoints(pts);
  polygon = pts;
}
// Judged on screen: 12 px is a deliberate drag at any zoom.
if (!drawnRect || drawnRect.width * zoom < 12 || drawnRect.height * zoom < 12) return;
const anchorIndex = isNb ? Math.max(0, activeIndex) : 0;
const region = clampCaptureRegion(activeBoard, anchorIndex, drawnRect);
if (!region || region.width < CAPTURE_MIN_SIDE || region.height < CAPTURE_MIN_SIDE) return;
useCapture.getState().open({
  source: { board: activeBoard, region, polygon, anchorIndex },
  origin: "area",
  shape: drawn.shape,
  zoom: camera.zoom,
  dpr: typeof window !== "undefined" ? window.devicePixelRatio || 1 : 1,
});
};

// ---- Shape recognition -----
// With it on, a finished pen stroke that is clearly a shape becomes that
// exact shape, where you drew it; a note offers the stroke back as drawn.
const [shapeNote, setShapeNote] = useState<ShapeNote | null>(null);
const shapeOriginalRef = useRef<{
  id: string;
  points: StrokeElement["points"];
  /** Earlier lines this one was joined to, as they were before. */
  joined: Array<{ id: string; points: StrokeElement["points"]; line: [[number, number], [number, number]] }>
  | null>(null);
const clearShapeNote = useCallback(() => setShapeNote(null), []);
/**
 * Shapes made recently on this page, in world units, for the next line to
 * join: a line's ends, a polygon's corners.
 */
const recentShapesRef = useRef<
  Array<{ id: string; place: string; at: number; line?: [[number, number], [number, number]]; corners?: [num
ber, number][] }>
>([]);
/** Earlier lines to move once the new line is in (the same Undo step). */
const pendingJoinRef = useRef<Array<{ id: string; points: StrokeElement["points"] }>>([]);

const asRecognizedShape = (stroke: StrokeElement): StrokeElement => {
  pendingJoinRef.current = [];
  if (!useWhiteboard.getState().shapeRecognition) return stroke;
  // Too small to be meant as a shape: a letter, a dot. Measured on screen.
  const zoom = Math.max(camera.zoom, 0.01);
  const r = recognizeShape(stroke.points, { minSize: 28 / zoom });
  if (!r) return stroke;
  const place = `${activeBoardId}:${activePage?.id ?? ""}`;
  const now = Date.now();
  const toWorld = (p: [number, number]): [number, number] => [stroke.x + p[0], stroke.y + p[1]];
  const live = new Map(
    getBoardElements(useWhiteboard.getState().boards.find((b) => b.id === activeBoardId) ?? null).map((el) =>
      > [el.id, el])
  );
  // Only what is still there, on this page, from the last two minutes.
  recentShapesRef.current = recentShapesRef.current.filter((t) => live.has(t.id) && now - t.at < 120_000).sl
ice(-20);
  let points = r.points;
  let vertices = r.vertices.map(toWorld);
  const joined: Array<{ id: string; points: StrokeElement["points"]; line: [[number, number], [number, numbe
r]] }> = [];
  if (r.kind === "line") {
    const len = Math.hypot(vertices[1][0] - vertices[0][0], vertices[1][1] - vertices[0][1]);
    // Within reach: about a fingertip on screen, more for a long line.
    const reach = Math.min(44, Math.max(18, len * zoom * 0.12)) / zoom;
    const targets: JoinTarget[] = recentShapesRef.current.filter((t) => t.place === place);
    const j = joinLine(vertices[0], vertices[1], targets, reach);
    if (j.start !== vertices[0] || j.end !== vertices[1]) {
      const pr = r.points[0]?.[2] ?? 0.5;
      points = lineBetween([j.start[0] - stroke.x, j.start[1] - stroke.y], [j.end[0] - stroke.x, j.end[1] -
stroke.y], pr);
      vertices = [j.start, j.end];
    }
    for (const a of j.adjust) {

```

```

    const el = live.get(a.id);
    const entry = recentShapesRef.current.find((t) => t.id === a.id);
    if (!el || el.type !== "stroke" || !entry?.line) continue;
    const ends: [[number, number], [number, number]] = [entry.line[0], entry.line[1]];
    ends[a.end] = a.to;
    joined.push({ id: el.id, points: el.points, line: [entry.line[0], entry.line[1]] });
    const pr = el.points[0]?.[2] ?? 0.5;
    pendingJoinRef.current.push({
      id: el.id,
      points: lineBetween([ends[0][0] - el.x, ends[0][1] - el.y], [ends[1][0] - el.x, ends[1][1] - el.y],
        pr),
    });
    entry.line = ends;
  }
}
recentShapesRef.current.push(
  r.kind === "line"
    ? { id: stroke.id, place, at: now, line: [vertices[0], vertices[1]] }
    : { id: stroke.id, place, at: now, corners: vertices }
);
shapeOriginalRef.current = { id: stroke.id, points: stroke.points, joined };
let minX = Infinity;
let minY = Infinity;
let maxX = -Infinity;
for (const [x, y] of r.points) {
  minX = Math.min(minX, x);
  maxX = Math.max(maxX, x);
  minY = Math.min(minY, y);
}
const top = worldToScreen(stroke.x + (minX + maxX) / 2, stroke.y + minY, camera, viewport);
setShapeNote({
  id: stroke.id,
  label: shapeLabel(r.kind),
  x: Math.max(90, Math.min(viewport.width - 90, top.x)),
  y: Math.max(8, top.y - 44),
  key: Date.now(),
});
// The exact shape: drawn as it is (no smoothing, no hook trimming) and
// with blunt ends, so a closed shape has no thin spot where it closes.
return { ...stroke, points, settled: true, cutStart: true, cutEnd: true };
};

/** Moves the ends of earlier lines a new one was joined to. After the new
 * stroke is added, so its Undo step takes them back too. */
const applyPendingJoins = () => {
  for (const j of pendingJoinRef.current) updateElement(j.id, { points: j.points } as Partial<StrokeElement>, false);
  pendingJoinRef.current = [];
};

const keepShapeAsDrawn = useCallback(() => {
  const orig = shapeOriginalRef.current;
  setShapeNote(null);
  if (!orig) return;
  shapeOriginalRef.current = null;
  updateElement(
    orig.id,
    { points: orig.points, settled: undefined, cutStart: undefined, cutEnd: undefined } as Partial<StrokeElement>,
    false
  );
  // The lines it was joined to go back to how they were, too.
  for (const j of orig.joined) {
    updateElement(j.id, { points: j.points } as Partial<StrokeElement>, false);
    const entry = recentShapesRef.current.find((t) => t.id === j.id);
    if (entry) entry.line = j.line;
  }
  recentShapesRef.current = recentShapesRef.current.filter((t) => t.id !== orig.id);
}, [updateElement]);

const handlePointerUp = (e: React.PointerEvent<SVGSVGElement>) => {
  laser.up(e.pointerId);
  clearAutoStraightenTimer();
  isAutoStraightenedRef.current = false;
  // A lifted finger is gone; a lifted pen is still hovering, so it keeps it.
  if (e.pointerType === "touch") {
    placeEraserCursor(0, 0, false);
    placePenHoverCursor(0, 0, false);
  }
};

```

```

}
// Apply whatever movement has not been flushed yet, so the camera ends
// exactly where the finger left it rather than up to one frame behind.
const wasPanning =
  interaction.kind === "panning" || activeTouchesRef.current.size >= 2;
// Only when the LAST finger lifts - lifting one of two is not a throw.
const lastContact = activeTouchesRef.current.size <= 1;
endPanGesture();
if (wasPanning && lastContact) startMomentum();

// End the erase sweep; a later event must not continue from here.
lastErasePosRef.current = null;
activeTouchesRef.current.delete(e.pointerId);
rejectedPalmsRef.current.delete(e.pointerId);
releaseGestureRefs(e.pointerId);
if (e.pointerType === "pen") {
  penActiveRef.current = false;
  // The suppression window is measured from here - the lift between words,
  // with the hand still resting, is exactly the gap it exists to cover.
  lastPenAtRef.current = Date.now();
  // Hand the pen back the moment the stylus leaves the glass, whether the
  // button was released first or not - otherwise letting go mid-stroke
  // would strand the user in the eraser.
  releaseStylusErase();
}

// Multi-finger tap gestures (2-finger = Undo, 3-finger = Redo) are judged
// exactly ONCE - when the last finger lifts - and then consumed, so a
// 2-finger tap can never fire undo twice (once per finger lift).
// Never while the inline editor is open (mid-edit tap would revert the
// whole session), and never if this touch sequence already sealed a
// gesture (pinch takeover commits - undoing it right away feels random).
if (
  activeTouchesRef.current.size === 0 &&
  touchSettings.twoFingerUndo &&
  maxTouchCountRef.current >= 2 &&
  !editingId &&
  !gestureSealedRef.current
) {
  const touchDuration = Date.now() - touchStartTimeRef.current;
  if (touchDuration < 320 && !touchMovedRef.current) {
    if (maxTouchCountRef.current === 2) {
      undo();
    } else if (maxTouchCountRef.current === 3) {
      redo();
    }
  }
  maxTouchCountRef.current = 0;
  touchMovedRef.current = false;
  gestureSealedRef.current = false;
}

if (activeTouchesRef.current.size < 2) {
  lastTouchMidRef.current = null;
  pinchStartDistanceRef.current = null;
  pinchEngagedRef.current = false;
}
if (activeTouchesRef.current.size === 0) {
  maxTouchCountRef.current = 0;
  pinchStartDistanceRef.current = null;
  pinchEngagedRef.current = false;
  lastTouchMidRef.current = null;
}

// Ends a text selection drag; the interaction below is idle for it.
textSelect.pointerUp(e);

if (interaction.kind === "erasing") {
  setInteraction({ kind: "idle" });
  return;
}

if (interaction.kind === "capturing") {
  // Only the pointer that drew the area ends it; a palm lifting does not.
  if (e.pointerId !== interaction.pointerId) return;
  finishCapture(interaction);
  setInteraction({ kind: "idle" });
  return;
}

```

```

}

if (interaction.kind === "lasso-selecting") {
  // The pen has lifted, so the loop closes – joining the last point back
  // to the first, exactly as Samsung does – and only NOW is there a
  // region to judge anything against.
  const loop = interaction.points;
  if (loop.length >= 3) {
    const hitIds: string[] = [];
    for (const el of elements) {
      // Connections live at (0,0) with zero-size bounds – lassoing the
      // origin would otherwise select EVERY connection on the board.
      // Locked elements are never selected by lasso unless in lockedMode.
      if (el.type === "connection" || !pickableNow(el, lockedMode)) continue;
      if (!lockedMode && !passesSelectFilter(el, selectFilter)) continue;
      // `strict` is "include partially selected objects", OFF: only what
      // the loop encloses completely comes along.
      const inside = interaction.strict
        ? isElementFullyInPolygon(el, loop)
        : isElementInPolygon(el, loop);
      if (inside) hitIds.push(el.id);
    }
    setSelected(hitIds);
  } else {
    // A tap or a flick is not a loop.
    setSelected([]);
  }
  setInteraction({ kind: "idle" });
  return;
}

// 1. Finish connection
if (interaction.kind === "connecting") {
  if (interaction.targetElementId && activeBoardId) {
    createConnection(
      {
        boardId: activeBoardId,
        elementId: interaction.sourceElementId,
        anchor: interaction.sourceAnchor,
      },
      {
        boardId: activeBoardId,
        elementId: interaction.targetElementId,
        anchor: interaction.targetAnchor || "auto",
      },
      style.connectionLinkType,
      style.connectionRouting
    );
  }
  setInteraction({ kind: "idle" });
  return;
}

// 2. Finish freehand drawing (commits points cleanly from transient ref to Zustand store)
if (liveDraftStrokeRef.current || interaction.kind === "drawing") {
  // R1-DEF-02: Ignore pointerup from non-drawing pointers (e.g. palm lifting off)
  if (drawingPointerIdRef.current !== null && e.pointerId !== drawingPointerIdRef.current) {
    return;
  }

  // Cancel any pending RAF
  if (draftStrokeRafRef.current !== null) {
    cancelAnimationFrame(draftStrokeRafRef.current);
    draftStrokeRafRef.current = null;
  }

  clearAutoStraightenTimer();
  isAutoStraightenedRef.current = false;
  autoStraightenAnchorRef.current = null;
  drawingPointerIdRef.current = null;
  predictedPointsRef.current = [];

  const live = liveDraftStrokeRef.current;
  if (live && live.points.length >= 1) {
    const cleanPoints =
      touchSettings.penAntiHook !== false
        ? pruneTerminalHooks(live.points)
        : live.points;
  }
}

```

```

const finalStroke: StrokeElement = {
  id: live.id,
  type: "stroke",
  x: live.originX,
  y: live.originY,
  rotation: 0,
  opacity: 1,
  points: cleanPoints,
  color: live.color,
  strokeWidth: live.strokeWidth,
  strokeStyle: live.strokeStyle,
  smoothing: live.smoothing,
  createdAt: live.createdAt,
  updatedAt: Date.now(),
};

// Commit to Zustand store and push undo/redo snapshot. The save
// this schedules waits until the pen has been quiet a moment.
deferSavesWhileWriting();
addElement(asRecognizedShape(finalStroke), true);
applyPendingJoins();
} else if (draftElement && draftElement.type === "stroke" && (draftElement as StrokeElement).points.length
  th >= 1) {
  const stroke = draftElement as StrokeElement;
  const cleanPoints =
    touchSettings.penAntiHook !== false
      ? pruneTerminalHooks(stroke.points)
      : stroke.points;
  const finalStroke: StrokeElement = {
    ...stroke,
    points: cleanPoints,
    smoothing: penFeelFor(touchSettings, stroke.strokeStyle).smoothing,
  };
  addElement(asRecognizedShape(finalStroke), true);
  applyPendingJoins();
}

// Hide and clear the direct DOM live overlay
if (liveDraftGroupRef.current) {
  liveDraftGroupRef.current.style.display = "none";
}
if (liveDraftPathRef.current) {
  liveDraftPathRef.current.setAttribute("d", "");
}
liveDraftStrokeRef.current = null;

setDraftElement(null);
setInteraction({ kind: "idle" });
return;
}

// 3. Finish shape creation
if (interaction.kind === "creating-shape" && draftElement) {
  const b = getElementBounds(draftElement);
  let final: WhiteboardElement | null = null;
  if (b.width > TAP_DRAG_SLOP_PX || b.height > TAP_DRAG_SLOP_PX) {
    final = draftElement;
  } else {
    // A tap, not a drag: plant a default-size shape centred on the press
    // instead of discarding it – or worse, committing the finger's
    // contact wobble as a thin sliver. Centred, snapped like a drag, and
    // never born bigger than the sheet.
    const rect = tapShapeRect(interaction.startX, interaction.startY, {
      snap: snapToGridActive ? (v: number) => snapToGrid(v, paperSize) : undefined,
      page: isNb ? { width: pageDim.width, height: pageDim.height } : null,
    });
    if (draftElement.type === "line" || draftElement.type === "arrow") {
      final = { ...draftElement, x: rect.x, y: rect.y, x2: rect.width, y2: 0 };
    } else if (
      draftElement.type === "rectangle" ||
      draftElement.type === "ellipse" ||
      draftElement.type === "diamond" ||
      draftElement.type === "triangle" ||
      draftElement.type === "star" ||
      draftElement.type === "cloud"
    ) {

```

```

    final = { ...draftElement, x: rect.x, y: rect.y, width: rect.width, height: rect.height };
  }
  // Any other type cannot reach this interaction (only the eight shape
  // tools start it) and stays discarded, exactly as before.
}
if (final) {
  addElement(final, true);
  setSelected([final.id]);
}
setDraftElement(null);
setInteraction({ kind: "idle" });
// Back to select: the shape is selected with its handles out, so resize
// and move work at once – staying in the shape tool meant every press
// drew a new shape instead of grabbing the one just made.
setTool("select");
return;
}

// 4. Finish moving / resizing / rotating
if (interaction.kind === "moving" || interaction.kind === "resizing" || interaction.kind === "rotating") {
  commitHistory();
  setInteraction({ kind: "idle" });
  return;
}

if (interaction.kind === "selecting") {
  setRubberBand(null);
  setInteraction({ kind: "idle" });
  return;
}

setInteraction({ kind: "idle" });
};

// Pointer Cancel: the browser cancelled the gesture (palm takeover, edge
// swipe, alert dialog). Cancelled gestures ROLL BACK – drafts are discarded
// and in-flight transforms restored – instead of being committed to history.
const handlePointerCancel = (e: React.PointerEvent<SVGSVGElement>) => {
  laser.up(e.pointerId);
  // A palm the system cancels must not undo the pen's text drag.
  textSelect.cancel(e.pointerId);
  endPanGesture();
  // End the erase sweep; a later event must not continue from here.
  lastErasePosRef.current = null;
  activeTouchesRef.current.delete(e.pointerId);
  rejectedPalmsRef.current.delete(e.pointerId);
  releaseGestureRefs(e.pointerId);
  if (e.pointerType === "pen") {
    penActiveRef.current = false;
    // The suppression window is measured from here – the lift between words,
    // with the hand still resting, is exactly the gap it exists to cover.
    lastPenAtRef.current = Date.now();
    // Hand the pen back the moment the stylus leaves the glass, whether the
    // button was released first or not – otherwise letting go mid-stroke
    // would strand the user in the eraser.
    releaseStylusErase();
  }
}

if (activeTouchesRef.current.size < 2) {
  lastTouchMidRef.current = null;
  pinchStartDistanceRef.current = null;
  pinchEngagedRef.current = false;
}
if (activeTouchesRef.current.size === 0) {
  maxTouchCountRef.current = 0;
  touchMovedRef.current = false;
}

// A cancelled capture drag is dropped, but the mode stays armed so the
// area can simply be drawn again. A cancelled palm leaves it alone.
if (interaction.kind === "capturing") {
  if (e.pointerId !== interaction.pointerId) return;
  setInteraction({ kind: "idle" });
  return;
}

// R1-DEF-02: Strict Pointer ID Isolation for Inking Cancellation
// If Android or the browser cancels a palm contact (pointercancel on touch pointer),

```



```

// the in-flight pen stroke MUST continue unaffected!
if (interaction.kind === "drawing") {
  if (drawingPointerIdRef.current !== null && e.pointerId !== drawingPointerIdRef.current) {
    // OS cancelled a palm contact – do not cancel active pen inking!
    return;
  }
  drawingPointerIdRef.current = null;
  if (draftStrokeRafRef.current !== null) {
    cancelAnimationFrame(draftStrokeRafRef.current);
    draftStrokeRafRef.current = null;
  }
  clearAutoStraightenTimer();
  isAutoStraightenedRef.current = false;
  autoStraightenAnchorRef.current = null;
  predictedPointsRef.current = [];

  // Keep what was written: a cancelled stream still drew real ink.
  const cancelled = liveDraftStrokeRef.current;
  if (cancelled && cancelled.points.length >= 1) {
    addElement(
      {
        id: cancelled.id,
        type: "stroke",
        x: cancelled.originX,
        y: cancelled.originY,
        rotation: 0,
        opacity: 1,
        points:
          touchSettings.penAntiHook !== false
            ? pruneTerminalHooks(cancelled.points)
            : cancelled.points,
        color: cancelled.color,
        strokeWidth: cancelled.strokeWidth,
        strokeStyle: cancelled.strokeStyle,
        smoothing: cancelled.smoothing,
        createdAt: cancelled.createdAt,
        updatedAt: Date.now(),
      },
      true
    );
  }
  if (liveDraftGroupRef.current) {
    liveDraftGroupRef.current.style.display = "none";
  }
  if (liveDraftPathRef.current) {
    liveDraftPathRef.current.setAttribute("d", "");
  }
  liveDraftStrokeRef.current = null;

  setDraftElement(null);
  setInteraction({ kind: "idle" });
  return;
}

if (
  interaction.kind === "moving" ||
  interaction.kind === "resizing" ||
  interaction.kind === "rotating"
) {
  const originals =
    interaction.kind === "moving"
      ? interaction.originalElements
      : [interaction.originalElement];
  for (const orig of originals) {
    updateElement(orig.id, orig, false);
  }
}

setDraftElement(null);
setRubberBand(null);
setInteraction({ kind: "idle" });
};

// Safety net for fingers the canvas never heard lift. The touch map is
// cleared whenever the live document changes or the split opens or closes,
// and whenever the app is hidden: a press in progress cannot survive any of
// those, and one stale entry turns every later one-finger touch into half a
// pinch - scrolling zooms, drawing and typing do nothing - with no way out.

```

```

const splitBoardIdForReset = useWhiteboard((s) => s.splitBoardId);
const resetGestureState = useCallback(() => {
  activeTouchesRef.current.clear();
  rejectedPalmsRef.current.clear();
  drawingPointerIdRef.current = null;
  penActiveRef.current = false;
  lastTouchMidRef.current = null;
  pinchStartDistanceRef.current = null;
  pinchEngagedRef.current = false;
  maxTouchCountRef.current = 0;
  touchMovedRef.current = false;
  gestureSealedRef.current = false;
  lastErasePosRef.current = null;
  setInteraction((cur) => (cur.kind === "capturing" ? { kind: "idle" } : cur));
  endPanGesture();
}, [endPanGesture]);
useEffect(() => {
  resetGestureState();
}, [activeBoardId, splitBoardIdForReset, resetGestureState]);
useEffect(() => {
  const onHidden = () => {
    if (document.visibilityState === "hidden") resetGestureState();
  };
  document.addEventListener("visibilitychange", onHidden);
  window.addEventListener("blur", resetGestureState);
  return () => {
    document.removeEventListener("visibilitychange", onHidden);
    window.removeEventListener("blur", resetGestureState);
  };
}, [resetGestureState]);

// Safety net: if the browser silently retracts pointer capture mid-gesture
// (e.g. a captured node unmounts), drop the pointer from the touch map so a
// stale entry can never poison later gesture math.
const handleLostPointerCapture = (e: React.PointerEvent<SVGSVGElement>) => {
  laser.up(e.pointerId);
  // A capture drag whose pointer is gone without an "up" must not leave
  // Select dead (every later press would wait for that pointer).
  setInteraction((cur) =>
    cur.kind === "capturing" && cur.pointerId === e.pointerId ? { kind: "idle" } : cur
  );
  endPanGesture();
  activeTouchesRef.current.delete(e.pointerId);
  rejectedPalmsRef.current.delete(e.pointerId);
  if (drawingPointerIdRef.current === e.pointerId) {
    drawingPointerIdRef.current = null;
  }
  releaseGestureRefs(e.pointerId);
  if (e.pointerType === "pen") {
    penActiveRef.current = false;
    lastPenAtRef.current = Date.now();
  }
};

// Double click canvas to create text anywhere
const handleDoubleClick = (e: React.MouseEvent<SVGSVGElement>) => {
  if (tool !== "select" && tool !== "text") return;
  // Screenshot mode: two quick short drags are not "make a text box".
  if (captureArmed) return;
  const wp = getWorldPoint(e.clientX, e.clientY);
  const hit = findElementAt(wp.x, wp.y, false);
  // Double-clicking the desk beside a notebook page must not drop a text box
  // out there; it lands on the paper instead.
  const ip = getInkPoint(e.clientX, e.clientY);

  if (hit) {
    if (lockedMode) return; // locked things are picked here, never edited
    if (hit.type === "text" || hit.type === "sticky") {
      setEditingId(hit.id);
    } else if (hit.type === "image") {
      setImageStudioId(hit.id);
    }
  } else if (!lockedMode) {
    const textId = nanoid(8);
    const posX = snapToGridActive ? snapToGrid(ip.x, paperSize) : ip.x;
    const posY = snapToGridActive ? snapToGrid(ip.y, paperSize) : ip.y;
    const newText: TextElement = {
      id: textId,

```

```

    type: "text",
    x: posX,
    y: posY,
    width: 220,
    height: 48,
    rotation: 0,
    opacity: 1,
    text: "",
    color: style.textColor,
    fontSize: style.fontSize,
    fontFamily: style.fontFamily,
    fontWeight: style.fontWeight,
    align: style.textAlign,
    createdAt: Date.now(),
    updatedAt: Date.now(),
  };
  addElement(newText, true);
  setEditingId(textId);
}
};

const cursorStyle = useMemo(() => {
  if (interaction.kind === "panning") return "grabbing";
  if (spaceHeld || tool === "pan") return "grab";
  if (captureArmed) return "crosshair";
  switch (tool) {
    case "laser": return "none";
    case "textselect": return "text";
    case "select": return "default";
    case "lasso": return "crosshair";
    case "pen":
    case "highlighter":
    case "marker":
    case "brush":
    case "calligraphy":
    case "pencil": return "crosshair";
    case "eraser": return "cell";
    case "connect": return "crosshair";
    case "text": return "text";
    default: return "crosshair";
  }
}, [interaction.kind, spaceHeld, tool, captureArmed]);

const connectionDraft = useMemo(() => {
  if (interaction.kind !== "connecting" || !activeBoard) return null;
  // Endpoints live on the active page in a notebook; reading the canvas
  // field meant the draft connection silently never rendered.
  const sourceEl = getBoardElements(activeBoard).find((el) => el.id === interaction.sourceElementId);
  if (!sourceEl) return null;

  const sourceAnchor = getResolvedAnchorPoint(
    sourceEl,
    interaction.sourceAnchor,
    { x: interaction.currentWorldX, y: interaction.currentWorldY }
  );

  let targetAnchor = {
    x: interaction.currentWorldX,
    y: interaction.currentWorldY,
    nx: 0,
    ny: 0,
    anchor: "auto" as NodeAnchor,
  };

  if (interaction.targetElementId) {
    const targetEl = getBoardElements(activeBoard).find((el) => el.id === interaction.targetElementId);
    if (targetEl) {
      targetAnchor = getResolvedAnchorPoint(
        targetEl,
        interaction.targetAnchor || "auto",
        sourceAnchor
      );
    }
  }

  const pathD = getConnectionPath(sourceAnchor, targetAnchor, style.connectionRouting);
  const color = getToolColor(style, "connect");

```

```

    return { pathD, color, targetAnchor };
  }, [interaction, activeBoard, style]);

return (
  <div
    ref={containerRef}
    className="w-full h-full relative overflow-hidden bg-background select-none touch-none"
    style={{ cursor: cursorStyle, touchAction: "none" }}
  >
    <CanvasContextMenu elementId={null} disabled={!contextMenuArmed}>
      <svg
        ref={svgRef}
        className="w-full h-full absolute inset-0 touch-none"
        onPointerDown={handlePointerDown}
        onPointerMove={handlePointerMove}
        onPointerUp={handlePointerUp}
        onPointerCancel={handlePointerCancel}
        onPointerLeave={handlePointerLeave}
        // Reflects the live tool so the active tool is observable from the DOM.
        // Nothing in the app reads it; it exists because the tool lives in a
        // zustand store with no other outside signal, which made the
        // borrow-the-eraser behaviour impossible to check from a test.
        data-wb-tool={tool}
        onLostPointerCapture={handleLostPointerCapture}
        onDoubleClick={handleDoubleClick}
        style={{ touchAction: "none" }}
      >
        <StrokeFilterDefs />

        {/* Board paper: colour + pattern, per board.
           The pattern is offset by the camera so it scrolls WITH the canvas
           rather than staying pinned to the screen. */}
        <defs>
          <pattern
            ref={boardPaperRef}
            id="board-paper"
            x={viewport.width / 2 - camera.x * camera.zoom}
            y={viewport.height / 2 - camera.y * camera.zoom}
            width={paperSize * camera.zoom}
            height={paperSize * camera.zoom}
            patternUnits="userSpaceOnUse"
          >
            {paperPattern === "dots" && (
              <circle
                cx={({paperSize * camera.zoom} / 2}
                cy={({paperSize * camera.zoom} / 2}
                r={Math.max(1, (paperSize / 20) * camera.zoom)}
                className="fill-zinc-300 dark:fill-zinc-700"
              />
            )}
            {paperPattern === "grid" && (
              <path
                d={`M 0 ${paperSize * camera.zoom} 0 L 0 0 0 0 ${paperSize * camera.zoom}`}
                fill="none"
                strokeWidth={Math.max(0.5, camera.zoom * 0.8)}
                className="stroke-zinc-300 dark:stroke-zinc-700"
              />
            )}
            {paperPattern === "lines" && (
              <path
                d={`M 0 ${paperSize * camera.zoom} L ${paperSize * camera.zoom} ${paperSize * camera.zoom}`}
                fill="none"
                strokeWidth={Math.max(0.5, camera.zoom * 0.8)}
                className="stroke-zinc-300 dark:stroke-zinc-700"
              />
            )}
          </pattern>

          {/* The same paper, but in WORLD units, for a notebook page. It is
             painted inside the zoomed group, so unlike #board-paper above it
             must not pre-multiply by the zoom – the group already does. */}
          {isNb && (
            <pattern
              id="page-paper"
              x={0}
              y={0}
              width={paperSize}
              height={paperSize}

```

```

        patternUnits="userSpaceOnUse"
      >
      <PaperTiles pattern={paperPattern} size={paperSize} />
    </pattern>
  )}
</defs>

{/* Solid colour first, pattern painted on top of it. Two rects rather
  than one keeps the pattern tiles transparent, so the board colour
  shows through instead of being hidden behind opaque tiles. */}
{!isNb && paperColor && <rect width="100%" height="100%" fill={paperColor} />}
{!isNb && paperPattern !== "plain" && (
  <rect width="100%" height="100%" fill="url(#board-paper)" />
)}

{/* The desk a notebook lies on.
  A whiteboard's background IS the drawing surface, so it carries the
  paper colour and pattern. A notebook's is not: the paper is the
  sheet, and everything around it is furniture. Painting it in the
  board colour made the page edge vanish and the space beside the
  page look like more canvas – which is precisely why it was not
  obvious that writing out there was a mistake. */}
{isNb && (
  <rect
    width="100%"
    height="100%"
    fill={resolvedTheme === "dark" ? "#0b0b0e" : "#e4e4e9"}
  />
)}

{/* World Space Transform Group */}
<g
  ref={worldRef}
  transform={`translate(${viewport.width / 2}, ${viewport.height / 2}) scale(${camera.zoom}) translate
    (${-camera.x}, ${-camera.y})`}
>
  {/* Neighbouring pages, drawn so the notebook scrolls continuously.
    They are display-only: editing happens on the anchored page,
    which the scroll handler keeps in step with what you are
    looking at. */}
  {isNb &&
    neighbourPages.map((n) => {
      // Each sheet wears its OWN paper (the page's, falling back to the
      // notebook's). Painting every sheet with the ACTIVE page's paper is
      // the bug where a grid page showed grid from the page above and
      // turned blank the moment you navigated to it. Each sheet gets its
      // own pattern def because rulings and spacings can differ per page.
      const np = allPages[n.index];
      const npPaper = paperForPage(activeBoard, np ?? null);
      const npColor = paperColorFor(
        np?.background ?? activeBoard?.background,
        resolvedTheme
      );
      const npPatternId = `page-paper-${np?.id ?? n.index}`;
      const npSheetFill =
        npColor || (resolvedTheme === "dark" ? "#18181b" : "#ffffff");
      return (
        <g key={n.id} transform={`translate(0, ${n.offsetY})`} pointerEvents="none">
          <defs>
            <pattern
              id={npPatternId}
              x={0}
              y={0}
              width={npPaper.size}
              height={npPaper.size}
              patternUnits="userSpaceOnUse"
            >
              <PaperTiles pattern={npPaper.pattern} size={npPaper.size} />
            </pattern>
          </defs>
          {/* Flat offset shadow. This was an SVG feDropShadow, which
            forces the whole sheet through an off-screen blur on EVERY
            frame of a scroll – with several sheets on screen that was
            the single biggest cost of panning a PDF notebook. Two
            opaque rectangles look near-identical and cost nothing. */}
          <rect
            x={3}

```

```

      y={4}
      width={pageDim.width}
      height={pageDim.height}
      fill={resolvedTheme === "dark" ? "#000000" : "#a1a1aa"}
      opacity={resolvedTheme === "dark" ? 0.55 : 0.28}
      rx={2}
    />
    <rect
      x={0}
      y={0}
      width={pageDim.width}
      height={pageDim.height}
      fill={npSheetFill}
      stroke={resolvedTheme === "dark" ? "#3f3f46" : "#d4d4d8"}
      strokeWidth={1 / camera.zoom}
      rx={2}
    />
    {np?.pdf && (
      <PdfPageLayer
        pdf={np.pdf}
        width={pageDim.width}
        height={pageDim.height}
      />
    )}
    {/* The sheet's own ruling, over the scan when there is one. */}
    {npPaper.pattern !== "plain" && (
      <rect
        x={0}
        y={0}
        width={pageDim.width}
        height={pageDim.height}
        fill={`url(#${npPatternId})`}
      />
    )}
    {(np?.elements ?? []).map((el) => (
      <ElementRenderer key={el.id} element={el} isSelected={false} />
    ))}
  </g>
);
}}}

{/* The notebook sheet. Drawn in world space so it pans and zooms
with the content, and painted before the elements so ink sits on
top of the paper. */}
{isNb && (
  <g>
    <rect
      x={3}
      y={4}
      width={pageDim.width}
      height={pageDim.height}
      fill={resolvedTheme === "dark" ? "#000000" : "#a1a1aa"}
      opacity={resolvedTheme === "dark" ? 0.55 : 0.28}
      rx={2}
    />
    <rect
      x={0}
      y={0}
      width={pageDim.width}
      height={pageDim.height}
      fill={paperColor || (resolvedTheme === "dark" ? "#18181b" : "#ffffff")}
      stroke={resolvedTheme === "dark" ? "#52525b" : "#a1a1aa"}
      strokeWidth={1.5 / camera.zoom}
      rx={2}
    />
    {/* The PDF page itself, under your ink and never a pointer
target – it is the sheet, not an object you can select or
drag. A ruling the page asks for is painted OVER the scan
below, so Paper works on PDF pages too (by default PDF
notebooks are plain, so scans stay clean). */}
    {activePage?.pdf && (
      <PdfPageLayer
        pdf={activePage.pdf}
        width={pageDim.width}
        height={pageDim.height}
      />
    )}
  )}
  {/* A pattern fill is re-tiled on every frame of a scroll. */}

```

```

    {paperPattern !== "plain" && (
      <rect
        x={0}
        y={0}
        width={pageDim.width}
        height={pageDim.height}
        fill="url(#page-paper)"
      />
    )}
  </g>
)}

{/* Committed Elements */}
{elements.map((el) => (
  <ElementRenderer
    key={el.id}
    element={el}
    isSelected={selectedSet.has(el.id)}
  />
))}

{/* Selected text and find matches, over the ink so they show. */}
{textSelect.worldLayer}

{/* ===== */}
{/* Live Inking Draft Overlay (Direct DOM - Zero React Render during inking) */}
{/* ===== */}
<g
  ref={liveDraftGroupRef}
  pointerEvents="none"
  style={{ display: "none" }}
>
  <path
    ref={liveDraftPathRef}
    stroke="none"
  />
</g>

{/* "+" in the gap between sheets. Drawn in world space but scaled
  by 1/zoom so it is the same size on screen at every zoom. It
  stops the press from reaching the canvas, so tapping it never
  pans or draws. */}
{isNb && gapPlus !== null && (
  <g
    transform={`translate(${pageDim.width / 2}, ${
      (gapPlus - activeIndex) * pageStride - PAGE_GAP / 2
    }) scale(${1 / camera.zoom})`}
    style={{ cursor: "pointer" }}
    data-gap-plus={gapPlus}
    role="button"
    aria-label={
      gapPlus === 0
        ? "Add a page above the first page"
        : gapPlus === allPages.length
        ? "Add a page after the last page"
        : `Add a page between pages ${gapPlus} and ${gapPlus + 1}`
    }
    onPointerDown={(e) => e.stopPropagation()}
    onPointerUp={(e) => {
      e.stopPropagation();
      const k = gapPlus;
      showGapPlus(null);
      insertPageAt(k);
    }}
  >
    <line x1={-150} y1={0} x2={-30} y2={0} stroke="#6366f1" strokeWidth={2} strokeDasharray="5 5" />
    <line x1={30} y1={0} x2={150} y2={0} stroke="#6366f1" strokeWidth={2} strokeDasharray="5 5" />
    <circle r={19} fill="#6366f1" stroke="ffffff" strokeWidth={2} />
    <path d="M -8 0 H 8 M 0 -8 V 8" stroke="ffffff" strokeWidth={3} strokeLinecap="round" />
    {/* Touch target only a little beyond the visible button, so it
      cannot reach far into the pages either side. */}
    <circle r={22} fill="transparent" />
  </g>
)}

{/* Draft Element (Shape Creation: rectangle, ellipse, line, arrow) */}
{draftElement && draftElement.type !== "stroke" && (
  <ElementRenderer element={draftElement} isSelected={false} isDraft />

```

```

    })

    { /* Draft Connection Arrow */
    {connectionDraft && (
      <g style={{ pointerEvents: "none" }}>
        <path
          d={connectionDraft.pathD}
          fill="none"
          stroke={connectionDraft.color}
          strokeWidth={2.5}
          strokeDasharray="6,4"
        />
        <circle
          cx={connectionDraft.targetAnchor.x}
          cy={connectionDraft.targetAnchor.y}
          r={6}
          fill={connectionDraft.color}
        />
      </g>
    )}

    { /* Anchor Ports Highlights */
    {(tool === "connect" || interaction.kind === "connecting") &&
      elements.map((el) => {
        if (el.type === "connection") return null;
        const ports = getNodeAnchorPorts(el);
        const isSource = interaction.kind === "connecting" && interaction.sourceElementId === el.id;
        const isTarget = interaction.kind === "connecting" && interaction.targetElementId === el.id;

        return (
          <g key={`anchors-${el.id}`} style={{ pointerEvents: "none" }}>
            {Object.entries(ports).map(([anchorKey, pt]) => (
              <circle
                key={anchorKey}
                cx={pt.x}
                cy={pt.y}
                r={isSource || isTarget ? 6 : 4.5}
                fill="white"
                stroke="#6366f1"
                strokeWidth={2}
                className="shadow-md transition-transform"
                style={{
                  filter: "drop-shadow(0 1px 3px rgba(0,0,0,0.2))",
                }}
              />
            ))}
          </g>
        );
      })}
    );
  }}}

  { /* Lasso preview: the trail the pen has actually drawn.
    This was a <polygon>, which SVG closes for you – so mid-stroke it
    painted a filled wedge from wherever you started to wherever the
    nib is now, and the enclosed area appeared to fan out from the
    starting point rather than follow the pen. A <polyline> with no
    fill is literally the line you drew, which is all Samsung Notes
    shows until you lift. */
    {interaction.kind === "lasso-selecting" && interaction.points.length >= 2 && (
      <polyline
        points={interaction.points.map((p) => `${p.x},${p.y}`).join(" ")}
        fill="none"
        stroke="#6366f1"
        strokeWidth={1.5 / camera.zoom}
        strokeDasharray={`${5 / camera.zoom},${4 / camera.zoom}`}
        strokeLinecap="round"
        strokeLinejoin="round"
        style={{ pointerEvents: "none" }}
      />
    )}
  }

  { /* Locked-items mode: every locked thing wears a dashed amber frame
    and a padlock, so what can be picked (and unlocked) is visible. */
    {lockedMode &&
      elements.map((el) => {
        if (!el.locked || el.type === "connection") return null;
        const b = getElementBounds(el);
        const z = camera.zoom;
        const picked = selectedIds.includes(el.id);

```



```

const r = 11 / z;
const bx = b.x + b.width;
const by = b.y;
return (
  <g key={`lk-${el.id}`} style={{ pointerEvents: "none" }}>
    <rect
      x={b.x}
      y={b.y}
      width={b.width}
      height={b.height}
      fill={picked ? "rgba(245, 158, 11, 0.10)" : "rgba(245, 158, 11, 0.04)"}
      stroke="#f59e0b"
      strokeWidth={(picked ? 2.5 : 1.5) / z}
      strokeDasharray={picked ? undefined : `${6 / z},${4 / z}`}
      rx={4 / z}
    />
    <circle cx={bx} cy={by} r={r} fill="#f59e0b" stroke="#fff" strokeWidth={1.5 / z} />
    <rect x={bx - 4.5 / z} y={by - 1 / z} width={9 / z} height={6.5 / z} rx={1.2 / z} fill="#fff"
  />
    <path
      d={`M ${bx - 2.8 / z} ${by - 1 / z} v ${-2 / z} a ${2.8 / z} ${2.8 / z} 0 0 1 ${5.6 / z} 0
      v ${2 / z}`}
      fill="none"
      stroke="#fff"
      strokeWidth={1.4 / z}
    />
  </g>
);
}}

{/* Selection Handles & Bounding Box */}
<SelectionOverlay
  selectedIds={selectedIds}
  elements={elements}
  camera={camera}
  viewport={viewport}
/>
</g>

{/* Rubber-band Selection Box (Screen Space) */}
{rubberBand && (
  <rect
    x={rubberBand.x}
    y={rubberBand.y}
    width={rubberBand.w}
    height={rubberBand.h}
    fill="rgba(99, 102, 241, 0.12)"
    stroke="#6366f1"
    strokeWidth={1}
    strokeDasharray="4,3"
    style={{ pointerEvents: "none" }}
  />
)}

{/* Screenshot mode: the viewfinder shows the area while it is drawn. */}
{interaction.kind === "capturing" && (
  <CaptureMarquee
    shape={interaction.shape}
    points={interaction.points}
    camera={camera}
    viewport={viewport}
  />
)}

{/* Eraser cursor. Positioned imperatively by placeEraserCursor - no
transform prop here, or every render would snap it back. Two
strokes so it reads on white paper AND on top of dark ink. */}
<g ref={eraserCursorRef} visibility="hidden" pointerEvents="none">
  <circle r={eraserScreenR} fill="rgba(113, 113, 122, 0.10)" stroke="ffffff" strokeWidth={3} />
  <circle r={eraserScreenR} fill="none" stroke="#52525b" strokeWidth={1.25} />
</g>

{/* S Pen & Stylus Hover Preview Cursor */}
<g ref={penHoverCursorRef} visibility="hidden" pointerEvents="none">
  <circle
    r={Math.max(2, hoverInkRadius)}
    fill={getToolColor(style, tool)}
    opacity={0.35}
  />

```

```

        stroke="#ffffff"
        strokeWidth={1.5}
      />
      <circle
        r={Math.max(1.5, hoverInkRadius)}
        fill="none"
        stroke={getToolColor(style, tool)}
        strokeWidth={1}
      />
    </g>
  </svg>
</CanvasContextMenu>
<LaserLayer />
{shapeNote && <ShapeNoteChip note={shapeNote} onKeepDrawn={keepShapeAsDrawn} onDone={clearShapeNote} />}
{textSelect.overlay}
</div>
);
}

```

```

// === END FILE: src/components/whiteboard/WhiteboardCanvas.tsx ===

```

```

// === START FILE: src/components/whiteboard/trashToast.tsx ===
// File 90 of 194 | Lines: 73 | Size: 2329 bytes

```

```

"use client";

/**
 * Sidebar deletes with Undo.
 * =====
 * Every destructive sidebar action (row delete, batch delete, folder delete)
 * goes through here instead of `window.confirm`: the store snapshots the
 * removed boards/folders into an undo buffer, the toast offers Undo, and the
 * buffer (plus its pictures/PDFs, which the pruner treats as referenced)
 * expires after UNDO_MS. Replaces, never duplicates: a new trash discards
 * the previous buffer first so only one Undo is ever pending.
 */

import * as React from "react";
import { toast } from "@hooks/use-toast";
import { ToastAction } from "@components/ui/toast";
import { useWhiteboard } from "@store/whiteboard";

export const TRASH_UNDO_MS = 8000;

let expiryTimer: ReturnType<typeof setTimeout> | null = null;

function clearExpiry() {
  if (expiryTimer !== null) {
    clearTimeout(expiryTimer);
    expiryTimer = null;
  }
}

function describeBuffer(): string | null {
  const buf = useWhiteboard.getState().lastDeleted;
  if (!buf) return null;
  const parts: string[] = [];
  if (buf.removedBoards.length > 0) {
    parts.push(`${buf.removedBoards.length} board${buf.removedBoards.length === 1 ? "" : "s"}`);
  }
  if (buf.removedFolders.length > 0) {
    parts.push(`${buf.removedFolders.length} folder${buf.removedFolders.length === 1 ? "" : "s"}`);
  }
  return parts.length > 0 ? parts.join(" and ") : "items";
}

/** Trash boards/folders with an Undo toast. No-ops (with no toast) when nothing matches. */
export function trashWithUndo(boardIds: string[], folderIds: string[]) {
  const st = useWhiteboard.getState();
  if (st.lastDeleted) st.discardTrash();
  clearExpiry();
  st.trashItems(boardIds, folderIds);
  const what = describeBuffer();

```

```

if (!what) return;
const t = toast({
  title: `Deleted ${what}`,
  description: "Undo within 8 seconds – pictures and PDFs survive until then.",
  duration: TRASH_UNDO_MS,
  action: (
    <ToastAction
      altText="Undo delete"
      onClick={() => {
        clearExpiry();
        useWhiteboard.getState().undoTrash();
        t.dismiss();
      }}
    >
    Undo
  </ToastAction>
),
});
expiryTimer = setTimeout(() => {
  expiryTimer = null;
  useWhiteboard.getState().discardTrash();
  t.dismiss();
}, TRASH_UNDO_MS);
}

```

```

=====
// === END FILE: src/components/whiteboard/trashToast.tsx ===
=====

```

```

=====
// === START FILE: src/hooks/use-long-press.ts ===
// File 91 of 194 | Lines: 51 | Size: 1809 bytes
=====

```

```

"use client";

import { useRef } from "react";

/**
 * Long-press = the touch equivalent of double-click.
 *
 * Some tool options are reachable on desktop by DOUBLE-CLICKING the tool
 * button, which is a mouse gesture: on the Android build a double tap is not
 * reliably delivered as `dblclick` to a button, so on the tablet there would
 * be no way to reach them at all. Long-press is what mobile users already
 * expect for "more options", and it costs the mouse path nothing (mouse
 * presses never start the timer).
 */
export function useLongPress(onLongPress: () => void, ms = 450) {
  const timer = useRef<ReturnType<typeof setTimeout> | null>(null);
  const firedRef = useRef(false);
  const startRef = useRef<{ x: number; y: number } | null>(null);

  const clear = () => {
    if (timer.current) clearTimeout(timer.current);
    timer.current = null;
  };

  return {
    /** True when the press already opened the panel, so the click is a no-op. */
    firedRef,
    handlers: {
      onPointerDown: (e: React.PointerEvent) => {
        // Mouse users have double-click; a long mouse press should not steal
        // an ordinary click-and-hold.
        if (e.pointerType === "mouse") return;
        firedRef.current = false;
        clear();
        startRef.current = { x: e.clientX, y: e.clientY };
        timer.current = setTimeout(() => {
          firedRef.current = true;
          onLongPress();
        }, ms);
      },
      onPointerMove: (e: React.PointerEvent) => {
        const s = startRef.current;

```

```
    if (timer.current && s && Math.hypot(e.clientX - s.x, e.clientY - s.y) > 10) clear();
  },
  onPointerUp: clear,
  onPointerLeave: clear,
  onPointerCancel: clear,
},
};
}
```

```
=====
// === END FILE: src/hooks/use-long-press.ts ===
=====
```

```
=====
// === START FILE: src/hooks/use-toast.ts ===
// File 92 of 194 | Lines: 194 | Size: 3917 bytes
=====
```

```
"use client"

// Inspired by react-hot-toast library
import * as React from "react"

import type {
  ToastActionElement,
  ToastProps,
} from "@components/ui/toast"

const TOAST_LIMIT = 1
const TOAST_REMOVE_DELAY = 1000000

type ToasterToast = ToastProps & {
  id: string
  title?: React.ReactNode
  description?: React.ReactNode
  action?: ToastActionElement
}

const actionTypes = {
  ADD_TOAST: "ADD_TOAST",
  UPDATE_TOAST: "UPDATE_TOAST",
  DISMISS_TOAST: "DISMISS_TOAST",
  REMOVE_TOAST: "REMOVE_TOAST",
} as const

let count = 0

function genId() {
  count = (count + 1) % Number.MAX_SAFE_INTEGER
  return count.toString()
}

type ActionType = typeof actionTypes

type Action =
  | {
    type: ActionType["ADD_TOAST"]
    toast: ToasterToast
  }
  | {
    type: ActionType["UPDATE_TOAST"]
    toast: Partial<ToasterToast>
  }
  | {
    type: ActionType["DISMISS_TOAST"]
    toastId?: ToasterToast["id"]
  }
  | {
    type: ActionType["REMOVE_TOAST"]
    toastId?: ToasterToast["id"]
  }

interface State {
  toasts: ToasterToast[]
}

const toastTimeouts = new Map<string, ReturnType<typeof setTimeout>>()
```

```
const addToRemoveQueue = (toastId: string) => {
  if (toastTimeouts.has(toastId)) {
    return
  }

  const timeout = setTimeout(() => {
    toastTimeouts.delete(toastId)
    dispatch({
      type: "REMOVE_TOAST",
      toastId: toastId,
    })
  }, TOAST_REMOVE_DELAY)

  toastTimeouts.set(toastId, timeout)
}

export const reducer = (state: State, action: Action): State => {
  switch (action.type) {
    case "ADD_TOAST":
      return {
        ...state,
        toasts: [action.toast, ...state.toasts].slice(0, TOAST_LIMIT),
      }

    case "UPDATE_TOAST":
      return {
        ...state,
        toasts: state.toasts.map((t) =>
          t.id === action.toast.id ? { ...t, ...action.toast } : t
        ),
      }

    case "DISMISS_TOAST": {
      const { toastId } = action

      // ! Side effects ! - This could be extracted into a dismissToast() action,
      // but I'll keep it here for simplicity
      if (toastId) {
        addToRemoveQueue(toastId)
      } else {
        state.toasts.forEach((toast) => {
          addToRemoveQueue(toast.id)
        })
      }

      return {
        ...state,
        toasts: state.toasts.map((t) =>
          t.id === toastId || toastId === undefined
            ? {
                ...t,
                open: false,
              }
            : t
          ),
      }
    }
    case "REMOVE_TOAST":
      if (action.toastId === undefined) {
        return {
          ...state,
          toasts: [],
        }
      }
      return {
        ...state,
        toasts: state.toasts.filter((t) => t.id !== action.toastId),
      }
  }
}

const listeners: Array<(state: State) => void> = []

let memoryState: State = { toasts: [] }

function dispatch(action: Action) {
  memoryState = reducer(memoryState, action)
}
```

```
listeners.forEach((listener) => {
  listener(memoryState)
})
}

type Toast = Omit<ToasterToast, "id">

function toast({ ...props }: Toast) {
  const id = genId()

  const update = (props: ToasterToast) =>
    dispatch({
      type: "UPDATE_TOAST",
      toast: { ...props, id },
    })
  const dismiss = () => dispatch({ type: "DISMISS_TOAST", toastId: id })

  dispatch({
    type: "ADD_TOAST",
    toast: {
      ...props,
      id,
      open: true,
      onOpenChange: (open) => {
        if (!open) dismiss()
      },
    },
  })

  return {
    id: id,
    dismiss,
    update,
  }
}

function useToast() {
  const [state, setState] = React.useState<State>(memoryState)

  React.useEffect(() => {
    listeners.push(setState)
    return () => {
      const index = listeners.indexOf(setState)
      if (index > -1) {
        listeners.splice(index, 1)
      }
    }
  }, [state])

  return {
    ...state,
    toast,
    dismiss: (toastId?: string) => dispatch({ type: "DISMISS_TOAST", toastId }),
  }
}

export { useToast, toast }

=====
// === END FILE: src/hooks/use-toast.ts ===
=====

=====
// === START FILE: src/lib/Utils.ts ===
// File 93 of 194 | Lines: 6 | Size: 166 bytes
=====

import { clsx, type ClassValue } from "clsx"
import { twMerge } from "tailwind-merge"

export function cn(...inputs: ClassValue[]) {
  return twMerge(clsx(inputs))
}

=====
// === END FILE: src/lib/Utils.ts ===
=====
```

```
=====
// === START FILE: src/lib/whiteboard/account.ts ===
// File 94 of 194 | Lines: 183 | Size: 6439 bytes
=====

/**
 * Accounts & cloud sync (Supabase)
 * =====
 * Self-service sign-up: anyone can create their own account with an email and
 * password – nobody has to be added by hand. Each account gets exactly one
 * workspace row, and Postgres row-level security makes it impossible for one
 * account to read another's, even with the public key in hand.
 *
 * Email + password is deliberate rather than OAuth: this app also ships as an
 * Android APK, and inside a WebView an OAuth redirect round-trip is fragile,
 * while an email/password call is a plain HTTPS request that just works.
 *
 * Not configured? Everything here reports "unavailable" and the app keeps
 * working offline with local storage plus backup files.
 */

import { createClient, type SupabaseClient, type User } from "@supabase/supabase-js";
import { parseWorkspace, serializeWorkspace, type WorkspaceSnapshot } from "../workspace";

const TABLE = "workspaces";

let client: SupabaseClient | null = null;

export function getConfig(): { url: string; anonKey: string } | null {
  const url = process.env.NEXT_PUBLIC_SUPABASE_URL?.trim();
  const anonKey = process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY?.trim();
  if (!url || !anonKey) return null;
  return { url, anonKey };
}

export function isConfigured(): boolean {
  return !!getConfig();
}

function getClient(): SupabaseClient {
  const cfg = getConfig();
  if (!cfg) throw new Error("Accounts are not set up in this build.");
  if (!client) {
    client = createClient(cfg.url, cfg.anonKey, {
      auth: {
        persistSession: true,
        autoRefreshToken: true,
        // No OAuth redirects are used, so there is nothing to detect in the URL;
        // leaving this on makes the APK's file:// origin log spurious warnings.
        detectSessionInUrl: false,
      },
    });
  }
  return client;
}

export interface AccountUser {
  id: string;
  email: string | null;
  /** False until the confirmation link is clicked, when confirmation is on. */
  confirmed: boolean;
}

function toAccountUser(user: User | null): AccountUser | null {
  if (!user) return null;
  return {
    id: user.id,
    email: user.email ?? null,
    confirmed: !!user.email_confirmed_at || !!user.confirmed_at,
  };
}

export async function getCurrentUser(): Promise<AccountUser | null> {
  if (!isConfigured()) return null;
  const { data } = await getClient().auth.getUser();
  return toAccountUser(data.user ?? null);
}
```

```

}

export function onAuthChange(cb: (user: AccountUser | null) => void): () => void {
  if (!isConfigured()) return () => {};
  const { data } = getClient().auth.onAuthStateChange((_event, session) => {
    cb(toAccountUser(session?.user ?? null));
  });
  return () => data.subscription.unsubscribe();
}

export interface SignUpResult {
  user: AccountUser | null;
  /** True when Supabase requires the emailed confirmation link first. */
  needsEmailConfirmation: boolean;
}

export async function signUp(email: string, password: string): Promise<SignUpResult> {
  const { data, error } = await getClient().auth.signUp({ email, password });
  if (error) throw new Error(friendlyAuthError(error.message));
  // Supabase returns a user with no session when confirmation is required.
  return {
    user: toAccountUser(data.user),
    needsEmailConfirmation: !data.session,
  };
}

export async function signIn(email: string, password: string): Promise<AccountUser | null> {
  const { data, error } = await getClient().auth.signInWithPassword({ email, password });
  if (error) throw new Error(friendlyAuthError(error.message));
  return toAccountUser(data.user);
}

export async function sendPasswordReset(email: string): Promise<void> {
  const { error } = await getClient().auth.resetPasswordForEmail(email);
  if (error) throw new Error(friendlyAuthError(error.message));
}

export async function signOut(): Promise<void> {
  await getClient().auth.signOut();
}

/** Supabase's raw messages are terse; make the common ones actionable. */
function friendlyAuthError(message: string): string {
  const m = message.toLowerCase();
  if (m.includes("invalid login credentials")) return "Wrong email or password.";
  if (m.includes("already registered") || m.includes("already been registered")) {
    return "That email already has an account – sign in instead.";
  }
  if (m.includes("password") && m.includes("6")) {
    return "Password must be at least 6 characters.";
  }
  if (m.includes("email not confirmed")) {
    return "Check your inbox and click the confirmation link first.";
  }
  if (m.includes("rate limit") || m.includes("too many")) {
    return "Too many attempts – wait a minute and try again.";
  }
  return message;
}

// -----
// Workspace row
// -----

export interface RemoteRow {
  snapshot: WorkspaceSnapshot;
  updatedAt: number;
}

/** The signed-in account's workspace, or null if they have never synced. */
export async function pullWorkspace(): Promise<RemoteRow | null> {
  const supa = getClient();
  const { data: userData } = await supa.auth.getUser();
  const uid = userData.user?.id;
  if (!uid) throw new Error("Not signed in.");

  const { data, error } = await supa
    .from(TABLE)

```



```

    .select("doc, updated_at")
    .eq("user_id", uid)
    .maybeSingle();

if (error) throw new Error(`Could not load your workspace: ${error.message}`);
if (!data?.doc) return null;

// The row is stored as the same portable document as a backup file, so it
// goes through the identical validation – a malformed row cannot brick the app.
const snapshot = parseWorkspace(
  typeof data.doc === "string" ? data.doc : JSON.stringify(data.doc)
);
return { snapshot, updatedAt: Date.parse(data.updated_at) || 0 };
}

export async function pushWorkspace(
  snapshot: WorkspaceSnapshot,
  deviceLabel?: string
): Promise<void> {
  const supa = getClient();
  const { data: userData } = await supa.auth.getUser();
  const uid = userData.user?.id;
  if (!uid) throw new Error("Not signed in.");

  const { error } = await supa.from(TABLE).upsert(
    {
      user_id: uid,
      doc: JSON.parse(serializeWorkspace(snapshot, deviceLabel)),
      updated_at: new Date().toISOString(),
    },
    { onConflict: "user_id" }
  );
  if (error) throw new Error(`Could not save your workspace: ${error.message}`);
}

=====
// === END FILE: src/lib/whiteboard/account.ts ===
=====

=====
// === START FILE: src/lib/whiteboard/appClipboard.ts ===
// File 95 of 194 | Lines: 88 | Size: 3047 bytes
=====

/**
 * ProNotes' own clipboard for pictures
 * =====
 * "Copy" on a capture has to work, and inside the Android app the system
 * clipboard often will not take a picture: the WebView refuses
 * `navigator.clipboard.write` for images (NotAllowedError), and the Capacitor
 * clipboard plugin pastes an image as a long line of text. There is no honest
 * way to put a PNG on the Android clipboard from here.
 *
 * So every copy lands in two places: the system clipboard when it allows it
 * (desktop browsers, Electron), and always this in-memory slot. Pasting inside
 * ProNotes - Ctrl+V on the canvas, or "Paste" in the Add-a-picture sheet -
 * falls back to this slot when the system clipboard has no picture, which
 * means a copy made here can always be pasted here.
 *
 * Memory only, on purpose: a clipboard that survived a restart would paste
 * something you copied last week.
 */

export interface AppClipImage {
  dataUrl: string;
  /** Pixel size of the picture. */
  width: number;
  height: number;
  /** The size it had on the page, in world units, so a paste lands 1:1. */
  worldSize?: { width: number; height: number };
  /** True when the system clipboard accepted it as well. */
  systemCopied: boolean;
  at: number;
}

let slot: AppClipImage | null = null;

```

```
// Anything copied since (text from a PDF, a note, a text box) is newer than
// our picture: a later paste must not bring the old capture back.
if (typeof document !== "undefined") {
  document.addEventListener("copy", () => {
    slot = null;
  }, true);
  document.addEventListener("cut", () => {
    slot = null;
  }, true);
}

export function putAppClipImage(clip: Omit<AppClipImage, "at">): void {
  slot = { ...clip, at: Date.now() };
}

export function getAppClipImage(): AppClipImage | null {
  return slot;
}

export function clearAppClip(): void {
  slot = null;
}

/**
 * Puts a PNG on the system clipboard. True only when the platform says it
 * did - the caller tells the user what really happened.
 */
export async function writeImageToSystemClipboard(blob: Blob): Promise<boolean> {
  try {
    if (
      typeof navigator === "undefined" ||
      !navigator.clipboard ||
      typeof navigator.clipboard.write !== "function" ||
      typeof ClipboardItem === "undefined"
    ) {
      return false;
    }
    const png = blob.type === "image/png" ? blob : new Blob([blob], { type: "image/png" });
    await navigator.clipboard.write([new ClipboardItem({ "image/png": png })]);
    return true;
  } catch (err) {
    console.warn("[clipboard] The system clipboard refused the picture.", err);
    return false;
  }
}

/**
 * Whether a paste that found no picture on the system clipboard should use
 * ours instead: only when our last copy did NOT reach the system clipboard.
 * If it did, the system clipboard is the truth - something copied elsewhere
 * since then must not be overridden by an older capture.
 */
export function appClipForPaste(): AppClipImage | null {
  return slot && !slot.systemCopied ? slot : null;
}

=====
// === END FILE: src/lib/whiteboard/appClipboard.ts ===
=====

=====
// === START FILE: src/lib/whiteboard/capture.ts ===
// File 96 of 194 | Lines: 626 | Size: 25165 bytes
=====

/**
 * Capture: a picture of what you see in one area
 * =====
 * The select tool's "screenshot". You draw a box or a loop over anything on
 * the canvas - ink, text, pictures, the paper and its ruling, the PDF / Word /
 * PowerPoint page underneath - and get back a PNG of exactly that, to copy,
 * share or drop into a notebook.
 *
 * It is NOT a screen grab. A WebView cannot read its own pixels, and a screen
 * grab would be stuck at screen resolution with the selection handles in it.
 * The area is re-drawn from the document instead, through the export engine
 * (export.ts) - the same serializer, the same sheet painter, the same PDF

```

```

* rasteriser - so a capture can never disagree with an export, and it can be
* rendered sharper than the screen showed it.
*
* Coordinates are WORLD units in the frame the canvas was in when the area
* was drawn: for a notebook that is the frame anchored on the active page
* (`anchorIndex` at y = 0, its neighbours one stride above and below), which
* is why the source carries the anchor with it. A capture that spans the gap
* between two sheets shows both halves, the way the screen did.
*
* Everything here except the last few async functions is pure, so the region
* maths and the SVG it produces are unit-tested in node.
*/

import type { Board, ConnectionElement, NotebookPage, WhiteboardElement } from "./types";
import { BOARD_BACKGROUNDDS } from "./types";
import { getBoardElements, getPages, isNotebook, pageSizeOf, PAGE_GAP } from "./notebook";
import { getElementBounds, getUnionBounds, rotatePoint } from "./geometry";
import {
  SVG_HEAD_STYLE,
  fitScale,
  paperFor,
  paperPatternSvg,
  rasterizeSvg,
  resolveExportImages,
  resolvePageBackdrops,
  serializeElements,
  sheetSvg,
  type ExportReport,
  type SheetLayers,
} from "./export";

export interface CapturePoint {
  x: number;
  y: number;
}

export interface CaptureRect {
  x: number;
  y: number;
  width: number;
  height: number;
}

export type CaptureShape = "rectangle" | "lasso";

/**
 * What goes behind the content.
 * - page:      what you see - paper colour, ruling, the PDF page.
 * - white:     a clean white sheet: no ruling, no tinted paper, but the PDF
 *              page stays, because on a lecture slide the slide IS the
 *              content.
 * - transparent: your ink and objects alone.
 */
export type CaptureBackground = "page" | "white" | "transparent";

/** One capture: a document, an area of it, and how the area was drawn. */
export interface CaptureSource {
  board: Board;
  /** World units, in the frame anchored on `anchorIndex`. */
  region: CaptureRect;
  /** A freeform loop: the picture is masked to it, transparent outside. */
  polygon?: CapturePoint[];
  /** The notebook page that sat at y = 0 when the area was drawn. */
  anchorIndex: number;
  /** "Selection only": draw just these elements (plus the paper, if asked). */
  onlyIds?: string[];
}

export interface CaptureLook {
  background: CaptureBackground;
  /** The app was in dark mode, so "page" means the dark paper you saw. */
  dark: boolean;
}

/** Smaller than this (world units) is a tap or a slip, not an area. */
export const CAPTURE_MIN_SIDE = 6;

/** The sharpness steps the panel offers: pixels per world unit. */

```

```

export const CAPTURE_SCALES = [1, 2, 3] as const;
export type CaptureScale = (typeof CAPTURE_SCALES)[number];

/** The canvas background in dark mode (globals.css `--background`). */
const THEME_DARK_BG = "#0a0a0a";

// -----
// Region maths
// -----

export function rectFromCorners(a: CapturePoint, b: CapturePoint): CaptureRect {
  return {
    x: Math.min(a.x, b.x),
    y: Math.min(a.y, b.y),
    width: Math.abs(b.x - a.x),
    height: Math.abs(b.y - a.y),
  };
}

export function boundsOfPoints(points: CapturePoint[]): CaptureRect | null {
  if (points.length === 0) return null;
  let x0 = Infinity;
  let y0 = Infinity;
  let x1 = -Infinity;
  let y1 = -Infinity;
  for (const p of points) {
    if (!Number.isFinite(p.x) || !Number.isFinite(p.y)) continue;
    x0 = Math.min(x0, p.x);
    y0 = Math.min(y0, p.y);
    x1 = Math.max(x1, p.x);
    y1 = Math.max(y1, p.y);
  }
  if (!Number.isFinite(x0)) return null;
  return { x: x0, y: y0, width: x1 - x0, height: y1 - y0 };
}

export function intersectRect(a: CaptureRect, b: CaptureRect): CaptureRect | null {
  const x0 = Math.max(a.x, b.x);
  const y0 = Math.max(a.y, b.y);
  const x1 = Math.min(a.x + a.width, b.x + b.width);
  const y1 = Math.min(a.y + a.height, b.y + b.height);
  if (x1 <= x0 || y1 <= y0) return null;
  return { x: x0, y: y0, width: x1 - x0, height: y1 - y0 };
}

/** Shoelace area; used to tell a real loop from a scribbled line. */
export function polygonArea(points: CapturePoint[]): number {
  let sum = 0;
  for (let i = 0; i < points.length; i++) {
    const a = points[i];
    const b = points[(i + 1) % points.length];
    sum += a.x * b.y - b.x * a.y;
  }
  return Math.abs(sum) / 2;
}

/**
 * The paper a notebook's pages cover, in the frame anchored on `anchorIndex`.
 * Null for a whiteboard, whose world has no edge.
 */
export function documentBounds(board: Board, anchorIndex: number): CaptureRect | null {
  if (!isNotebook(board)) return null;
  const count = getPages(board).length;
  if (count === 0) return null;
  const { width, height } = pageSizeOf(board);
  const stride = height + PAGE_GAP;
  return {
    x: 0,
    y: -anchorIndex * stride,
    width,
    height: count * height + (count - 1) * PAGE_GAP,
  };
}

/**
 * The area actually captured from what was drawn, or null when it is too
 * small to be one.
 */

```

```

* In a notebook the grey desk around the pages is furniture, not content, so
* the area is trimmed to the paper: a box dragged generously past the page
* edge gives the page, not a frame of empty desk. A whiteboard is captured as
* drawn.
*/
export function clampCaptureRegion(
  board: Board,
  anchorIndex: number,
  drawn: CaptureRect
): CaptureRect | null {
  const doc = documentBounds(board, anchorIndex);
  const r = doc ? intersectRect(drawn, doc) : drawn;
  if (!r || r.width < CAPTURE_MIN_SIDE || r.height < CAPTURE_MIN_SIDE) return null;
  return r;
}

/** The notebook sheets an area touches, with where each sits in the frame. */
export function sheetsInRegion(
  board: Board,
  anchorIndex: number,
  region: CaptureRect
): Array<{ page: NotebookPage; index: number; offsetY: number }> {
  if (!isNotebook(board)) return [];
  const { height } = pageSizeOf(board);
  const stride = height + PAGE_GAP;
  const out: Array<{ page: NotebookPage; index: number; offsetY: number }> = [];
  getPages(board).forEach((page, index) => {
    const offsetY = (index - anchorIndex) * stride;
    if (offsetY + height <= region.y || offsetY >= region.y + region.height) return;
    out.push({ page, index, offsetY });
  });
  return out;
}

/**
* The area to capture for a selection: its bounds plus a little air, so ink
* right at the edge of the box is not shaved. The air is in SCREEN pixels
* (divided by the zoom), which is what the eye judges.
*/
export function selectionCaptureRegion(
  elements: WhiteboardElement[],
  ids: string[],
  zoom: number,
  airPx = 14
): CaptureRect | null {
  const set = new Set(ids);
  const picked = elements.filter((el) => set.has(el.id) && el.type !== "connection");
  if (picked.length === 0 || !getUnionBounds(picked)) return null;
  // Turned pictures and shapes reach past their unturned box.
  const boxes = picked.map(rotatedBounds);
  const x0 = Math.min(...boxes.map((q) => q.x));
  const y0 = Math.min(...boxes.map((q) => q.y));
  const b = {
    x: x0,
    y: y0,
    width: Math.max(...boxes.map((q) => q.x + q.width)) - x0,
    height: Math.max(...boxes.map((q) => q.y + q.height)) - y0,
  };
  const air = airPx / (zoom > 0 ? zoom : 1);
  return { x: b.x - air, y: b.y - air, width: b.width + air * 2, height: b.height + air * 2 };
}

/**
* The sharpness to start with: as sharp as the screen showed it (zoom times
* the device's pixel ratio), and never below 2, which is what a shared picture
* of handwriting needs to stay readable on a phone.
*/
export function defaultCaptureScale(zoom: number, devicePixelRatio: number): CaptureScale {
  const seen = Math.ceil((zoom > 0 ? zoom : 1) * (devicePixelRatio > 0 ? devicePixelRatio : 1) - 0.05);
  return Math.min(3, Math.max(2, seen)) as CaptureScale;
}

/** Largest capture, in pixels (about 5000 x 4000). */
const CAPTURE_MAX_PIXELS = 20_000_000;

/**
* The picture's real size in pixels. A huge area at 3x is scaled down to what
* a canvas can hold (the same limits every export obeys) rather than failing,

```

```

    * so the panel shows the size the file will actually have.
    */
export function capturePixelSize(region: CaptureRect, scale: number) {
    // Tighter than an export's cap: a capture's PNG then crosses to Android
    // (share, gallery) as one base64 string, several copies of it at once.
    const s = Math.min(
        fitScale(region.width, region.height, scale),
        Math.sqrt(CAPTURE_MAX_PIXELS / Math.max(region.width * region.height, 1))
    );
    return {
        width: Math.max(1, Math.round(region.width * s)),
        height: Math.max(1, Math.round(region.height * s)),
        scale: s,
    };
}

/**
 * Which elements a capture draws. Selection-only keeps the chosen elements,
 * plus any connector whose BOTH ends were chosen: connectors cannot be picked
 * by a box or a loop, so without this a captured diagram lost its arrows.
 */
export function captureKeep(onlyIds?: string[]): ((el: WhiteboardElement) => boolean) | undefined {
    if (!onlyIds) return undefined;
    const set = new Set(onlyIds);
    return (el) => {
        if (set.has(el.id)) return true;
        if (el.type === "connection") {
            const c = el as ConnectionElement;
            return set.has(c.source?.elementId) && set.has(c.target?.elementId);
        }
        return false;
    };
}

/** An element's box as it is drawn: its unturned box, turned by its rotation. */
function rotatedBounds(el: WhiteboardElement): CaptureRect {
    const b = getElementBounds(el);
    const rot = el.rotation || 0;
    if (!rot) return b;
    const cx = b.x + b.width / 2;
    const cy = b.y + b.height / 2;
    const pts = [
        rotatePoint(b.x, b.y, cx, cy, rot),
        rotatePoint(b.x + b.width, b.y, cx, cy, rot),
        rotatePoint(b.x, b.y + b.height, cx, cy, rot),
        rotatePoint(b.x + b.width, b.y + b.height, cx, cy, rot),
    ];
    const xs = pts.map((p) => p.x);
    const ys = pts.map((p) => p.y);
    return {
        x: Math.min(...xs),
        y: Math.min(...ys),
        width: Math.max(...xs) - Math.min(...xs),
        height: Math.max(...ys) - Math.min(...ys),
    };
}

function boundsTouch(el: WhiteboardElement, r: CaptureRect): boolean {
    // Connections store no geometry of their own; they are always kept, and the
    // serializer draws them only when both ends are present anyway.
    if (el.type === "connection") return true;
    const b = rotatedBounds(el);
    return b.x <= r.x + r.width && b.x + b.width >= r.x && b.y <= r.y + r.height && b.y + b.height >= r.y;
}

// -----
// The picture
// -----

const MASK_ID = "pn-capture-mask";
const PAPER_ID = "pn-capture-paper";

function num(n: number): string {
    return Number(n.toFixed(2)).toString();
}

/**
 * The capture as a standalone SVG whose viewBox is the area itself.

```

```

*
* Notebook sheets are painted by the export's own `sheetSvg`, each clipped to
* its paper, so the gap between two sheets stays empty (transparent) exactly
* as it is empty desk on screen. A whiteboard's paper has no edge, so its
* colour and ruling simply fill the area, tiled from the world origin the way
* the canvas tiles it.
*/
export function buildCaptureSvg(
  src: CaptureSource,
  look: CaptureLook,
  customLinkTypes: Array<{ id: string; name: string; color: string }> = [],
  resolvedImages?: Map<string, string>,
  backdrops?: Map<string, string>,
  /** The canvas' own text boxes and their fonts (see collectLiveText). */
  live?: LiveText
): { svg: string; width: number; height: number; report: ExportReport } {
  const { board, region } = src;
  const report: ExportReport = { missingImages: 0, missingPdfPages: 0, pages: 1 };
  const tone = look.dark ? "dark" : "white";
  // Default-coloured text follows the paper it will sit on.
  const inkTone = look.background === "white" ? "white" : tone;
  const keep = captureKeep(src.onlyIds);
  const liveMarkup = live?.markup;
  const override = liveMarkup?.size
    ? (el: WhiteboardElement) => liveMarkup.get(el.id) ?? null
    : undefined;
  const areaRect = (fill: string) =>
    `<rect x="${num(region.x)}" y="${num(region.y)}" width="${num(region.width)}" height="${num(region.height)}" fill="${fill}" />`;

  // The canvas' fonts, embedded: an SVG drawn as an image cannot load the
  // page's web fonts, and text would fall back to a serif.
  let defs = live?.fontCss ? `<style><![CDATA[\n${live.fontCss}\n]]></style>` : "";
  let body = "";

  if (isNotebook(board)) {
    const { width, height } = pageSizeOf(board);
    const sheets = sheetsInRegion(board, src.anchorIndex, region);
    report.pages = sheets.length;
    const layers: SheetLayers =
      look.background === "page"
        ? { keep, override }
        : look.background === "white"
          ? { paper: "ffffff", ruling: false, keep, override }
          : { paper: false, ruling: false, backdrop: false, keep, override };
    // White means white: the desk showing between two sheets too.
    if (look.background === "white") body += ` ${areaRect("ffffff")}\n`;
    for (const s of sheets) {
      body += sheetSvg(
        board, s.page, s.index, s.offsetY, width, height,
        { background: inkTone }, customLinkTypes, resolvedImages, backdrops, report, false, layers
      );
    }
  } else {
    if (look.background === "page") {
      const paper = paperFor(board, null, tone);
      const own = board.background?.color;
      const preset = BOARD_BACKGROUNDS.find((b) => b.id === own);
      const hasColour = preset ? !(look.dark ? preset.dark : preset.light) : !!own;
      // No colour of its own: the canvas shows the theme's background.
      body += ` ${areaRect(hasColour ? paper.fill : look.dark ? THEME_DARK_BG : "ffffff")}\n`;
      const pattern = paperPatternSvg(PAPER_ID, paper);
      if (pattern) {
        defs += pattern;
        body += ` ${areaRect(`url(#${PAPER_ID})`)}\n`;
      }
    } else if (look.background === "white") {
      body += ` ${areaRect("ffffff")}\n`;
    }
    const elements = getBoardElements(board).filter((el) => !keep || keep(el));
    body += serializeElements(elements, inkTone, customLinkTypes, resolvedImages, report, override);
  }

  const poly = src.polygon && src.polygon.length >= 3 ? src.polygon : null;
  if (poly) {
    defs += `<clipPath id="${MASK_ID}"><polygon points="${poly.map((p) => `${num(p.x)} ${num(p.y)}`).join(" ")}" /></clipPath>`;
    body = `<g clip-path="url(#${MASK_ID})">\n${body} </g>\n`;
  }
}

```

```

    }

    const svg = `<<?xml version="1.0" encoding="UTF-8"?>
<svg xmlns="http://www.w3.org/2000/svg" viewBox="${num(region.x)} ${num(region.y)} ${num(region.width)} ${num(
  region.height)}" width="${num(region.width)}" height="${num(region.height)}">
${SVG_HEAD_STYLE}
  ${defs ? `<defs>${defs}</defs>` : ""}
  ${body}</svg>`;
    return { svg, width: region.width, height: region.height, report };
  }

  /** Pictures and PDF pages a capture needs, fetched once per sharpness. */
  export interface CaptureAssets {
    images: Map<string, string>;
    backdrops: Map<string, string>;
  }

  /**
   * Loads only what the area touches: the pictures that overlap it and the PDF
   * pages of the sheets under it, the PDF rendered at the capture's own density
   * so a crop of a slide is as crisp as the ink over it.
   */
  export async function resolveCaptureAssets(src: CaptureSource, scale: number): Promise<CaptureAssets> {
    const { board, region } = src;
    if (isNotebook(board)) {
      const sheets = sheetsInRegion(board, src.anchorIndex, region);
      const trimmed: Board = {
        ...board,
        pages: sheets.map((s) => {
          const local = { ...region, y: region.y - s.offsetY };
          return { ...s.page, elements: (s.page.elements ?? []).filter((el) => boundsTouch(el, local)) };
        }),
      };
      const [images, backdrops] = await Promise.all([
        resolveExportImages(trimmed),
        resolvePageBackdrops(sheets.map((s) => s.page), pageSizeOf(board).width, scale),
      ]);
      return { images, backdrops };
    }
    const trimmed: Board = { ...board, elements: getBoardElements(board).filter((el) => boundsTouch(el, region)) };
    return { images: await resolveExportImages(trimmed), backdrops: new Map() };
  }

  /** The canvas' text boxes as it draws them, plus the fonts they use. */
  export interface LiveText {
    /** Element id -> that element's own markup, ready to drop into the SVG. */
    markup: Map<string, string>;
    /** @font-face rules with the font files inlined. */
    fontCss: string;
  }

  /** Computed properties baked onto class-styled nodes inside a text box. */
  const BAKED_PROPS = ["color", "background-color", "font-weight", "font-style", "text-decoration-line", "text-decoration-color"];

  /**
   * The canvas' own markup for the text boxes and sticky notes in the area.
   *
   * The export serializer lays text out itself, one SVG <text> per line with
   * no wrapping, so a paragraph the canvas wraps inside its box came out as one
   * long line running off the capture - acceptable in a vector export, wrong
   * for "a picture of what I see". The canvas draws text as inline-styled HTML
   * in a <foreignObject>; reusing that exact markup makes the capture wrap,
   * clip and space every line the way the screen does.
   *
   * Two things do not survive the trip into an SVG image and are fixed here:
   * CSS variables (the fonts are `var(--font-inter)`) resolve to nothing, so
   * each is replaced by its computed value; and web fonts cannot be fetched by
   * an image, so the @font-face rules the text uses are inlined as data URLs.
   * Tailwind classes (a link's colour) are baked in as computed styles too.
   *
   * Only elements actually drawn can be read (the canvas culls, and hides the
   * one being edited); anything missing falls back to the serializer.
   */
  export async function collectLiveText(src: CaptureSource): Promise<LiveText> {
    const markup = new Map<string, string>();
    if (typeof document === "undefined" || typeof XMLSerializer === "undefined") return { markup, fontCss: "" };
  }

```



```

const host = document.querySelector("svg[data-wb-tool]");
if (!host) return { markup, fontCss: "" };
const candidates = isNotebook(src.board)
  ? sheetsInRegion(src.board, src.anchorIndex, src.region).flatMap((s) => s.page.elements ?? [])
  : getBoardElements(src.board);
const xml = new XMLSerializer();
const families = new Set<string>();
for (const el of candidates) {
  if (el.type !== "text" && el.type !== "sticky") continue;
  const node = host.querySelector(`g[data-element-id="${CSS.escape(el.id)}"]`);
  if (!node || !node.querySelector("foreignObject")) continue;
  const clone = node.cloneNode(true) as Element;
  const lives = [node, ...Array.from(node.querySelectorAll("*"))];
  const clones = [clone, ...Array.from(clone.querySelectorAll("*"))];
  lives.forEach((live, k) => {
    const copy = clones[k] as HTMLElement | undefined;
    const inline = (live as HTMLElement).style;
    if (!copy?.style || !inline) return;
    const computed = getComputedStyle(live);
    for (let n = 0; n < inline.length; n++) {
      const prop = inline[n];
      if (inline.getPropertyValue(prop).includes("var(")) copy.style.setProperty(prop, computed.getPropertyValue(prop));
    }
    if (live.closest("foreignObject")) {
      if (live.getAttribute("class")) {
        for (const prop of BAKED_PROPS) copy.style.setProperty(prop, computed.getPropertyValue(prop));
        copy.removeAttribute("class");
      }
      for (const f of computed.fontFamily.split(",")) families.add(f.trim().replace(/^[\'"]|[\']$/g, ""));
    }
  });
  clone.removeAttribute("style");
  clone.removeAttribute("data-element-id");
  markup.set(el.id, xml.serializeToString(clone));
}
const fontCss = markup.size ? await embedFontFaces(families) : "";
return { markup, fontCss };
}

```

```

/** Font files already read, by URL: every capture after the first is free. */
const fontData = new Map<string, Promise<string | null>>();

```

```

function fontDataUrl(url: string): Promise<string | null> {
  let p = fontData.get(url);
  if (!p) {
    p = fetch(url)
      .then((r) => (r.ok ? r.blob() : null))
      .then(
        (b) =>
          b &&
          new Promise<string>((resolve, reject) => {
            const reader = new FileReader();
            reader.onload = () => resolve(String(reader.result));
            reader.onerror = () => reject(reader.error);
            reader.readAsDataURL(b);
          })
      )
      .catch(() => null);
    fontData.set(url, p);
  }
  return p;
}

```

```

/** The page's @font-face rules for these families, with their files inlined. */
async function embedFontFaces(families: Set<string>): Promise<string> {
  if (families.size === 0) return "";
  const out: string[] = [];
  for (const sheet of Array.from(document.styleSheets)) {
    let rules: CSSRuleList;
    try {
      rules = sheet.cssRules;
    } catch {
      continue; // a cross-origin sheet; the app ships none, but never throw
    }
    for (const rule of Array.from(rules)) {
      if (!(rule instanceof CSSFontFaceRule)) continue;
      const family = rule.style.getPropertyValue("font-family").trim().replace(/^[\'"]|[\']$/g, "");
    }
  }
}

```

```

    if (!families.has(family)) continue;
    let css = rule.cssText;
    const base = sheet.href ?? document.baseURI;
    for (const m of Array.from(css.matchAll(/url\(((["']?)([^\1\2\/g])) {
        if (m[2].startsWith("data:")) continue;
        const data = await fontDataUrl(new URL(m[2], base).href);
        if (data) css = css.split(m[0]).join(`url("${data}")`);
    }
    out.push(css);
}
}
return out.join("\n");
}

/** How long a WebView gets to draw HTML-in-SVG before the plain path is used. */
// Generous: when it fires, the first render cannot be stopped and a second
// one starts beside it, so it must only catch a real stall, never a slow
// large capture on the tablet.
const LIVE_RASTER_TIMEOUT_MS = 15000;

/** The finished PNG, its pixel size and what it had to leave out. */
export async function renderCapture(
    src: CaptureSource,
    look: CaptureLook,
    scale: number,
    customLinkTypes: Array<{ id: string; name: string; color: string }>,
    assets: CaptureAssets,
    live?: LiveText
): Promise<{ blob: Blob; width: number; height: number; report: ExportReport }> {
    const size = capturePixelSize(src.region, scale);
    // Render at the capped scale, so the file is the size the panel showed.
    scale = size.scale;
    // The SVG paints its own background (or none): the rasteriser must not
    // fill the canvas first, or the outside of a freeform loop would not be
    // transparent.
    const draw = async (withLive?: LiveText) => {
        const built = buildCaptureSvg(src, look, customLinkTypes, assets.images, assets.backdrops, withLive);
        const blob = await rasterizeSvg(built.svg, built.width, built.height, scale, "transparent", "png", !!withLive);
        return { blob, report: built.report };
    };
    let out: { blob: Blob; report: ExportReport };
    if (live && live.markup.size > 0) {
        // HTML inside an SVG image is the one thing a WebView may stall on or
        // refuse (officeImport met the same). The plain serializer never does,
        // so it is the fallback rather than an error.
        let timer: ReturnType<typeof setTimeout> | undefined;
        try {
            out = await Promise.race([
                draw(live),
                new Promise<never>((_, reject) => {
                    timer = setTimeout(() => reject(new Error("timed out")), LIVE_RASTER_TIMEOUT_MS);
                })
            ]);
        } catch (err) {
            console.warn("[capture] Drawing the canvas' own text failed; using the export serializer.", err);
            out = await draw();
        } finally {
            clearTimeout(timer);
        }
    } else {
        out = await draw();
    }
    return { blob: out.blob, width: size.width, height: size.height, report: out.report };
}

/** e.g. `Physics_notes_capture_2026-09-29_143205.png`. */
export function captureFilename(boardName: string, when: Date = new Date()): string {
    const base = (boardName || "capture").replace(/^[a-z0-9- ]/gi, "_").trim().replace(/ +/g, "_") || "capture";
    const p = (n: number) => String(n).padStart(2, "0");
    // Seconds too: screenshot mode takes several in a row, and the gallery
    // would otherwise rename them "(1)", "(2)".
    const stamp = `${when.getFullYear()}-${p(when.getMonth() + 1)}-${p(when.getDate())}_${p(when.getHours())}${p(when.getMinutes())}${p(when.getSeconds())}`;
    return `${base}_capture_${stamp}.png`;
}

```

```

=====
// === END FILE: src/lib/whiteboard/capture.ts ===
=====

=====
// === START FILE: src/lib/whiteboard/export.ts ===
// File 97 of 194 | Lines: 1229 | Size: 52326 bytes
=====

/**
 * Whiteboard Export Engine (v3 — SVG Vector & 2x Retina PNG Canvas)
 * =====
 * Serializes whiteboard boards and elements to standalone vector SVG or high-resolution PNG.
 */

import type {
  Board,
  WhiteboardElement,
  StrokeElement,
  RectangleElement,
  EllipseElement,
  DiamondElement,
  TriangleElement,
  StarElement,
  CloudElement,
  LineElement,
  ArrowElement,
  TextElement,
  StickyElement,
  ImageElement,
  ConnectionElement,
  FillStyle,
  DashStyle,
} from "./types";
import { STICKY_COLORS, PEN_TYPE_PRESETS, getLinkTypeColor, getLinkTypeLabel, BOARD_BACKGROUNDS } from "./types";
import type { NotebookPage } from "./types";
import {
  getElementBounds,
  getElementCenter,
  getUnionBounds,
  getOptimalConnectionAnchors,
  getConnectionPath,
  getArrowheadPath,
} from "./geometry";
import { buildStrokePath } from "./stroke";
import { imageSvgMarkup, isUnedited } from "./imageEdit";
import { detectLineDir, markdownToPlainLines, resolveLineDir } from "./textParser";

/**
 * Baked writing direction for one exported line: an explicit override wins,
 * otherwise the same first-strong-character rule the canvas uses. SVG/PNG/PDF
 * all rasterize through this SVG in a browser, so one attribute covers them.
 */
function exportLineDir(
  lineDirs: Array<"ltr" | "rtl" | "auto"> | undefined,
  lineIndex: number,
  lineText: string
): "ltr" | "rtl" {
  const d = resolveLineDir(lineDirs, lineIndex);
  return d === "auto" ? detectLineDir(lineText) : d;
}

import {
  getBoardElements,
  getPages,
  getActivePage,
  isNotebook,
  pageSizeOf,
  backgroundForPage,
  resolvePageRange,
  formatPageSelection,
  PAGE_GAP,
} from "./notebook";

/** A 1-based, inclusive run of notebook pages. */
export interface PageRange {
  from: number;

```

```

    to: number;
}
import { blobToDataURL, saveBlob, shareBlob } from "./share";
import type { ShareOutcome } from "./share";
import { zipSync } from "fflate";

/**
 * Packs finished export files into one .zip (batch sidebar export).
 * Filenames must already be unique – see exportFilename, which derives them
 * per board. Pure bytes in/out, so the sidebar's zip round-trips in node.
 */
export function zipExportFiles(files: Array<{ filename: string; data: Uint8Array }>): Uint8Array {
    const entries: Record<string, Uint8Array> = {};
    for (const f of files) entries[f.filename] = f.data;
    return zipSync(entries);
}

export { isNativeApp, canChooseApp, shareRoute, downloadBlob, saveBlob, shareBlob } from "./share";
export type { ShareOutcome, ShareRoute } from "./share";

/**
 * Every raster format the canvas can encode. JPEG and WEBP are here because
 * "export an image" means different things in different places - a mail client
 * that chokes on a 40 MB PNG takes the same notebook as a 2 MB JPEG happily.
 */
export type RasterFormat = "png" | "jpeg" | "webp";

export const RASTER_MIME: Record<RasterFormat, string> = {
    png: "image/png",
    jpeg: "image/jpeg",
    webp: "image/webp",
};

/** Every format an export can be written as. */
export type ExportFormat = RasterFormat | "svg" | "pdf";

export const EXPORT_EXTENSION: Record<ExportFormat, string> = {
    png: "png",
    jpeg: "jpg",
    webp: "webp",
    svg: "svg",
    pdf: "pdf",
};

/** JPEG and WEBP are lossy; 0.92 is the usual "you cannot see it" threshold. */
const LOSSY_QUALITY = 0.92;

export interface ExportOptions {
    background?: "white" | "dark" | "transparent";
    padding?: number;
    scale?: number;
    includeLocked?: boolean;
    /**
     * Notebooks only. "all" (the default) exports every page, "current" the page
     * on screen, and `{ from, to }` a 1-based inclusive range - "pages 4 to 9 of
     * this notebook", which is what you want when handing in one question of a
     * problem set. Whiteboards ignore it.
     */
    /**
     * Export used to read only the active page while the dialog counted every
     * page's elements, so a 20-page notebook advertised all of its content and
     * wrote out a single sheet.
     */
    pages?: "all" | "current" | PageRange | number[];
    /** Called as each sheet is rasterised, so the UI can show real progress. */
    onProgress?: ProgressFn;
}

/**
 * Progress while an export runs. A 40-page notebook takes long enough that a
 * button which simply sits there reads as a hang, so every page reports in.
 */
export interface ExportProgress {
    /** Sheets finished so far. */
    done: number;
    /** Sheets in total, 1 for a whiteboard. */
    total: number;
    /** A short line to show, e.g. "Rendering page 3 of 10". */
    label: string;
}

```

```

}

export type ProgressFn = (p: ExportProgress) => void;

/** What an export could not include, so the UI can say so instead of hiding it. */
export interface ExportReport {
  /** Pictures whose pixels were missing from IndexedDB, drawn as grey boxes. */
  missingImages: number;
  /** PDF-backed pages whose PDF could not be loaded, exported as plain paper. */
  missingPdfPages: number;
  /** Sheets exported (1 for a whiteboard). */
  pages: number;
}

function escapeXml(unsafe: string): string {
  if (!unsafe) return "";
  return unsafe
    .replace(/&/g, "&amp;")
    .replace(/</g, "&lt;")
    .replace(/>/g, "&gt;")
    .replace(/"/g, "&quot;")
    .replace(/'/g, "&apos;");
}

function getFillAndDash(
  fillStyle: string = "transparent",
  dashStyle: string = "solid",
  strokeColor: string = "#1e293b",
  fillColor: string = "transparent"
) {
  let fill = "transparent";
  if (fillStyle === "solid") fill = fillColor === "transparent" ? strokeColor : fillColor;
  else if (fillStyle === "semi") fill = fillColor === "transparent" ? `${strokeColor}33` : `${fillColor}40`;

  let strokeDasharray = "";
  if (dashStyle === "dashed") strokeDasharray = 'stroke-dasharray="8,6"';
  else if (dashStyle === "dotted") strokeDasharray = 'stroke-dasharray="3,4"';

  return { fill, strokeDasharray };
}

function getRotationTransform(el: WhiteboardElement): string {
  if (!el.rotation || el.rotation === 0) return "";
  const center = getElementCenter(el);
  const deg = (el.rotation * 180) / Math.PI;
  return `transform="rotate(${deg.toFixed(2)}, ${center.x.toFixed(2)}, ${center.y.toFixed(2)})"`;
}

/**
 * The transform the ON-SCREEN renderer wraps every element in:
 * `translate(x, y)` plus a rotation about the element's own centre.
 *
 * Most element types are written here with absolute coordinates already baked
 * in (a rect emits `x=${rect.x}`), so they must NOT be wrapped. Strokes are
 * the exception: their points are stored RELATIVE to the element origin, which
 * is why they need the same translate the renderer applies.
 */
function localTransform(el: WhiteboardElement): string {
  const rotate =
    el.rotation && el.rotation !== 0
      ? `rotate(${(el.rotation * 180) / Math.PI}.toFixed(2)}, ${((getElementCenter(el).x - el.x).toFixed(2)),
        ${{
          getElementCenter(el).y - el.y
        }.toFixed(2)})`
      : "";
  return `translate(${el.x}, ${el.y})${rotate}`;
}

/**
 * The SVG markup for a list of elements, in their own coordinates.
 *
 * Shared by whole-board export and per-page notebook export. `elements` is
 * also the lookup set for connections, so a connector only draws when both
 * ends are in the same exported frame.
 */
export function serializeElements(
  elements: WhiteboardElement[],
  background: "white" | "dark" | "transparent",

```

```

customLinkTypes: Array<{ id: string; name: string; color: string }>,
resolvedImages: Map<string, string> | undefined,
stats: { missingImages: number },
/**
 * Markup to use INSTEAD of the serializer for an element, or null to
 * serialize it. A capture hands in the canvas' own text boxes and notes,
 * so their wrapping matches the screen exactly.
 */
override?: (el: WhiteboardElement) => string | null
): string {
  let svgElements = "";

  // Render elements in order
  for (const el of elements) {
    const replaced = override?.(el);
    if (replaced) {
      svgElements += ` ${replaced}\n`;
      continue;
    }
    const opacity = el.opacity ?? 1;
    const opacityAttr = opacity < 1 ? `opacity="${opacity.toFixed(2)}"` : "";
    const rot = getRotationTransform(el);

    switch (el.type) {
      case "stroke": {
        const str = el as StrokeElement;
        // ALWAYS derive the path from the points, exactly as the on-screen
        // renderer does. The legacy `str.path` cache froze a stroke's geometry
        // at draw time; it is stripped on load and must never be preferred
        // here, or an export could disagree with the screen.
        const pathData = buildStrokePath(str, false);
        if (pathData) {
          const preset = PEN_TYPE_PRESETS[str.strokeStyle] || PEN_TYPE_PRESETS.pen;
          const strokeOpacity = (preset.opacity ?? 1) * opacity;
          // Canvas applies mixBlendMode for multiply pens (highlighter);
          // mirror it so exports don't flatten to opaque.
          const blend = preset.mixBlendMode === "multiply" ? ' style="mix-blend-mode: multiply"' : "";
          // A stroke's points are stored RELATIVE to its origin, so the path
          // has to be placed by the same translate the renderer uses. Without
          // it every stroke on every page collapsed into the top-left corner
          // of the sheet and was clipped away - handwriting simply vanished
          // from exported PDFs while looking perfect on screen.
          svgElements += ` <g transform="${localTransform(str)}"><path d="${pathData}" fill="${escapeXml(
            str.color
          )}" opacity="${strokeOpacity.toFixed(2)}"${blend} /></g>\n`;
        }
        break;
      }

      case "rectangle": {
        const rect = el as RectangleElement;
        const { fill, strokeDasharray } = getFillAndDash(rect.fillStyle, rect.dashStyle, rect.strokeColor, rect.fillColor);
        const rx = rect.cornerRadius || 6;
        svgElements += ` <rect x="${rect.x}" y="${rect.y}" width="${Math.max(rect.width, 2)}" height="${Math.max(rect.height, 2)}" rx="${rx}" ry="${ry}" fill="${escapeXml(fill)}" stroke="${escapeXml(rect.strokeColor)}" stroke-width="${rect.strokeWidth || 2}" ${strokeDasharray} ${opacityAttr} ${rot} />\n`;
        break;
      }

      case "ellipse": {
        const ell = el as EllipseElement;
        const { fill, strokeDasharray } = getFillAndDash(ell.fillStyle, ell.dashStyle, ell.strokeColor, ell.fillColor);
        const rx = Math.max(ell.width / 2, 1);
        const ry = Math.max(ell.height / 2, 1);
        const cx = ell.x + rx;
        const cy = ell.y + ry;
        svgElements += ` <ellipse cx="${cx}" cy="${cy}" rx="${rx}" ry="${ry}" fill="${escapeXml(fill)}" stroke="${escapeXml(ell.strokeColor)}" stroke-width="${ell.strokeWidth || 2}" ${strokeDasharray} ${opacityAttr} ${rot} />\n`;
        break;
      }

      case "diamond": {
        const dia = el as DiamondElement;
        const { fill, strokeDasharray } = getFillAndDash(dia.fillStyle, dia.dashStyle, dia.strokeColor, dia.fillColor);

```

```

        llColor);
const w = dia.width;
const h = dia.height;
const pts = `${dia.x + w / 2},${dia.y} ${dia.x + w},${dia.y + h / 2} ${dia.x + w / 2},${dia.y + h} ${dia.x},${dia.y + h / 2}`;
svgElements += `<polygon points="${pts}" fill="${escapeXml(fill)}" stroke="${escapeXml(dia.strokeColor)}" stroke-width="${dia.strokeWidth || 2}" ${strokeDasharray} ${opacityAttr} ${rot} />\n`;
break;
}

case "triangle": {
const tri = el as TriangleElement;
const { fill, strokeDasharray } = getFillAndDash(tri.fillStyle, tri.dashStyle, tri.strokeColor, tri.fillColor);
const w = tri.width;
const h = tri.height;
const pts = `${tri.x + w / 2},${tri.y} ${tri.x + w},${tri.y + h} ${tri.x},${tri.y + h}`;
svgElements += `<polygon points="${pts}" fill="${escapeXml(fill)}" stroke="${escapeXml(tri.strokeColor)}" stroke-width="${tri.strokeWidth || 2}" ${strokeDasharray} ${opacityAttr} ${rot} />\n`;
break;
}

case "star": {
const star = el as StarElement;
const { fill, strokeDasharray } = getFillAndDash(star.fillStyle, star.dashStyle, star.strokeColor, star.fillColor);
const cx = star.x + star.width / 2;
const cy = star.y + star.height / 2;
const spikes = 5;
const outerR = Math.min(star.width, star.height) / 2;
const innerR = outerR * 0.45;
let pts = "";
for (let i = 0; i < spikes * 2; i++) {
const r = i % 2 === 0 ? outerR : innerR;
const angle = (i * Math.PI) / spikes - Math.PI / 2;
const px = cx + Math.cos(angle) * r;
const py = cy + Math.sin(angle) * r;
pts += `${px.toFixed(1)},${py.toFixed(1)} `;
}
svgElements += `<polygon points="${pts.trim()}" fill="${escapeXml(fill)}" stroke="${escapeXml(star.strokeColor)}" stroke-width="${star.strokeWidth || 2}" ${strokeDasharray} ${opacityAttr} ${rot} />\n`;
break;
}

case "cloud": {
const cld = el as CloudElement;
const { fill, strokeDasharray } = getFillAndDash(cld.fillStyle, cld.dashStyle, cld.strokeColor, cld.fillColor);
const w = cld.width;
const h = cld.height;
const r = Math.min(w, h) * 0.2;
const d = `M ${cld.x + r * 1.5},${cld.y + h - r} A ${r},${r} 0 0,1 ${cld.x + r},${cld.y + h * 0.5} A ${r * 1.2},${r * 1.2} 0 0,1 ${cld.x + w * 0.35},${cld.y + r} A ${r * 1.5},${r * 1.5} 0 0,1 ${cld.x + w * 0.7},${cld.y + r * 1.2} A ${r},${r} 0 0,1 ${cld.x + w - r * 0.5},${cld.y + h * 0.55} A ${r * 0.9},${r * 0.9} 0 0,1 ${cld.x + w - r},${cld.y + h - r} Z`;
svgElements += `<path d="${d}" fill="${escapeXml(fill)}" stroke="${escapeXml(cld.strokeColor)}" stroke-width="${cld.strokeWidth || 2}" ${strokeDasharray} ${opacityAttr} ${rot} />\n`;
break;
}

case "line": {
const line = el as LineElement;
let strokeDasharray = "";
if (line.dashStyle === "dashed") strokeDasharray = 'stroke-dasharray="8,6"';
else if (line.dashStyle === "dotted") strokeDasharray = 'stroke-dasharray="3,4"';
svgElements += `<line x1="${line.x}" y1="${line.y}" x2="${line.x + line.x2}" y2="${line.y + line.y2}" stroke="${escapeXml(line.strokeColor)}" stroke-width="${line.strokeWidth || 2}" stroke-linecap="round" ${strokeDasharray} ${opacityAttr} ${rot} />\n`;
break;
}

case "arrow": {
const arr = el as ArrowElement;
let strokeDasharray = "";
if (arr.dashStyle === "dashed") strokeDasharray = 'stroke-dasharray="8,6"';
else if (arr.dashStyle === "dotted") strokeDasharray = 'stroke-dasharray="3,4"';
const head = getArrowheadPath(

```

```

    arr.x + arr.x2,
    arr.y + arr.y2,
    arr.x,
    arr.y,
    arr.endArrowhead || "arrow",
    arr.arrowheadSize || 12
  );
  svgElements += ` <g ${opacityAttr} ${rot}>\n`;
  svgElements += ` <line x1="${arr.x}" y1="${arr.y}" x2="${arr.x + arr.x2}" y2="${arr.y + arr.y2}" st
    roke="${escapeXml(arr.strokeColor)}" stroke-width="${arr.strokeWidth || 2}" stroke-linecap="round"
    ${strokeDasharray} />\n`;
  if (head) {
    svgElements += ` <path d="${head.path}" fill="${escapeXml(arr.strokeColor)}" stroke="${head.strok
      e || "none"}" />\n`;
  }
  svgElements += ` </g>\n`;
  break;
}

case "text": {
  const txt = el as TextElement;
  const fontSize = txt.fontSize || 18;
  const fontColor = txt.color || (background === "dark" ? "#f4f4f5" : "#1e293b");
  // The lines as the canvas shows them: markdown markers become their
  // visual form (■, •, headings) and [[WikiLink|alias]] its alias.
  const lines = markdownToPlainLines(txt.text || "");
  // Card mode draws the same box the canvas does, and its text sits
  // inside the padding.
  const isCard = !(txt.cardMode || (txt.backgroundColor && txt.backgroundColor !== "transparent"));
  const pad = isCard ? txt.padding || 12 : 0;
  // Anchor x must account for the element width – the on-canvas renderer
  // centers/right-aligns inside the text box, not at its left edge.
  const anchorX =
    txt.align === "center"
      ? txt.x + txt.width / 2
      : txt.align === "right"
        ? txt.x + txt.width - pad
        : txt.x + pad;
  svgElements += ` <g ${opacityAttr} ${rot}>\n`;
  if (isCard) {
    svgElements += ` <rect x="${txt.x}" y="${txt.y}" width="${txt.width}" height="${txt.height}" rx="
      ${txt.borderRadius || 8}" ry="${txt.borderRadius || 8}" fill="${escapeXml(txt.backgroundColor ||
        "ffffff")}" stroke="${escapeXml(txt.borderColor || "rgba(0,0,0,0.12)")}" stroke-width="1.5" />
      \n`;
  }
  let yPos = txt.y + pad;
  lines.forEach((line) => {
    const size = fontSize * line.scale;
    yPos += size * 1.35;
    const weight = Math.max(line.weight ?? 0, Number(txt.fontWeight) || 400);
    svgElements += ` <text x="${anchorX.toFixed(2)}" y="${yPos.toFixed(2)}" fill="${escapeXml(fontCol
      or)}" font-family="${escapeXml(txt.fontFamily || "Inter, system-ui, sans-serif")}" font-size="${
      size.toFixed(2)}" font-weight="${weight}" text-anchor="${txt.align === "center" ? "middle" : txt
        .align === "right" ? "end" : "start"}" direction="${exportLineDir(txt.lineDirs, line.lineIndex,
        line.text)}" style="unicode-bidi:embed" xml:space="preserve">${escapeXml(line.text)}</text>\n`;
  });
  svgElements += ` </g>\n`;
  break;
}

case "sticky": {
  const st = el as StickyElement;
  const colors = STICKY_COLORS[st.color] || STICKY_COLORS.yellow;
  const fontSize = st.fontSize || 16;
  const lines = markdownToPlainLines(st.text || "");
  const pad = 14;
  const stickyAnchorX =
    st.align === "center"
      ? st.x + st.width / 2
      : st.align === "right"
        ? st.x + st.width - pad
        : st.x + pad;
  svgElements += ` <g ${opacityAttr} ${rot}>\n`;
  svgElements += ` <rect x="${st.x}" y="${st.y}" width="${st.width}" height="${st.height}" rx="8" ry=
    "8" fill="${escapeXml(colors.bg)}" stroke="${escapeXml(colors.border)}" stroke-width="1.5" />\n`;
  let stickyY = st.y + 24;
  lines.forEach((line) => {
    const size = fontSize * line.scale;

```



```

    stickyY += size * 1.3;
    if (stickyY < st.y + st.height - 8) {
        svgElements += `
            <text x="${stickyAnchorX.toFixed(2)}" y="${stickyY.toFixed(2)}" fill="${escape
                Xml(colors.text)}" font-family="Inter, system-ui, sans-serif" font-size="${size.toFixed(2)}"${
                    line.weight ? `font-weight="${line.weight}"` : ""} text-anchor="${st.align === "center" ? "mi
                    ddle" : st.align === "right" ? "end" : "start"}" direction="${exportLineDir(st.lineDirs, line
                    .lineIndex, line.text)}" style="unicode-bidi:embed" xml:space="preserve">${escapeXml(line.text)}
            </text>\n`;
    }
});
svgElements += ` </g>\n`;
break;
}

case "image": {
    const img = el as ImageElement;
    // An unresolved asset ref would write a href no viewer can load, so the
    // image is omitted rather than exported broken.
    const href = resolvedImages?.get(img.src) ?? img.src;
    if (href && !href.startsWith("asset:") && !isUnedited(img)) {
        // Cropped, flipped, filtered or framed: the same layout the canvas draws.
        svgElements += ` <g ${opacityAttr} ${rot}>${imageSvgMarkup(img, href)}</g>\n`;
    } else if (href && !href.startsWith("asset:")) {
        svgElements += ` <image href="${escapeXml(href)}" x="${img.x}" y="${img.y}" width="${img.width}" he
            ight="${img.height}" preserveAspectRatio="xMidYMid meet" ${opacityAttr} ${rot} />\n`;
    } else {
        // The pixels are gone (IndexedDB cleared, or never stored). A grey
        // placeholder keeps the layout honest, and it is COUNTED so the user
        // is told, instead of finding a hole in the printout later.
        stats.missingImages++;
        svgElements += ` <rect x="${img.x}" y="${img.y}" width="${img.width}" height="${img.height}" rx="6"
            fill="#e4e4e7" stroke="#a1a1aa" ${opacityAttr} ${rot} />\n`;
    }
}
break;
}

case "connection": {
    const conn = el as ConnectionElement;
    const sourceEl = elements.find((e) => e.id === conn.source.elementId);
    const targetEl = elements.find((e) => e.id === conn.target.elementId);
    if (sourceEl && targetEl) {
        const { source, target } = getOptimalConnectionAnchors(sourceEl, targetEl, conn.source.anchor, conn
            .target.anchor);
        const pathD = getConnectionPath(source, target, conn.routing || "curved");
        const connColor = getLinkTypeColor(conn.linkType, customLinkTypes);
        const endHead = getArrowheadPath(
            target.x,
            target.y,
            target.x + target.nx * 20,
            target.y + target.ny * 20,
            conn.endArrowhead || "arrow",
            12
        );
        const startHead =
            conn.startArrowhead && conn.startArrowhead !== "none"
                ? getArrowheadPath(
                    source.x,
                    source.y,
                    source.x - source.nx * 20,
                    source.y - source.ny * 20,
                    conn.startArrowhead,
                    12
                )
                : null;

        let strokeDasharray = "";
        if (conn.dashStyle === "dashed") strokeDasharray = 'stroke-dasharray="6,4"';
        else if (conn.dashStyle === "dotted") strokeDasharray = 'stroke-dasharray="2,4"';

        svgElements += ` <g ${opacityAttr}>\n`;
        svgElements += ` <path d="${pathD}" fill="none" stroke="${escapeXml(connColor)}" stroke-width="${
            conn.strokeWidth || 2.5}" stroke-linecap="round" ${strokeDasharray} />\n`;
        if (startHead) {
            svgElements += ` <path d="${startHead.path}" fill="${escapeXml(connColor)}" stroke="none" />\n`
                ;
        }
        if (endHead) {
            svgElements += ` <path d="${endHead.path}" fill="${escapeXml(connColor)}" stroke="none" />\n`;
        }
    }
}

```

```

    }

    const label = conn.label ?? getLinkTypeLabel(conn.linkType, customLinkTypes);
    if (label) {
      const pillLabel = label.length > 40 ? `${label.slice(0, 39)}...` : label;
      const midX = (source.x + target.x) / 2;
      const midY = (source.y + target.y) / 2;
      const rectW = pillLabel.length * 8.2 + 24;
      svgElements += `      <g transform="translate(${midX}, ${midY})">\n`;
      svgElements += `        <rect x="${-(rectW / 2)}" y="-11" width="${rectW}" height="22" rx="11" fill=
        "${background} === "dark" ? "#27272a" : "ffffff" stroke="${connColor}" stroke-width="1.5" />
        \n`;
      svgElements += `        <text x="0" y="3.5" fill="${escapeXml(connColor)}" text-anchor="middle" font
        -size="11" font-weight="600" font-family="Inter, system-ui, sans-serif">${escapeXml(pillLabel)}
        </text>\n`;
      svgElements += `      </g>\n`;
    }
    svgElements += `    </g>\n`;
  }
  break;
}
}
}

return svgElements;
}

/**
 * Serializes a whiteboard board into a standalone, valid XML SVG string.
 */
/**
 * Image pixels now live in IndexedDB, referenced as `asset:<id>` (see
 * lib/whiteboard/imagestore.ts). An exported SVG is a standalone file, so
 * those references mean nothing outside the app - every image would come out
 * blank. This resolves them to real data URLs first.
 */
export async function resolveExportImages(board: Board): Promise<Map<string, string>> {
  const { isAssetRef, assetIdOf, loadImage } = await import("./imagestore");
  const { allElements } = await import("./notebook");

  const refs = new Set<string>();
  for (const el of allElements(board)) {
    const src = (el as { src?: string }).src;
    if (src && isAssetRef(src)) refs.add(src);
  }

  const out = new Map<string, string>();
  await Promise.all(
    [...refs].map(async (ref) => {
      const url = await loadImage(assetIdOf(ref));
      if (url) out.set(ref, url);
    })
  );
  return out;
}

/**
 * PDF page rasters for notebook pages that sit on an imported PDF, keyed by
 * page id. Without these an annotated lecture exports as the annotations alone,
 * floating on blank paper, which is useless as a study copy.
 *
 * A page whose PDF bytes are missing is left out of the map and counted by the
 * caller, so the export is still produced and the user is told.
 */
export async function resolvePageBackdrops(
  pages: NotebookPage[],
  targetWidth: number,
  /** Raster density per world unit. A capture asks for up to 3 so a small
   * crop of a lecture slide stays as sharp as the screen showed it. */
  dpr = 2
): Promise<Map<string, string>> {
  const out = new Map<string, string>();
  const withPdf = pages.filter((p) => p.pdf);
  if (withPdf.length === 0) return out;

  const { getPdf } = await import("./pdfstore");
  const { openPdf, renderPdfPage } = await import("./pdf");
  type Doc = Awaited<ReturnType<typeof openPdf>>;

```

```

const docs = new Map<string, Doc | null>();

for (const page of withPdf) {
  const ref = page.pdf!;
  if (!docs.has(ref.docId)) {
    try {
      const stored = await getPdf(ref.docId);
      docs.set(ref.docId, stored ? await openPdf(stored.bytes) : null);
    } catch {
      docs.set(ref.docId, null);
    }
  }
  const doc = docs.get(ref.docId);
  if (!doc) continue;
  try {
    out.set(page.id, await renderPdfPage(doc, ref.pageNumber, targetWidth, dpr));
  } catch {
    /* a page that will not render exports as plain paper, and is counted */
  }
}
for (const doc of docs.values()) {
  try {
    // Frees the rendered pages; the document itself is garbage collected.
    await doc?.cleanup();
  } catch {
    /* already torn down */
  }
}
return out;
}

/** The pages an export covers, per `options.pages`. */
function exportPages(board: Board, options: ExportOptions): NotebookPage[] {
  const scope = options.pages;
  if (scope === "current") {
    const active = getActivePage(board);
    return active ? [active] : [];
  }
  const all = getPages(board);
  // An explicit list of 1-based page numbers - what the page picker produces.
  if (Array.isArray(scope)) {
    return scope
      .filter((n) => Number.isFinite(n) && n >= 1 && n <= all.length)
      .sort((a, b) => a - b)
      .map((n) => all[n - 1]);
  }
  if (scope && typeof scope === "object") {
    return resolvePageRange(all.length, scope.from, scope.to).indices.map((i) => all[i]);
  }
  return all;
}

/**
 * Paper colour and ruling for one sheet, matching what the canvas draws.
 * `page` null means a whiteboard's own paper.
 */
export function paperFor(board: Board, page: NotebookPage | null, background: "white" | "dark" | "transparent") {
  const bg = backgroundForPage(board, page) ?? board.background;
  const dark = background === "dark";
  const preset = BOARD_BACKGROUNDS.find((x) => x.id === bg?.color);
  const chosen = preset ? (dark ? preset.dark : preset.light) : bg?.color || "";
  return {
    fill: chosen || (dark ? "#18181b" : "#ffffff"),
    // zinc-300 / zinc-700: the colours the on-screen paper pattern uses.
    ink: dark ? "#3f3f46" : "#d4d4d8",
    pattern: bg?.pattern ?? "dots",
    size: Math.max(6, Math.min(120, bg?.size ?? 24)),
  };
}

/**
 * The ruling as an SVG `` in world units, tiled from the origin of
 * whatever user space references it - a sheet's top-left corner, or a
 * whiteboard's world origin, exactly as the canvas tiles it. Empty for plain
 * paper.
 */
export function paperPatternSvg(id: string, paper: ReturnType<typeof paperFor>): string {

```

```

if (paper.pattern === "plain") return "";
const z = paper.size;
const shape =
  paper.pattern === "dots"
    ? `<circle cx="${z / 2}" cy="${z / 2}" r="${Math.max(0.5, z / 20)}" fill="${paper.ink}" />`
    : paper.pattern === "grid"
    ? `<path d="M ${z} 0 L 0 0 0 ${z}" fill="none" stroke="${paper.ink}" stroke-width="0.8" />`
    : `<path d="M 0 ${z} L ${z} ${z}" fill="none" stroke="${paper.ink}" stroke-width="0.8" />`;
return `<pattern id="${id}" x="0" y="0" width="${z}" height="${z}" patternUnits="userSpaceOnUse">${shape}</pattern>`;
}

/**
 * Which layers of a sheet to paint. Export wants all of them; a capture with
 * "white" or "no" background drops the paper and ruling but, for white, keeps
 * the PDF page - that is content the user is looking at, not decoration.
 */
export interface SheetLayers {
  /** false: no paper at all; a string: paint this colour instead of the page's. */
  paper?: false | string;
  ruling?: boolean;
  backdrop?: boolean;
  /** Which of the page's elements to draw (a capture of the selection only). */
  keep?: (el: WhiteboardElement) => boolean;
  /** Replacement markup per element; see `serializeElements`. */
  override?: (el: WhiteboardElement) => string | null;
}

/**
 * One sheet as an SVG group, drawn at `offsetY`.
 *
 * Everything is CLIPPED to the sheet. Ink from before sheets were enforced can
 * sit outside the page; on screen it is invisible past the paper edge, and an
 * export that included it came out as a smear the user never saw.
 */
export function sheetSvg(
  board: Board,
  page: NotebookPage,
  index: number,
  offsetY: number,
  width: number,
  height: number,
  options: ExportOptions,
  customLinkTypes: Array<{ id: string; name: string; color: string }>,
  resolvedImages: Map<string, string> | undefined,
  backdrops: Map<string, string> | undefined,
  report: ExportReport,
  outlined: boolean,
  layers: SheetLayers = {}
): string {
  const background = options.background ?? "white";
  const includeLocked = options.includeLocked ?? true;
  const paper = paperFor(board, page, background);
  const clipId = `pn-sheet-${index}`;
  const patId = `pn-paper-${index}`;
  const wantBackdrop = layers.backdrop !== false;
  const backdrop = wantBackdrop ? backdrops?.get(page.id) : undefined;
  if (wantBackdrop && page.pdf && !backdrop) report.missingPdfPages++;

  // Ruling goes OVER a PDF scan when the page asks for it, matching the
  // canvas. PDF notebooks default to plain, so scans stay clean unless you
  // pick a ruling for that page.
  const pattern = layers.ruling === false ? "" : paperPatternSvg(patId, paper);
  const patternRect = pattern
    ? `<rect x="0" y="0" width="${width}" height="${height}" fill="url(#${patId})" />`
    : "";
  const fill = layers.paper === false ? null : layers.paper ?? paper.fill;

  const elements = (page.elements ?? []).filter(
    (el) => (includeLocked || !el.locked) && (!layers.keep || layers.keep(el))
  );
  const body = serializeElements(elements, background, customLinkTypes, resolvedImages, report, layers.override);

  // The PDF page is letterboxed ("meet"), never stretched, exactly as
  // PdfPageLayer draws it on the canvas: a slide inserted into an A4 notebook
  // keeps its own proportions in the file too.
  return `<g transform="translate(0, ${offsetY})">`;
}

```

```

<defs><clipPath id="${clipId}"><rect x="0" y="0" width="${width}" height="${height}" /></clipPath>${pattern}
</defs>
${fill ? `<rect x="0" y="0" width="${width}" height="${height}" fill="${fill}" />` : ""}
${backdrop ? `<image href="${escapeXml(backdrop)}" x="0" y="0" width="${width}" height="${height}" preserv
eAspectRatio="xMidYMid meet" />` : ""}${patternRect}
<g clip-path="url(#${clipId})">
${body} </g>
${outlined ? `<rect x="0" y="0" width="${width}" height="${height}" fill="none" stroke="#d4d4d8" stroke-wi
dth="1" />` : ""}
</g>
`;
}

// No webfont @import here. An SVG rasterised through an Image cannot load
// external resources at all - the rule was silently dropped every time - and
// reaching for fonts.googleapis.com would break the app's offline promise on
// a tablet with no connection. The stack below resolves locally on every
// platform the app ships to.
export const SVG_HEAD_STYLE = ` <defs>
  <style>
    text { font-family: 'Inter', -apple-system, BlinkMacSystemFont, 'Segoe UI', Roboto, 'Helvetica Neue', Ar
      ial, sans-serif; }
  </style>
</defs>`;

/**
 * Builds the export SVG and reports its size and anything left out.
 *
 * A notebook exports as its SHEETS - every page by default, stacked with the
 * same gap as the scroll view - each clipped to the paper with its ruling and
 * any PDF underneath. A whiteboard exports the bounding box of its content, as
 * before.
 */
export function buildExportSvg(
  board: Board,
  options: ExportOptions = {},
  customLinkTypes: Array<{ id: string; name: string; color: string }> = [],
  resolvedImages?: Map<string, string>,
  backdrops?: Map<string, string>
): { svg: string; width: number; height: number; report: ExportReport } {
  const background = options.background ?? "white";
  const report: ExportReport = { missingImages: 0, missingPdfPages: 0, pages: 1 };
  const bgFill = background === "white" ? "#ffffff" : background === "dark" ? "#18181b" : "transparent";

  if (isNotebook(board)) {
    const { width, height } = pageSizeOf(board);
    const pages = exportPages(board, options);
    report.pages = pages.length;
    const count = Math.max(1, pages.length);
    const totalH = count * height + (count - 1) * PAGE_GAP;
    const stacked = pages.length > 1;

    let sheets = "";
    pages.forEach((page, i) => {
      sheets += sheetSvg(
        board, page, i, i * (height + PAGE_GAP), width, height,
        options, customLinkTypes, resolvedImages, backdrops, report, stacked
      );
    });

    const svg = `<?xml version="1.0" encoding="UTF-8"?>
<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 ${width} ${totalH}" width="${width}" height="${totalH}">
${SVG_HEAD_STYLE}
  ${stacked && bgFill !== "transparent" ? `<rect x="0" y="0" width="${width}" height="${totalH}" fill="${bgFil
l}" />` : ""}
${sheets}</svg>`;
    return { svg, width, height: totalH, report };
  }

  const padding = options.padding ?? 40;
  const includeLocked = options.includeLocked ?? true;
  const elements = getBoardElements(board).filter((el) => includeLocked || !el.locked);

  const rawBounds = getUnionBounds(elements);
  const bounds = rawBounds ?? { x: -400, y: -300, width: 800, height: 600 };
  const exportX = bounds.x - padding;
  const exportY = bounds.y - padding;
  const exportW = Math.max(bounds.width + padding * 2, 100);

```

```

const exportH = Math.max(bounds.height + padding * 2, 100);

const svgElements = serializeElements(elements, background, customLinkTypes, resolvedImages, report);

const svg = `<?xml version="1.0" encoding="UTF-8"?>
<svg xmlns="http://www.w3.org/2000/svg" viewBox="${exportX.toFixed(2)} ${exportY.toFixed(2)} ${exportW.toFixed(
2)} ${exportH.toFixed(2)}" width="${exportW.toFixed(2)}" height="${exportH.toFixed(2)}">
${SVG_HEAD_STYLE}
  {bgFill !== "transparent" ? `<rect x="${exportX.toFixed(2)}" y="${exportY.toFixed(2)}" width="${exportW.toF
ixed(2)}" height="${exportH.toFixed(2)}" fill="${bgFill}" />` : ""}
${svgElements}</svg>`;
return { svg, width: exportW, height: exportH, report };
}

export function exportBoardToSvg(
  board: Board,
  options: ExportOptions = {},
  customLinkTypes: Array<{ id: string; name: string; color: string }> = [],
  /** `asset:<id>` -> data URL, from `resolveExportImages`. */
  resolvedImages?: Map<string, string>,
  /** page id -> PDF raster, from `resolvePageBackdrops`. */
  backdrops?: Map<string, string>
): string {
  return buildExportSvg(board, options, customLinkTypes, resolvedImages, backdrops).svg;
}

/** Everything an export needs from IndexedDB, fetched once. */
async function resolveExportAssets(board: Board, options: ExportOptions) {
  const images = await resolveExportImages(board);
  const backdrops = isNotebook(board)
    ? await resolvePageBackdrops(exportPages(board, options), pageSizeOf(board).width)
    : new Map<string, string>();
  return { images, backdrops };
}

/**
 * Canvas limits. Desktop Chrome allows ~16k px a side, but an Android WebView
 * gives out far earlier on AREA - a 30-page notebook stacked at 2x is ~1600 x
 * 70000 px and simply produces a blank or failed canvas. The scale is lowered
 * to fit rather than failing.
 */
const MAX_CANVAS_SIDE = 16000;
const MAX_CANVAS_AREA = 40_000_000;

export function fitScale(width: number, height: number, wanted: number): number {
  let s = wanted;
  s = Math.min(s, MAX_CANVAS_SIDE / Math.max(width, height, 1));
  s = Math.min(s, Math.sqrt(MAX_CANVAS_AREA / Math.max(width * height, 1)));
  return Math.max(0.1, s);
}

/** Draws an SVG string onto a canvas and returns it as a PNG. */
export function rasterizeSvg(
  svgString: string,
  width: number,
  height: number,
  scale: number,
  background: "white" | "dark" | "transparent",
  format: RasterFormat = "png",
  /**
   * Load the SVG from a data: URL instead of a blob: URL. Chrome taints the
   * canvas when a BLOB-loaded SVG contains a <foreignObject> (HTML), which
   * makes toBlob throw; the same SVG from a data URL stays exportable. Only
   * a capture carries HTML, so exports keep the cheaper blob URL.
   */
  viaDataURL = false
): Promise<Blob> {
  const s = fitScale(width, height, scale);
  const canvas = document.createElement("canvas");
  canvas.width = Math.max(1, Math.round(width * s));
  canvas.height = Math.max(1, Math.round(height * s));
  const ctx = canvas.getContext("2d");
  if (!ctx) return Promise.reject(new Error("Could not create 2D canvas context for PNG export"));

  const url = viaDataURL
    ? `data:image/svg+xml;charset=utf-8,${encodeURIComponent(svgString)}`
    : URL.createObjectURL(new Blob([svgString], { type: "image/svg+xml;charset=utf-8" }));
  return new Promise<Blob>((resolve, reject) => {

```

```

const img = new Image();
img.crossOrigin = "anonymous";
img.onload = () => {
  try {
    // JPEG cannot store transparency: without a painted background the
    // encoder fills it black, so ink and paper both disappear.
    const opaque = background !== "transparent" || format === "jpeg";
    if (opaque) {
      ctx.fillStyle = background === "dark" ? "#18181b" : "#ffffff";
      ctx.fillRect(0, 0, canvas.width, canvas.height);
    }
    ctx.drawImage(img, 0, 0, canvas.width, canvas.height);
    URL.revokeObjectURL(url);
    canvas.toBlob(
      (blob) => {
        if (!blob) {
          reject(new Error("Canvas toBlob failed"));
          return;
        }
        // A browser that cannot encode WEBP silently hands back a PNG.
        // Saying so beats a file whose contents do not match its name.
        if (blob.type && blob.type !== RASTER_MIME[format]) {
          reject(new Error(`This browser cannot write ${format.toUpperCase()} images. Try PNG.`));
          return;
        }
        resolve(blob);
      },
      RASTER_MIME[format],
      format === "png" ? undefined : LOSSY_QUALITY
    );
  } catch (err) {
    URL.revokeObjectURL(url);
    reject(err);
  }
};
img.onerror = (err) => {
  URL.revokeObjectURL(url);
  reject(new Error(`Failed to load SVG into image for rasterization: ${err}`));
};
img.src = url;
});
}

/** Raster export (PNG/JPEG/WEBP) plus what it had to leave out. */
export async function exportBoardToRasterWithReport(
  board: Board,
  options: ExportOptions = {},
  customLinkTypes: Array<{ id: string; name: string; color: string }> = [],
  format: RasterFormat = "png"
): Promise<{ blob: Blob; report: ExportReport }> {
  options.onProgress?.({ done: 0, total: 1, label: "Rendering" });
  const { images, backdrops } = await resolveExportAssets(board, options);
  const built = buildExportSvg(board, options, customLinkTypes, images, backdrops);
  await yieldToPaint();
  const blob = await rasterizeSvg(
    built.svg,
    built.width,
    built.height,
    options.scale ?? 2,
    options.background ?? "white",
    format
  );
  options.onProgress?.({ done: 1, total: 1, label: "Finishing" });
  return { blob, report: built.report };
}

/** PNG export plus what it had to leave out. */
export async function exportBoardToPngWithReport(
  board: Board,
  options: ExportOptions = {},
  customLinkTypes: Array<{ id: string; name: string; color: string }> = []
): Promise<{ blob: Blob; report: ExportReport }> {
  return exportBoardToRasterWithReport(board, options, customLinkTypes, "png");
}

/**
 * Rasterizes a board to a high-resolution PNG. A notebook comes out as its
 * pages stacked top to bottom.

```

```

*/
export async function exportBoardToPng(
  board: Board,
  options: ExportOptions = {},
  customLinkTypes: Array<{ id: string; name: string; color: string }> = []
): Promise<Blob> {
  return (await exportBoardToPngWithReport(board, options, customLinkTypes)).blob;
}

/**
 * A filename that every target accepts.
 *
 * Android's share sheet, the Windows file dialog and ext4 disagree about what
 * is legal, so this keeps to the intersection. The page range is part of the
 * name on purpose: three exports of one notebook otherwise land in Downloads
 * as three files nobody can tell apart.
 */
export function exportFilename(board: Board, format: ExportFormat, options: ExportOptions = {}): string {
  const base =
    (board.name || "whiteboard").replace(/^[a-z0-9-_]/gi, "_").trim().replace(/ +/g, "_") || "whiteboard";
  let suffix = "";
  const scope = options.pages;
  if (isNotebook(board)) {
    if (scope === "current") {
      const activeId = getActivePage(board)?.id;
      const i = getPages(board).findIndex((p) => p.id === activeId);
      if (i >= 0) suffix = `_p${i + 1}`;
    } else if (Array.isArray(scope)) {
      const all = getPages(board).length;
      const chosen = scope.filter((n) => n >= 1 && n <= all);
      // Only name the pages when it is a subset worth distinguishing, and keep
      // it short: a 40-page pick would otherwise produce an unusable filename.
      if (chosen.length > 0 && chosen.length < all) {
        const label = formatPageSelection(chosen).replace(/,\\s*/g, "_");
        suffix = label.length <= 24 ? `_p${label}` : `_${chosen.length}pages`;
      }
    } else if (scope && typeof scope === "object") {
      const r = resolvePageRange(getPages(board).length, scope.from, scope.to);
      suffix = r.from === r.to ? `_p${r.from}` : `_p${r.from}-${r.to}`;
    }
  }
  return `${base}${suffix}.${EXPORT_EXTENSION[format]}`;
}

/** Builds the file for any format, without deciding where it goes. */
export async function buildExportBlob(
  board: Board,
  format: ExportFormat,
  options: ExportOptions = {},
  customLinkTypes: Array<{ id: string; name: string; color: string }> = []
): Promise<{ blob: Blob; report: ExportReport; filename: string }> {
  const filename = exportFilename(board, format, options);
  if (format === "svg") {
    const { images, backdrops } = await resolveExportAssets(board, options);
    const built = buildExportSvg(board, options, customLinkTypes, images, backdrops);
    return {
      blob: new Blob([built.svg], { type: "image/svg+xml;charset=utf-8" }),
      report: built.report,
      filename,
    };
  }
  if (format === "pdf") {
    const { blob, report } = await exportBoardToPdfWithReport(board, options, customLinkTypes);
    return { blob, report, filename };
  }
  const { blob, report } = await exportBoardToRasterWithReport(board, options, customLinkTypes, format);
  return { blob, report, filename };
}

/**
 * Exports and delivers a board in one call.
 *
 * `destination` is the user's choice between "put it in my Downloads" and
 * "let me pick an app", and the outcome comes back so the UI can report what
 * really happened - including that they dismissed the share sheet.
 */
export async function exportBoard(
  board: Board,

```



```

    format: ExportFormat,
    options: ExportOptions = {},
    customLinkTypes: Array<{ id: string; name: string; color: string }> = [],
    destination: "save" | "share" = "save"
  ): Promise<{ report: ExportReport; outcome: ShareOutcome; filename: string }> {
    const { blob, report, filename } = await buildExportBlob(board, format, options, customLinkTypes);
    const outcome =
      destination === "share" ? await shareBlob(blob, filename, board.name) : await saveBlob(blob, filename);
    return { report, outcome, filename };
  }

/**
 * Exports and downloads SVG vector file.
 */
export async function downloadSvg(
  board: Board,
  options: ExportOptions = {},
  filename?: string,
  customLinkTypes: Array<{ id: string; name: string; color: string }> = []
): Promise<ExportReport> {
  const { images, backdrops } = await resolveExportAssets(board, options);
  const built = buildExportSvg(board, options, customLinkTypes, images, backdrops);
  const blob = new Blob([built.svg], { type: "image/svg+xml;charset=utf-8" });
  const safeName = (filename || board.name || "whiteboard").replace(/^[a-z0-9-]/gi, "_");
  await saveBlob(blob, `${safeName}.svg`);
  return built.report;
}

/**
 * Exports and downloads PNG raster file.
 */
export async function downloadPng(
  board: Board,
  options: ExportOptions = {},
  filename?: string,
  customLinkTypes: Array<{ id: string; name: string; color: string }> = []
): Promise<ExportReport> {
  const { blob, report } = await exportBoardToPngWithReport(board, options, customLinkTypes);
  const safeName = (filename || board.name || "whiteboard").replace(/^[a-z0-9-]/gi, "_");
  await saveBlob(blob, `${safeName}.png`);
  return report;
}

// -----
// PDF
// -----

/**
 * Largest page side most PDF viewers will open (200in at 72pt/in). A big
 * whiteboard can exceed it, so the page is scaled down to fit rather than
 * producing a file that silently fails to render.
 */
const PDF_MAX_POINTS = 14400;

/**
 * Yields to the browser between sheets.
 *
 * Rasterising is synchronous enough to starve the main thread, which freezes
 * the very progress bar that is meant to show the export moving - on a tablet
 * it reads as the app having hung. One frame between pages costs nothing and
 * keeps the UI alive.
 */
function yieldToPaint(): Promise<void> {
  return new Promise((resolve) => {
    let settled = false;
    const finish = () => {
      if (settled) return;
      settled = true;
      resolve();
    };
    // requestAnimationFrame paints before the next frame, which is what makes
    // the progress bar actually move - but it NEVER fires while the document
    // is hidden: a backgrounded tab, a locked tablet, or the Android app
    // sitting behind its own share sheet. Waiting on it alone deadlocks the
    // export exactly when the user has looked away. A timer races it so the
    // work always continues, and whichever arrives first wins.
    if (typeof requestAnimationFrame === "function") requestAnimationFrame(finish);
    setTimeout(finish, 32);
  });
}

```

```

    });
}

/**
 * Exports the board as a PDF.
 *
 * A notebook gets ONE PDF PAGE PER SHEET, each at the sheet's own size - the
 * thing you print or hand in. A whiteboard is a single page sized to the
 * drawing.
 *
 * Pages carry a raster image rather than vector art. Vector would come from the
 * SVG, but the highlighter pens rely on `mix-blend-mode`, which SVG-to-PDF
 * converters do not reproduce faithfully. SVG export already covers the vector
 * case, so PDF optimises for looking exactly like the board. Each page is 1:1
 * in points while its image is `scale` times denser, so it stays sharp printed.
 */
export async function exportBoardToPdfWithReport(
  board: Board,
  options: ExportOptions = {},
  customLinkTypes: Array<{ id: string; name: string; color: string }> = []
): Promise<{ blob: Blob; report: ExportReport }> {
  // A transparent page is meaningless here: a PDF viewer paints it over its
  // own background, so dark ink can land on a dark page and vanish.
  const background = options.background === "transparent" ? "white" : options.background ?? "white";
  const scale = options.scale ?? 2;
  const opts: ExportOptions = { ...options, background };
  const { images, backdrops } = await resolveExportAssets(board, opts);
  const { jsPDF } = await import("jspdf");

  // Each entry is one PDF page: the SVG to rasterise and its size in points.
  const sheets: Array<{ svg: string; width: number; height: number }> = [];
  const report: ExportReport = { missingImages: 0, missingPdfPages: 0, pages: 0 };

  if (isNotebook(board)) {
    for (const page of exportPages(board, opts)) {
      // Build each sheet on its own, as a one-page notebook, so it is exactly
      // the paper with nothing of its neighbours.
      const single: Board = { ...board, pages: [page], activePageId: page.id };
      const built = buildExportSvg(single, { ...opts, pages: "current" }, customLinkTypes, images, backdrops);
      sheets.push({ svg: built.svg, width: built.width, height: built.height });
      report.missingImages += built.report.missingImages;
      report.missingPdfPages += built.report.missingPdfPages;
    }
  } else {
    const built = buildExportSvg(board, opts, customLinkTypes, images, backdrops);
    sheets.push({ svg: built.svg, width: built.width, height: built.height });
    report.missingImages += built.report.missingImages;
  }
  report.pages = sheets.length;
  if (sheets.length === 0) throw new Error("There is nothing to export - the notebook has no pages.");

  const report_ = options.onProgress;
  let doc: InstanceType<typeof jsPDF> | null = null;
  let sheetNo = 0;
  for (const sheet of sheets) {
    sheetNo++;
    report_?.({
      done: sheetNo - 1,
      total: sheets.length,
      label: sheets.length > 1 ? `Rendering page ${sheetNo} of ${sheets.length}` : "Rendering",
    });
    await yieldToPaint();
    const png = await rasterizeSvg(sheet.svg, sheet.width, sheet.height, scale, background);
    const dataUrl = await blobToDataUrl(png);

    let pageW = sheet.width;
    let pageH = sheet.height;
    const overflow = Math.max(pageW, pageH) / PDF_MAX_POINTS;
    if (overflow > 1) {
      pageW /= overflow;
      pageH /= overflow;
    }
    // jsPDF reorders `format` to match the orientation, so the two must agree
    // or the page comes out with its dimensions swapped.
    const orientation = pageW >= pageH ? "landscape" : "portrait";
    if (!doc) {
      doc = new jsPDF({ orientation, unit: "pt", format: [pageW, pageH], compress: true });
    } else {

```

```

        doc.addPage([pageW, pageH], orientation);
    }
    doc.addImage(dataUrl, "PNG", 0, 0, pageW, pageH, undefined, "FAST");
}
report_?({ done: sheets.length, total: sheets.length, label: "Writing the PDF" });
await yieldToPaint();
return { blob: doc!.output("blob"), report };
}

export async function exportBoardToPdf(
    board: Board,
    options: ExportOptions = {},
    customLinkTypes: Array<{ id: string; name: string; color: string }> = []
): Promise<Blob> {
    return (await exportBoardToPdfWithReport(board, options, customLinkTypes)).blob;
}

/** Exports and saves a PDF (share sheet on Android, download elsewhere). */
export async function downloadPdf(
    board: Board,
    options: ExportOptions = {},
    filename?: string,
    customLinkTypes: Array<{ id: string; name: string; color: string }> = []
): Promise<ExportReport> {
    const { blob, report } = await exportBoardToPdfWithReport(board, options, customLinkTypes);
    const safeName = (filename || board.name || "whiteboard").replace(/^[a-z0-9-]/gi, "_");
    await saveBlob(blob, `${safeName}.pdf`);
    return report;
}

/**
 * Copies high-resolution PNG image directly to system clipboard.
 */
export async function copyPngToClipboard(
    board: Board,
    options: ExportOptions = {},
    customLinkTypes: Array<{ id: string; name: string; color: string }> = []
): Promise<boolean> {
    try {
        const blob = await exportBoardToPng(board, options, customLinkTypes);
        if (typeof navigator !== "undefined" && navigator.clipboard && typeof ClipboardItem !== "undefined") {
            await navigator.clipboard.write([new ClipboardItem({ "image/png": blob })]);
            return true;
        }
        return false;
    } catch (err) {
        console.error("Failed to copy image to clipboard:", err);
        return false;
    }
}

=====
// === END FILE: src/lib/whiteboard/export.ts ===
=====

=====
// === START FILE: src/lib/whiteboard/fflate.d.ts ===
// File 98 of 194 | Lines: 10 | Size: 439 bytes
=====

/**
 * Minimal declarations for the fflate functions ProNotes uses.
 * fflate ships no bundled types; this covers zip-building only.
 */
declare module "fflate" {
    export function zipSync(data: Record<string, Uint8Array>, options?: unknown): Uint8Array;
    export function unzipSync(data: Uint8Array): Record<string, Uint8Array>;
    export function strToU8(str: string): Uint8Array;
    export function strFromU8(data: Uint8Array): string;
}

=====
// === END FILE: src/lib/whiteboard/fflate.d.ts ===
=====

=====

```

```
// === START FILE: src/lib/whiteboard/gallery.ts ===
// File 99 of 194 | Lines: 53 | Size: 2227 bytes
=====

/**
 * "Save to Gallery"
 * =====
 * A screenshot is a photo-shaped thing, and on the tablet people look for it
 * in the Gallery app - not in a Files folder, and not behind a share sheet.
 *
 * Android APK -> the app's own GallerySaver plugin (GallerySaverPlugin.java)
 *                inserts it into MediaStore under Pictures/ProNotes, which
 *                is exactly what Gallery / Google Photos list. The export
 *                path cannot do this: it writes to the app cache for the
 *                share sheet, and nothing there is ever indexed.
 * Web/desktop -> an ordinary download.
 *
 * An APK built before the plugin existed has no GallerySaver; the call then
 * falls back to the share sheet (which offers "Save to device"), and says so.
 */

import { registerPlugin } from "@capacitor/core";
import { blobToDataUrl, downloadBlob, isNativeApp, saveBlob } from "../share";

interface GallerySaverPlugin {
  saveImage(options: { data: string; filename: string; album?: string }): Promise<{ uri: string }>;
}

const GallerySaver = registerPlugin<GallerySaverPlugin>("GallerySaver");

/** The album (Pictures/<album>) screenshots land in. */
export const GALLERY_ALBUM = "ProNotes";

/** Where the picture really went, so the UI can say something true. */
export type GalleryOutcome = "gallery" | "downloaded" | "shared" | "cancelled";

export async function saveImageToGallery(blob: Blob, filename: string): Promise<GalleryOutcome> {
  if (!isNativeApp()) {
    downloadBlob(blob, filename);
    return "downloaded";
  }
  const dataUrl = await blobToDataUrl(blob);
  const data = dataUrl.slice(dataUrl.indexOf(",") + 1);
  try {
    await GallerySaver.saveImage({ data, filename, album: GALLERY_ALBUM });
    return "gallery";
  } catch (err) {
    const e = err as { code?: string; message?: string };
    // Only a MISSING plugin (an older APK) falls back; a real failure, such as
    // a refused permission, is reported rather than papered over.
    if (e?.code === "UNIMPLEMENTED" || /not implemented/i.test(e?.message ?? "")) {
      const outcome = await saveBlob(blob, filename);
      return outcome === "cancelled" ? "cancelled" : "shared";
    }
    throw err;
  }
}

```

```
// === END FILE: src/lib/whiteboard/gallery.ts ===
=====
```

```
// === START FILE: src/lib/whiteboard/geometry.ts ===
// File 100 of 194 | Lines: 1026 | Size: 33044 bytes
=====

/**
 * Whiteboard Geometry & Math Utilities (v3 – Obsidian Canvas & Vector Grade)
 * =====
 *
 * Provides:
 * - World <-> Screen coordinate conversions
 * - Bounding boxes & hit testing for all primitives
 * - Smart Anchor Port calculation for diagram nodes
 * - 3 Connection Routing Engines: Smooth Cubic Bezier, Orthogonal Step, Straight
 * - Directional Arrowhead polygon generators

```

```

* - Viewport culling
*/

import type {
  ArrowElement,
  BoundingBox,
  ConnectionElement,
  DiamondElement,
  EllipseElement,
  ImageElement,
  LineElement,
  NodeAnchor,
  RectangleElement,
  StarElement,
  CloudElement,
  StickyElement,
  StrokeElement,
  TextElement,
  TriangleElement,
  WhiteboardElement,
  ArrowheadType,
} from "./types";

export type { BoundingBox };
export const HIT_THRESHOLD = 8;

export interface Viewport {
  width: number;
  height: number;
}

// =====
// Coordinate Transforms
// =====

export function worldToScreen(
  wx: number,
  wy: number,
  camera: { x: number; y: number; zoom: number },
  viewport: Viewport
): { x: number; y: number } {
  return {
    x: (wx - camera.x) * camera.zoom + viewport.width / 2,
    y: (wy - camera.y) * camera.zoom + viewport.height / 2,
  };
}

export function screenToWorld(
  sx: number,
  sy: number,
  camera: { x: number; y: number; zoom: number },
  viewport: Viewport
): { x: number; y: number } {
  return {
    x: (sx - viewport.width / 2) / camera.zoom + camera.x,
    y: (sy - viewport.height / 2) / camera.zoom + camera.y,
  };
}

/**
 * SVG transform for a group that lives INSIDE the canvas' world group but wants
 * to draw in SCREEN coordinates (the selection overlay does this: it computes
 * handle positions in screen space so handles keep a constant pixel size).
 *
 * The world group is `translate(vw/2, vh/2) scale(z) translate(-cx, -cy)`, and
 * an SVG transform list applies right-to-left, so the exact inverse is the same
 * three operations REVERSED and INVERTED. Listing them in the forward order
 * composes to the identity only at zoom === 1 and otherwise leaves a constant
 * offset of (camera + viewport/2) * (1 - zoom).
 */
export function screenSpaceOverlayTransform(
  camera: { x: number; y: number; zoom: number },
  viewport: Viewport
): string {
  return `translate(${camera.x}, ${camera.y}) scale(${1 / camera.zoom}) translate(${viewport.width / 2}, ${viewport.height / 2})`;
}

```

```

export function worldToScreenDelta(d: number, zoom: number): number {
  return d * zoom;
}

export function screenToWorldDelta(d: number, zoom: number): number {
  return d / zoom;
}

export function rotatePoint(
  px: number,
  py: number,
  cx: number,
  cy: number,
  angleRad: number
): { x: number; y: number } {
  if (angleRad === 0) return { x: px, y: py };
  const cos = Math.cos(angleRad);
  const sin = Math.sin(angleRad);
  const dx = px - cx;
  const dy = py - cy;
  return {
    x: cx + dx * cos - dy * sin,
    y: cy + dx * sin + dy * cos,
  };
}

export function clamp(v: number, min: number, max: number): number {
  return Math.max(min, Math.min(max, v));
}

export function lerp(a: number, b: number, t: number): number {
  return a + (b - a) * t;
}

// =====
// Bounding Boxes & Centers
// =====

export function getElementBounds(el: WhiteboardElement): BoundingBox {
  switch (el.type) {
    case "stroke":
      return getStrokeBounds(el);
    case "connection":
      return { x: el.x, y: el.y, width: 0, height: 0 };
    case "rectangle":
    case "ellipse":
    case "diamond":
    case "triangle":
    case "star":
    case "cloud":
    case "text":
    case "sticky":
    case "image":
      return { x: el.x, y: el.y, width: el.width, height: el.height };
    case "line":
    case "arrow": {
      const minX = Math.min(el.x, el.x + el.x2);
      const minY = Math.min(el.y, el.y + el.y2);
      const maxX = Math.max(el.x, el.x + el.x2);
      const maxY = Math.max(el.y, el.y + el.y2);
      return { x: minX, y: minY, width: Math.max(maxX - minX, 2), height: Math.max(maxY - minY, 2) };
    }
    default: {
      // Every known element type is handled above, so this is only reached by
      // data this build does not understand: a corrupt save, or a type added
      // by a newer version. With a usable position it gets a small box there.
      // Without one it gets a NON-FINITE box, which getUnionBounds skips - the
      // old `?? 0` placed it at 0,0, dragging union bounds (and so export and
      // zoom-to-fit) all the way back to the origin.
      const unknown = el as { x?: unknown; y?: unknown };
      const x = typeof unknown.x === "number" && Number.isFinite(unknown.x) ? unknown.x : NaN;
      const y = typeof unknown.y === "number" && Number.isFinite(unknown.y) ? unknown.y : NaN;
      return { x, y, width: 10, height: 10 };
    }
  }
}

function getStrokeBounds(el: StrokeElement): BoundingBox {

```

```

    if (!el.points || el.points.length === 0) {
      return { x: el.x, y: el.y, width: 0, height: 0 };
    }
    let minX = Infinity;
    let minY = Infinity;
    let maxX = -Infinity;
    let maxY = -Infinity;
    for (const [px, py] of el.points) {
      if (px < minX) minX = px;
      if (py < minY) minY = py;
      if (px > maxX) maxX = px;
      if (py > maxY) maxY = py;
    }
    const pad = Math.max((el.strokeWidth || 3) * 1.5, 4);
    return {
      x: el.x + minX - pad,
      y: el.y + minY - pad,
      width: Math.max(maxX - minX + pad * 2, pad * 2),
      height: Math.max(maxY - minY + pad * 2, pad * 2),
    };
  };
}

export function getElementCenter(el: WhiteboardElement): { x: number; y: number } {
  if (el.type === "line" || el.type === "arrow") {
    return { x: el.x + el.x2 / 2, y: el.y + el.y2 / 2 };
  }
  const b = getElementBounds(el);
  return { x: b.x + b.width / 2, y: b.y + b.height / 2 };
}

export function getUnionBounds(elements: WhiteboardElement[]): BoundingBox | null {
  if (elements.length === 0) return null;
  let minX = Infinity;
  let minY = Infinity;
  let maxX = -Infinity;
  let maxY = -Infinity;
  for (const el of elements) {
    if (el.type === "connection") continue;
    const b = getElementBounds(el);
    // An element with no usable position (unrecognised or corrupt data) has a
    // non-finite box; including it would make the whole union NaN or pin it
    // to the origin.
    if (!Number.isFinite(b.x) || !Number.isFinite(b.y)) continue;
    if (b.x < minX) minX = b.x;
    if (b.y < minY) minY = b.y;
    if (b.x + b.width > maxX) maxX = b.x + b.width;
    if (b.y + b.height > maxY) maxY = b.y + b.height;
  }
  if (!isFinite(minX)) return null;
  return { x: minX, y: minY, width: maxX - minX, height: maxY - minY };
}

// =====
// Smart Anchor Ports for Diagram Nodes (Obsidian Canvas Grade)
// =====

export interface AnchorPoint {
  x: number;
  y: number;
  nx: number; // outward normal X (-1 to 1)
  ny: number; // outward normal Y (-1 to 1)
  anchor: NodeAnchor;
}

export function getNodeAnchorPorts(el: WhiteboardElement): Record<"top" | "right" | "bottom" | "left", AnchorPoint> {
  const b = getElementBounds(el);
  const cx = b.x + b.width / 2;
  const cy = b.y + b.height / 2;
  const rot = el.rotation || 0;

  const topUnrot = { x: cx, y: b.y };
  const rightUnrot = { x: b.x + b.width, y: cy };
  const bottomUnrot = { x: cx, y: b.y + b.height };
  const leftUnrot = { x: b.x, y: cy };

  const top = rotatePoint(topUnrot.x, topUnrot.y, cx, cy, rot);
  const right = rotatePoint(rightUnrot.x, rightUnrot.y, cx, cy, rot);

```

```

const bottom = rotatePoint(bottomUnrot.x, bottomUnrot.y, cx, cy, rot);
const left = rotatePoint(leftUnrot.x, leftUnrot.y, cx, cy, rot);

const cos = Math.cos(rot);
const sin = Math.sin(rot);

return {
  top: { x: top.x, y: top.y, nx: sin, ny: -cos, anchor: "top" },
  right: { x: right.x, y: right.y, nx: cos, ny: sin, anchor: "right" },
  bottom: { x: bottom.x, y: bottom.y, nx: -sin, ny: cos, anchor: "bottom" },
  left: { x: left.x, y: left.y, nx: -cos, ny: -sin, anchor: "left" },
};
}

export function getResolvedAnchorPoint(
  el: WhiteboardElement,
  anchor: NodeAnchor,
  targetPos?: { x: number; y: number }
): AnchorPoint {
  const ports = getNodeAnchorPorts(el);
  const b = getElementBounds(el);
  const cx = b.x + b.width / 2;
  const cy = b.y + b.height / 2;

  if (anchor === "center") {
    return { x: cx, y: cy, nx: 0, ny: 0, anchor: "center" };
  }

  if (anchor === "auto" || !anchor) {
    if (!targetPos) return ports.right;
    // Pick the closest port to targetPos
    const dTop = Math.hypot(ports.top.x - targetPos.x, ports.top.y - targetPos.y);
    const dRight = Math.hypot(ports.right.x - targetPos.x, ports.right.y - targetPos.y);
    const dBottom = Math.hypot(ports.bottom.x - targetPos.x, ports.bottom.y - targetPos.y);
    const dLeft = Math.hypot(ports.left.x - targetPos.x, ports.left.y - targetPos.y);
    const min = Math.min(dTop, dRight, dBottom, dLeft);
    if (min === dTop) return ports.top;
    if (min === dRight) return ports.right;
    if (min === dBottom) return ports.bottom;
    return ports.left;
  }

  return ports[anchor as "top" | "right" | "bottom" | "left"] || ports.right;
}

export function getOptimalConnectionAnchors(
  sourceEl: WhiteboardElement,
  targetEl: WhiteboardElement,
  preferredSourceAnchor: NodeAnchor = "auto",
  preferredTargetAnchor: NodeAnchor = "auto"
): { source: AnchorPoint; target: AnchorPoint } {
  const sCenter = getElementCenter(sourceEl);
  const tCenter = getElementCenter(targetEl);

  const sAnchor = getResolvedAnchorPoint(sourceEl, preferredSourceAnchor, tCenter);
  const tAnchor = getResolvedAnchorPoint(targetEl, preferredTargetAnchor, sAnchor);

  return { source: sAnchor, target: tAnchor };
}

// =====
// 3 Connection Routing Engines (Obsidian / Heptabase / Flow Grade)
// =====

/**
 * 1. Smooth Cubic Bezier S-Curve with Tangent Normals
 */
export function getCurvedBezierPath(
  p1: AnchorPoint,
  p2: AnchorPoint
): string {
  const dx = p2.x - p1.x;
  const dy = p2.y - p1.y;
  const dist = Math.hypot(dx, dy);

  if (dist < 2) {
    return `M ${p1.x},${p1.y} L ${p2.x},${p2.y}`;
  }
}

```